INSTITUTO FEDERAL
Goiás

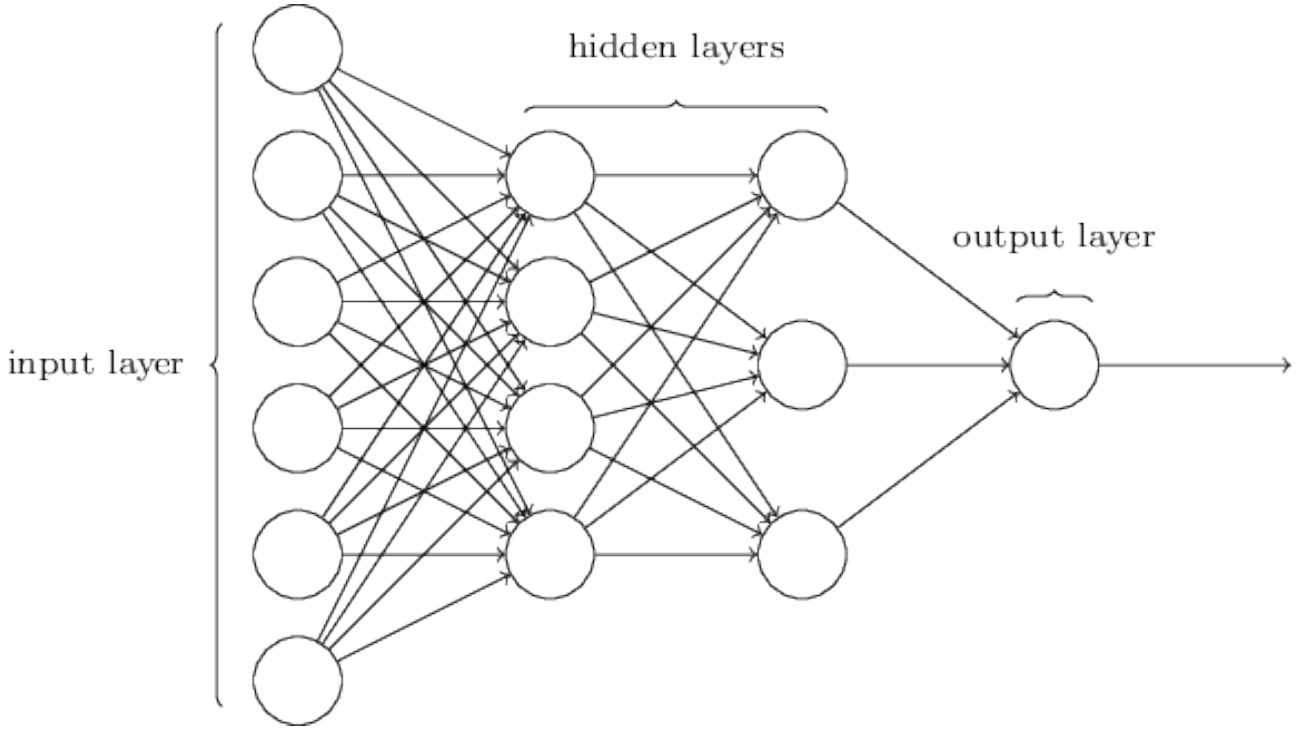
Redes de Convolução

Prof. Sérgio Canuto - sergio.canuto@ifg.edu.br

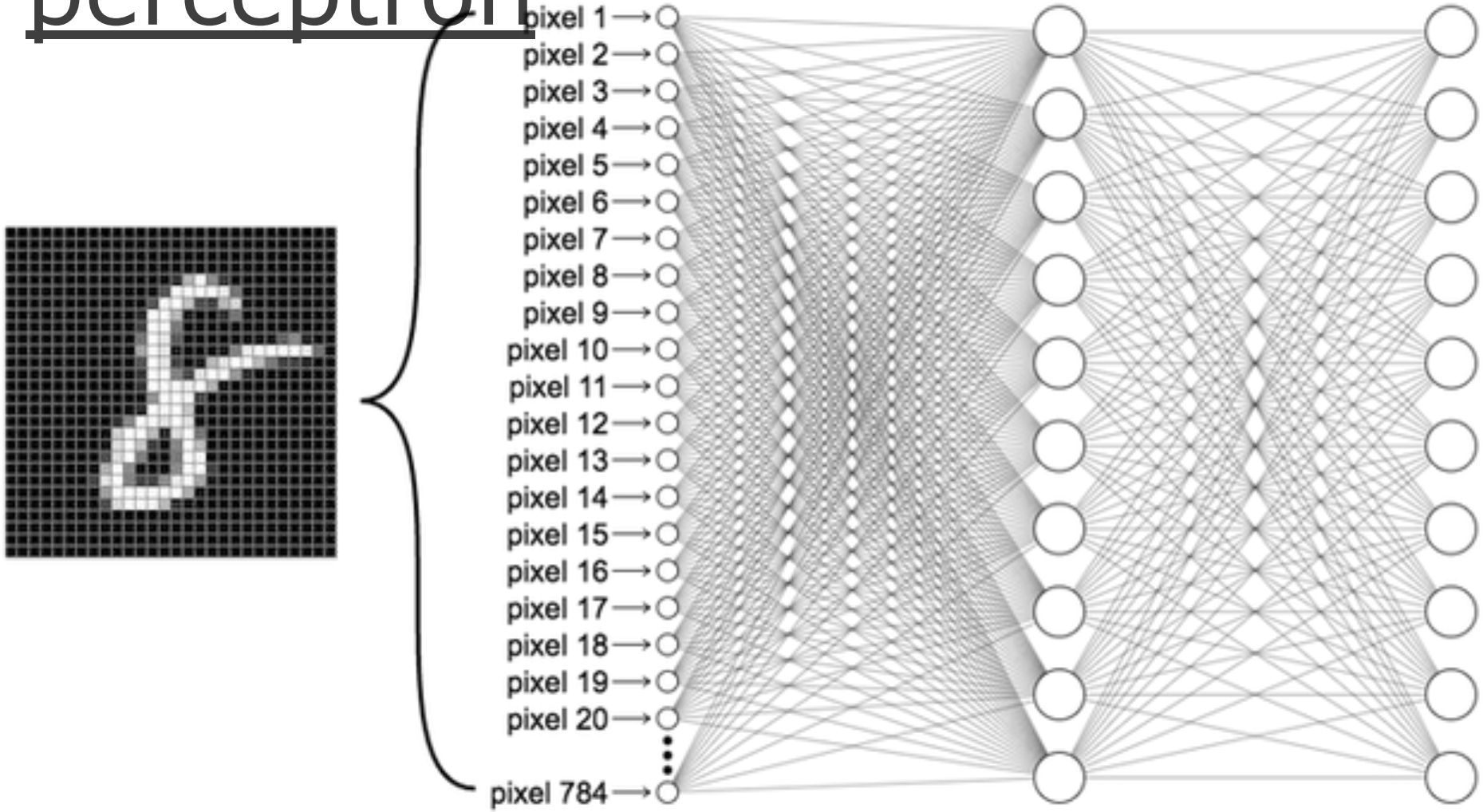
Visão Geral

- Contextualização/exemplos
- Convolução
- Camadas de convolução
- Filtro/kernel de convolução
- Mapa de ativação
- Pooling
- Camada de saída

Recapitulando: arquitetura ML perceptron

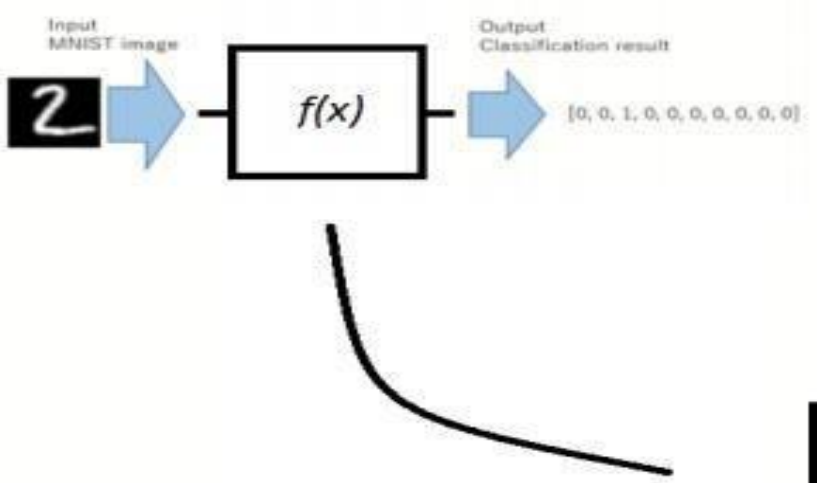


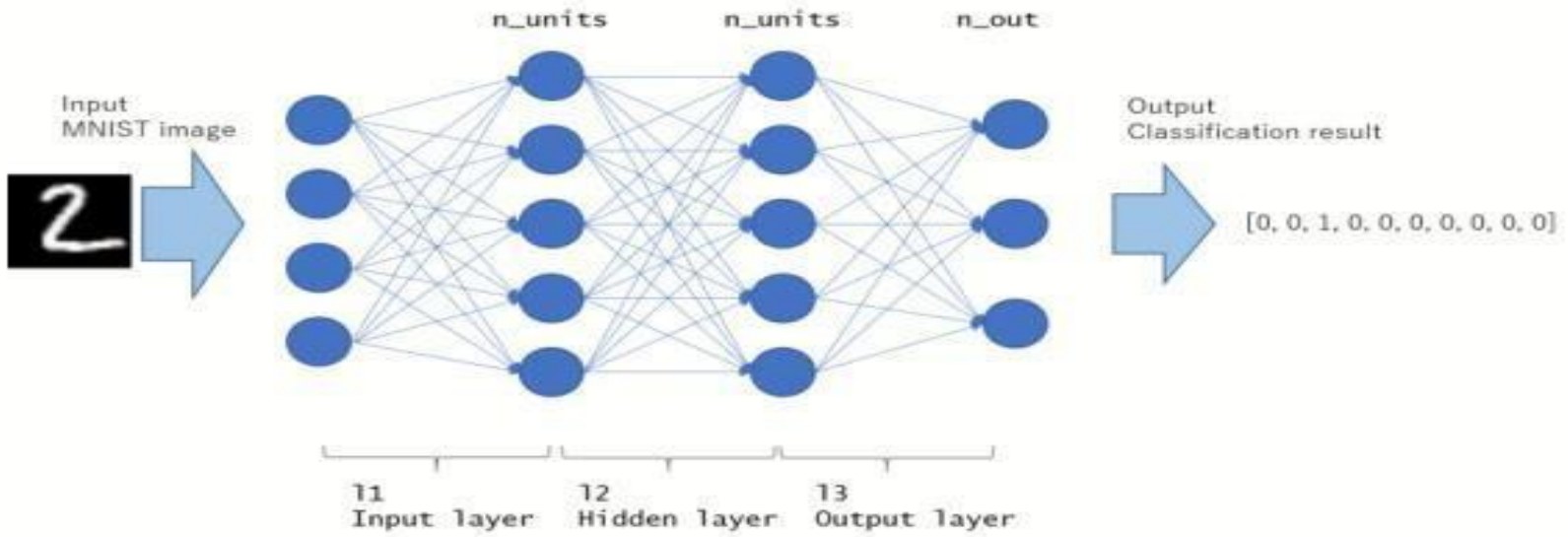
Recapitulando: arquitetura ML perceptron



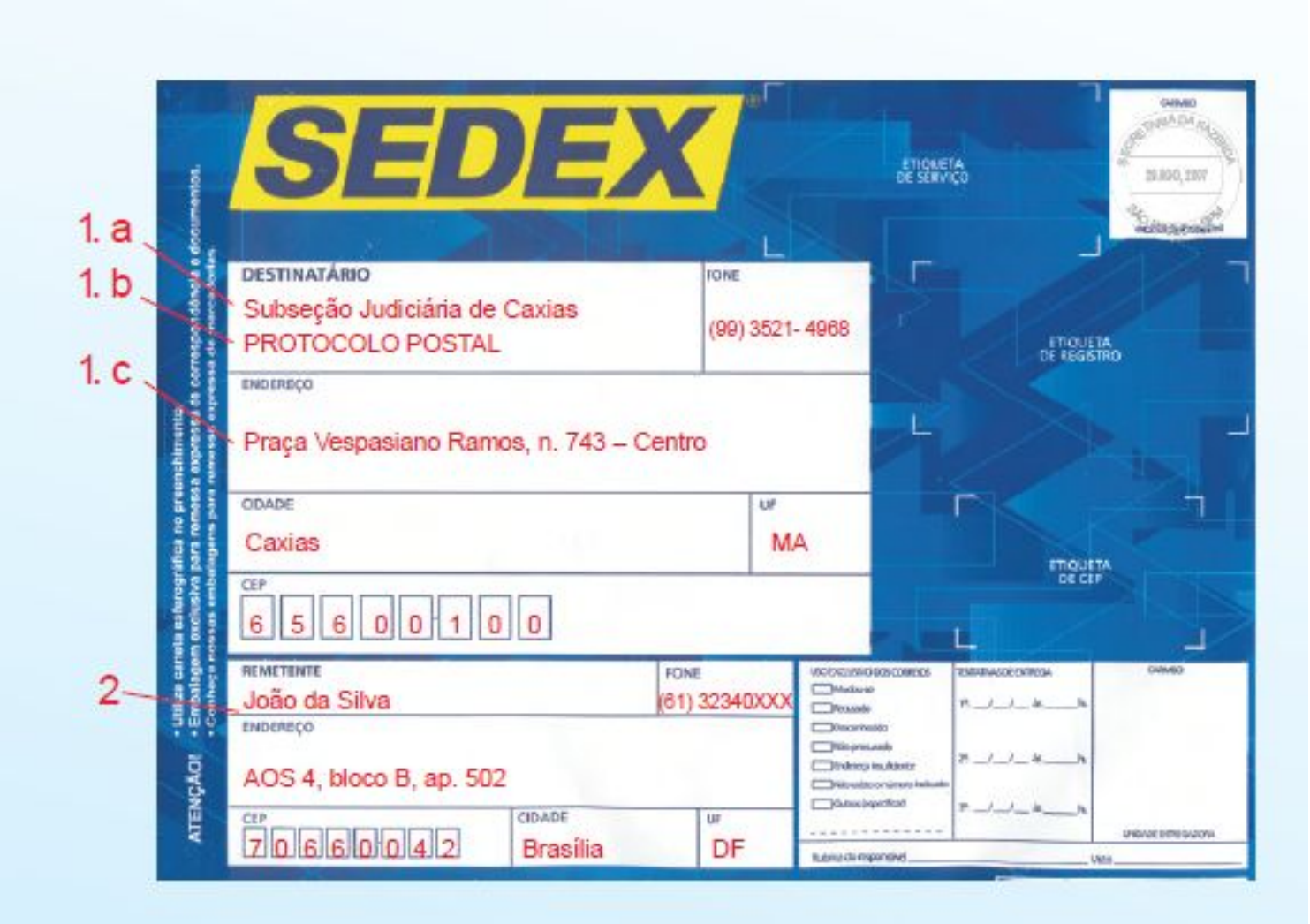
Recapitulando: arquitetura ML perceptron

- MLP





Contexto


SEDEX
1. a
1. b
1. c
2
DESTINATÁRIO
Subseção Judiciária de Caxias
PROTOCOLO POSTAL
FONE
(99) 3521- 4968
ENDEREÇO
Praça Vespasiano Ramos, n. 743 – Centro
CIDADE
Caxias
UF
MA
CEP
6 5 6 0 0 1 0 0
REMETENTE
João da Silva
FONE
(61) 32340XXX
ENDEREÇO
AOS 4, bloco B, ap. 502
CEP
7 0 6 6 0 0 4 2
CIDADE
Brasília
UF
DF
ETIQUETA DE SERVIÇO
ETIQUETA DE REGISTRO
ETIQUETA DE CEP
CARIMBO
ATENÇÃO!

Contexto

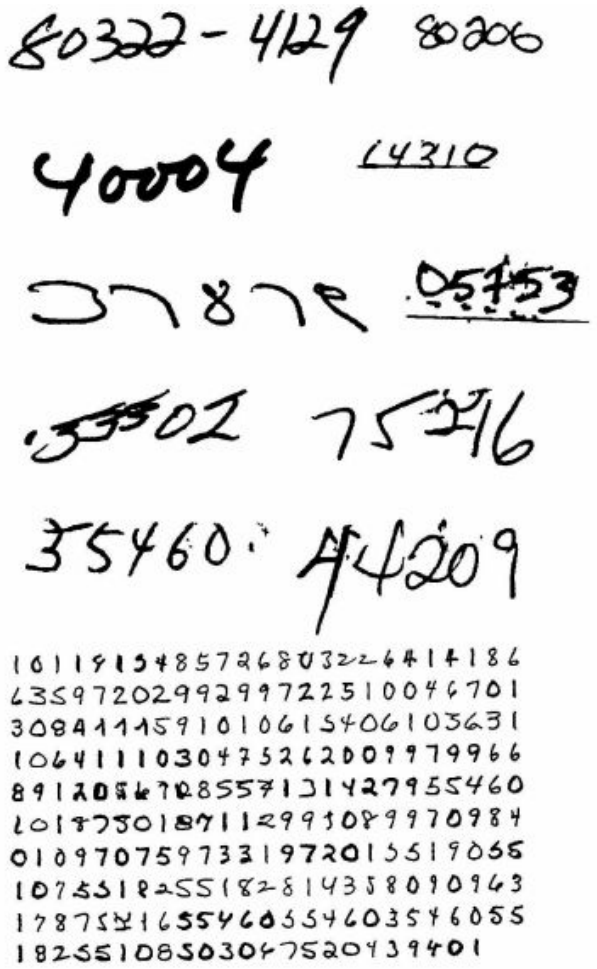


Figure 1 Examples of original zip codes (top) and normalized digits from the testing set (bottom).

Backpropagation Applied to Handwritten Zip Code Recognition

Y. LeCun
B. Boser
J. S. Denker
D. Henderson
R. E. Howard
W. Hubbard
L. D. Jackel
AT&T Bell Laboratories Holmdel, NJ 07733 USA

The ability of learning networks to generalize can be greatly enhanced by providing constraints from the task domain. This paper demonstrates how such constraints can be integrated into a backpropagation network through the architecture of the network. This approach has been successfully applied to the recognition of handwritten zip code digits provided by the U.S. Postal Service. A single network learns the entire recognition operation, going from the normalized image of the character to the final classification.

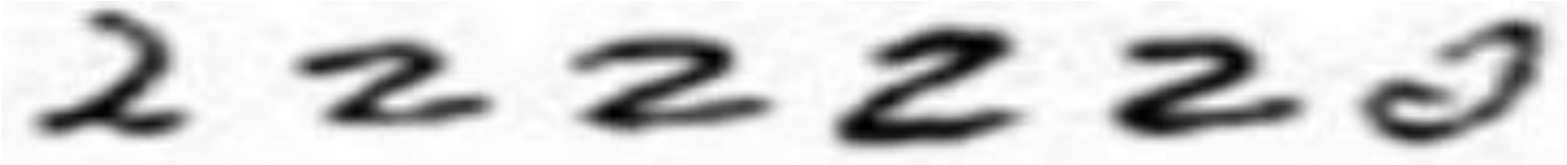
1 Introduction

Previous work performed on recognizing simple digit images (LeCun 1989) showed that good generalization on complex tasks can be obtained by designing a network architecture that contains a certain amount of a priori knowledge about the task. The basic design principle is to reduce the number of free parameters in the network as much as possible without overly reducing its computational power. Application of this principle increases the probability of correct generalization because it results in a specialized network architecture that has a reduced entropy (Denker *et al* 1987; Patarnello and Carnevali 1987, Tishby *et al*. 1989; LeCun 1989), and a reduced Vapnik–Chervonenkis dimensionality (Baum and Haussler 1989).

In this paper, we apply the backpropagation algorithm (Rumelhart *et al* 1986) to a real-world problem in recognizing handwritten digits taken from the U S Mail. Unlike previous results reported by our group on this problem (Denker *et al* 1989), the learning network is directly fed with images, rather than feature vectors, thus demonstrating the ability of backpropagation networks to deal with large amounts of low-level information.

Neural Computation **1, 541–551** (1989) © 1989 Massachusetts Institute of Technology

Contexto

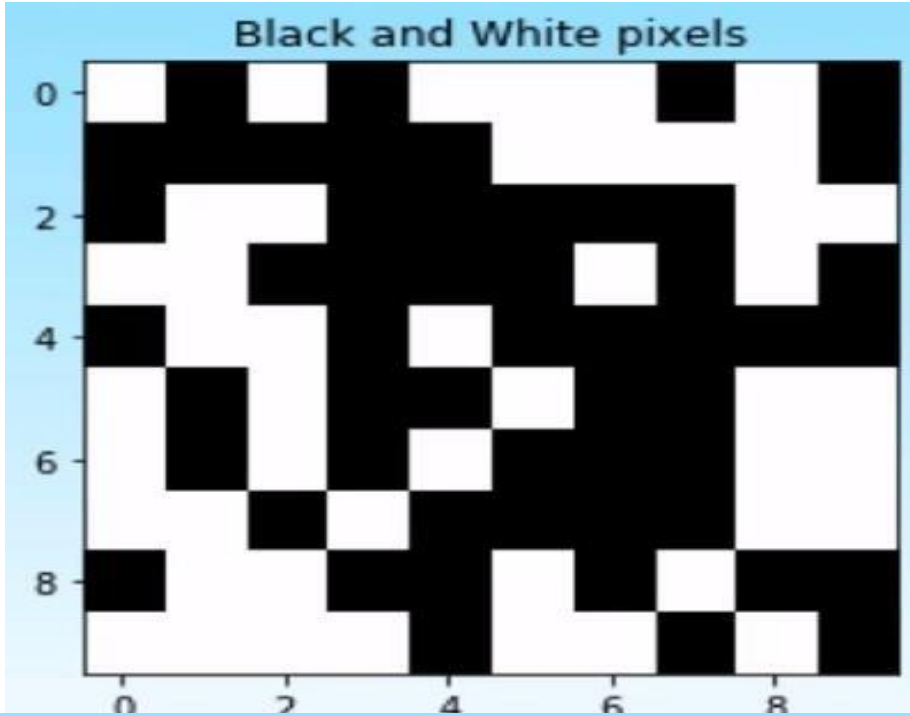


MNIST

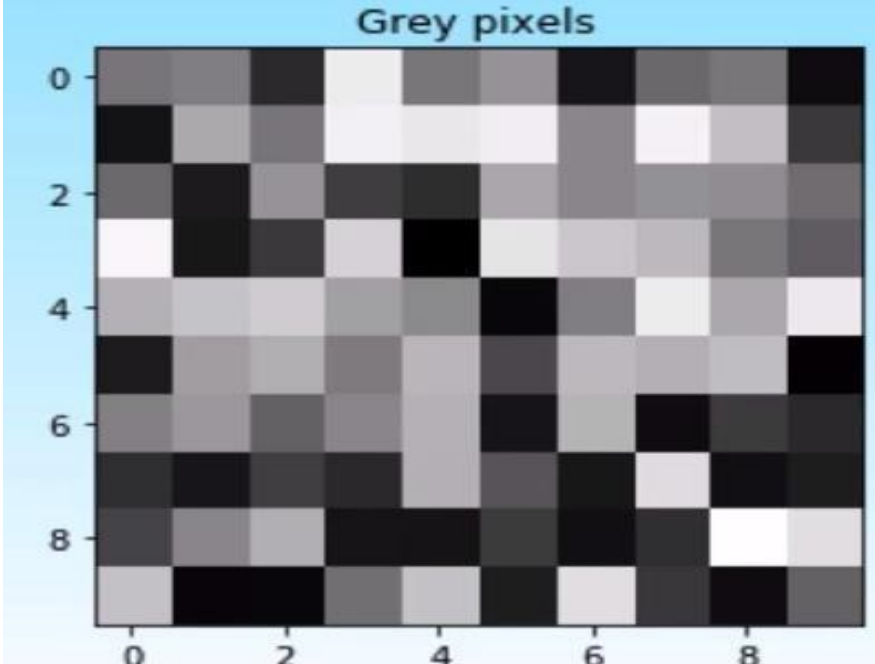


Mixed National Institute of Standards and Technology (MNIST)

- Dataset para o problema de reconhecimento de dígitos escritos a mão
- 60,000 imagens de treino, 10,000 de teste, codificados como imagens 28×28 em escala de cinza
- Como representar a informação?
- Quantas formas temos para escrever?




```
array([[1, 0, 1, 0, 1, 1, 1, 0, 1, 0],
       [0, 0, 0, 0, 0, 1, 1, 1, 1, 0],
       [0, 1, 1, 0, 0, 0, 0, 0, 1, 1],
       [1, 1, 0, 0, 0, 0, 1, 0, 1, 0],
       [0, 1, 1, 0, 1, 0, 0, 0, 0, 0],
       [1, 0, 1, 0, 0, 1, 0, 0, 1, 1],
       [1, 0, 1, 0, 1, 0, 0, 0, 1, 1],
       [1, 1, 0, 1, 0, 0, 0, 0, 1, 1],
       [0, 1, 1, 0, 0, 1, 0, 1, 0, 0],
       [1, 1, 1, 1, 0, 1, 1, 0, 1, 0]])
```




```
array([[127, 136,  48, 234, 128, 155,  24, 114, 128,  12],
       [ 18, 177, 127, 239, 230, 236, 145, 240, 195,  65],
       [116,  29, 155,  70,  51, 174, 144, 153, 149, 120],
       [243,  26,  66, 213,   1, 227, 203, 191, 128, 105],
       [183, 199, 207, 167, 146,   7, 136, 234, 175, 232],
       [ 31, 166, 180, 133, 187,  82, 192, 182, 194,   2],
       [139, 161, 108, 144, 183,  23, 185,  12,  67,  47],
       [ 53,  24,  72,  46, 182,  94,  25, 221,  16,  34],
       [ 76, 144, 181,  22,  21,  67,  16,  53, 250, 222],
       [197,   6,   7, 122, 198,  34, 222,  64,  12, 108]])
```

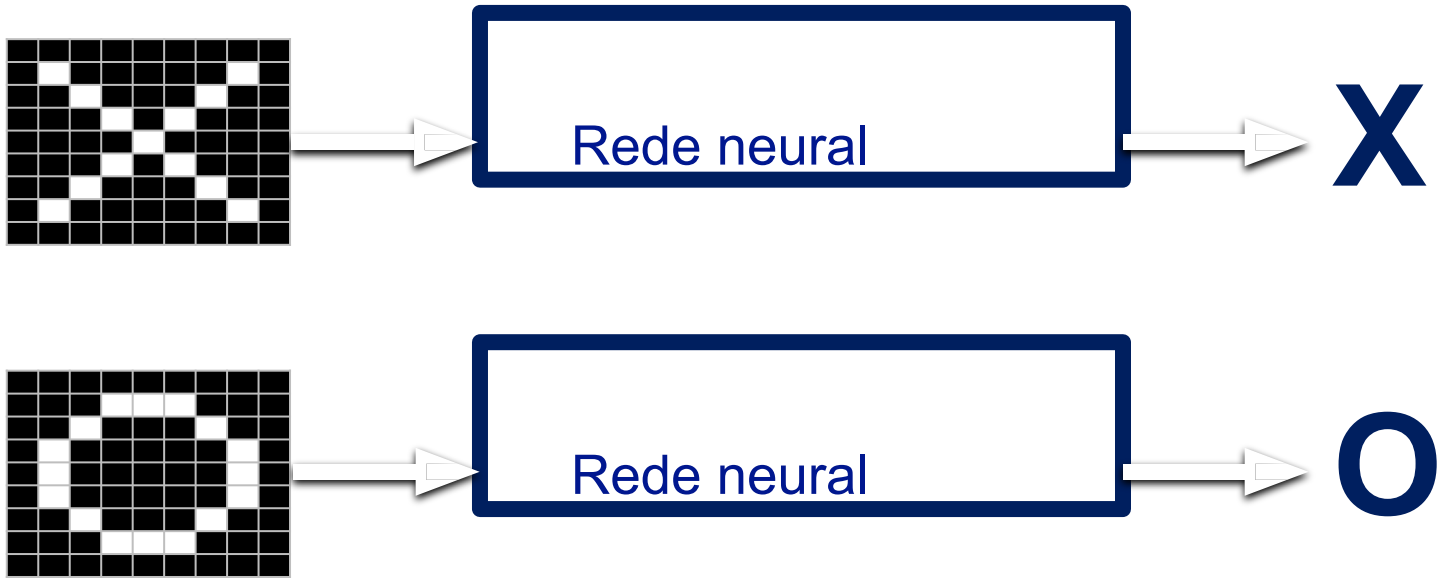
Exemplo

Decidir se uma figura contém X ou O

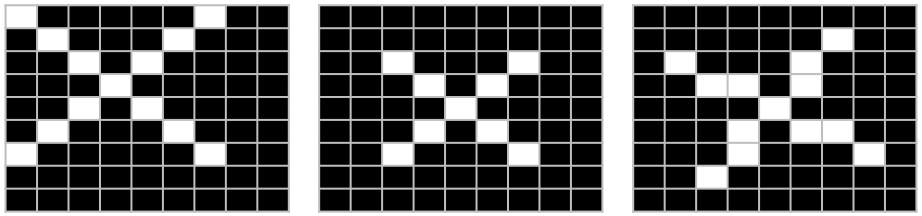
Matriz de pixeis



Exemplo

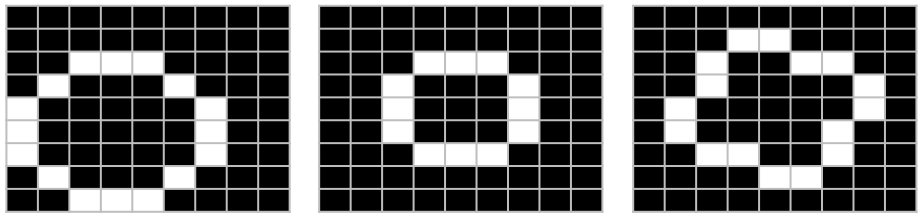


Casos mais complicados



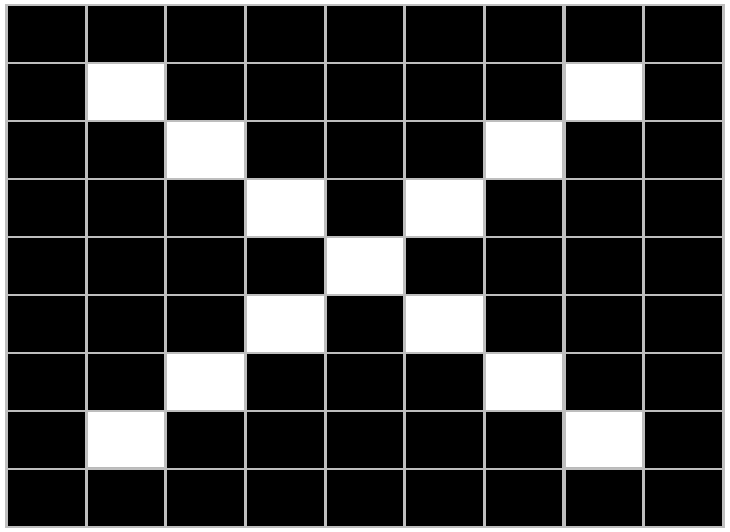
Translação escala rotação





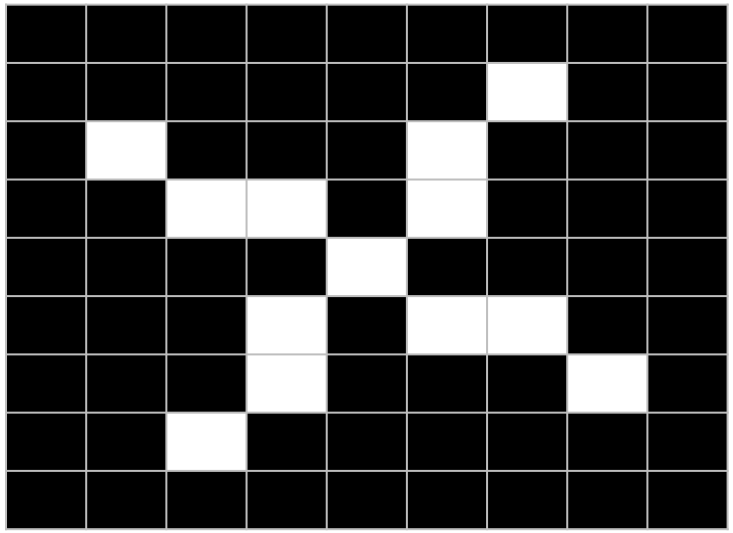


Decisão difícil



=

?



O que a rede “enxerga”?

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

=

?

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	1	-1	-1	-1
-1	-1	1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	1	-1	-1
-1	-1	-1	1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

O que a rede “enxerga”?

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	X	-1	-1	-1	-1	X	X	-1
-1	X	X	-1	-1	X	X	-1	-1
-1	-1	X	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	X	-1	-1
-1	-1	X	X	-1	-1	X	X	-1
-1	X	X	-1	-1	-1	-1	X	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

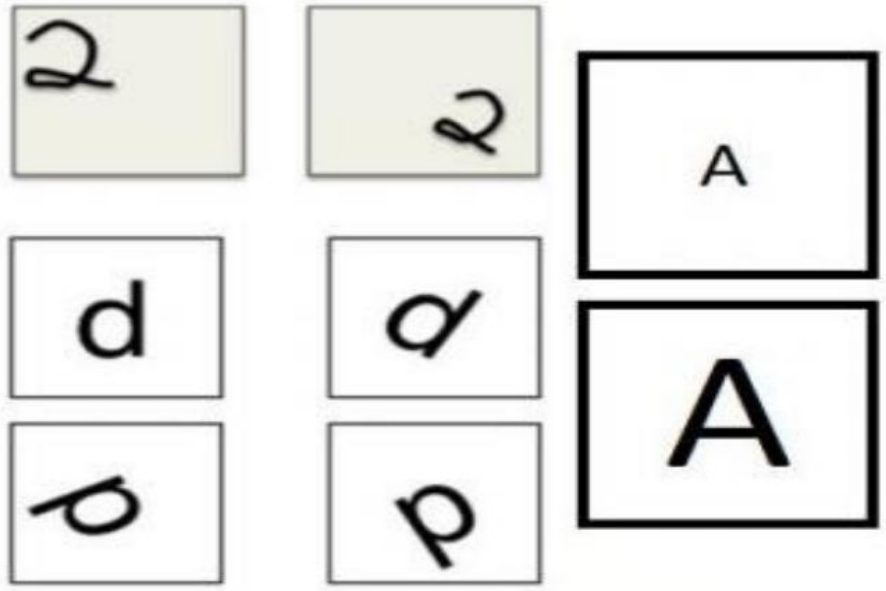
O que a rede “enxerga”?

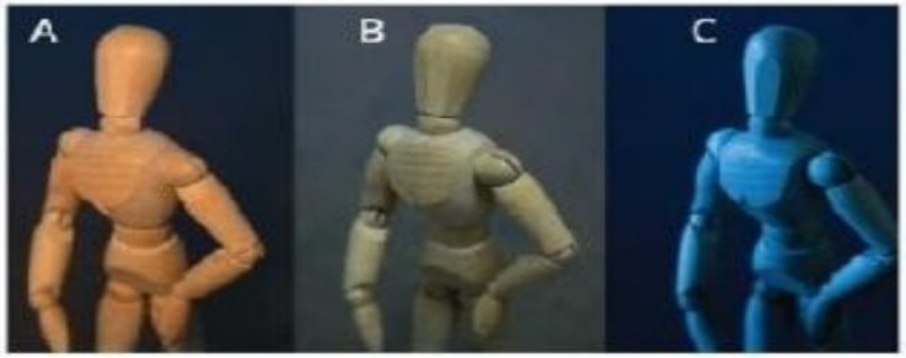
-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1



-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	1	-1	-1	-1
-1	-1	1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	1	-1	-1
-1	-1	-1	1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

olá - oláaaaa
bemyguest - be my guest
schwarzenegger - schwazeneger





MNIST

	Test Error Rate (%)	References
Linear classifier (1-layer neural net)	12.0	LeCun et al. (1998)
K-nearest-neighbors, Euclidean (L2)	5.0	LeCun et al. (1998)
2-Layer neural net, 300 hidden units, mean square error	4.7	LeCun et al. (1998)
Support vector machine, Gaussian kernel	1.4	MNIST Website
Convolutional net, LeNet-5 (no distortions)	**0.95**	LeCun et al. (1998)

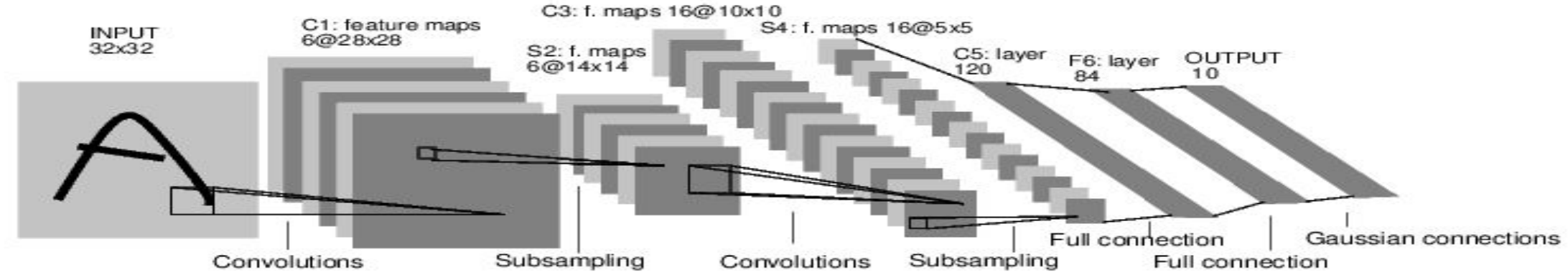
Y. LeCun, et al. Gradient-Based Learning Applied to Document Recognition. IEEE. 1998



Yann LeCun Professor of Computer Science
The Courant Institute of Mathematical Sciences
New York University
Room 1220, 715 Broadway, New York, NY 10003, USA.
(212)998-3283yann@cs.nyu.edu

Yann LeCun e Yoshua Bengio Introduziram o conceito de redes convolucionais (1995).

-> Ganharam o “nobel” da computação por essa e outras contribuições! **https://www.acm.org/media-center/2019/march/turing-award-2018**
-> Considerados os “pais” do deep learning!



Rede Neural Convolucional

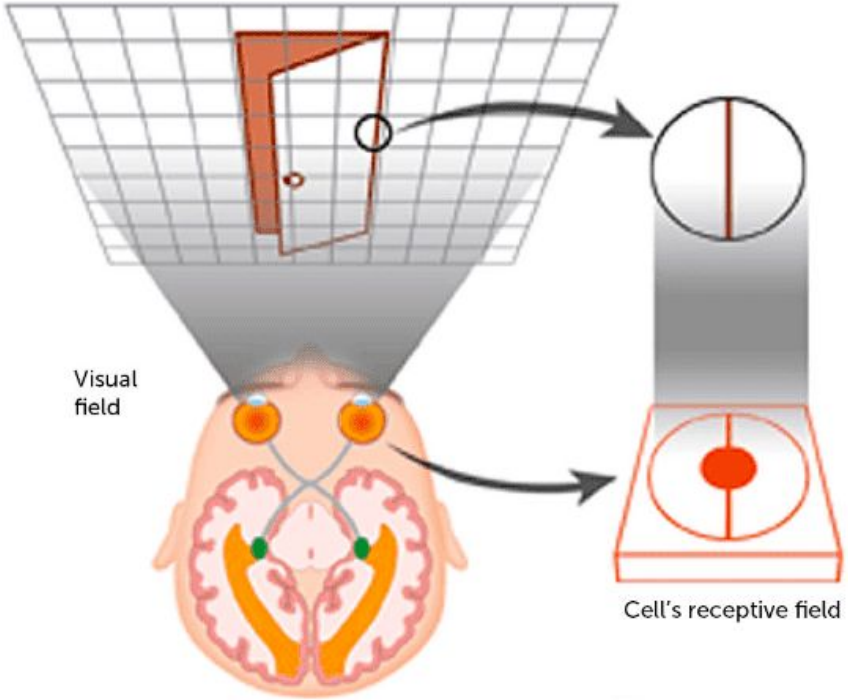
- CNN's foram inspiradas pela sensibilidade local e orientação seletiva do cérebro.

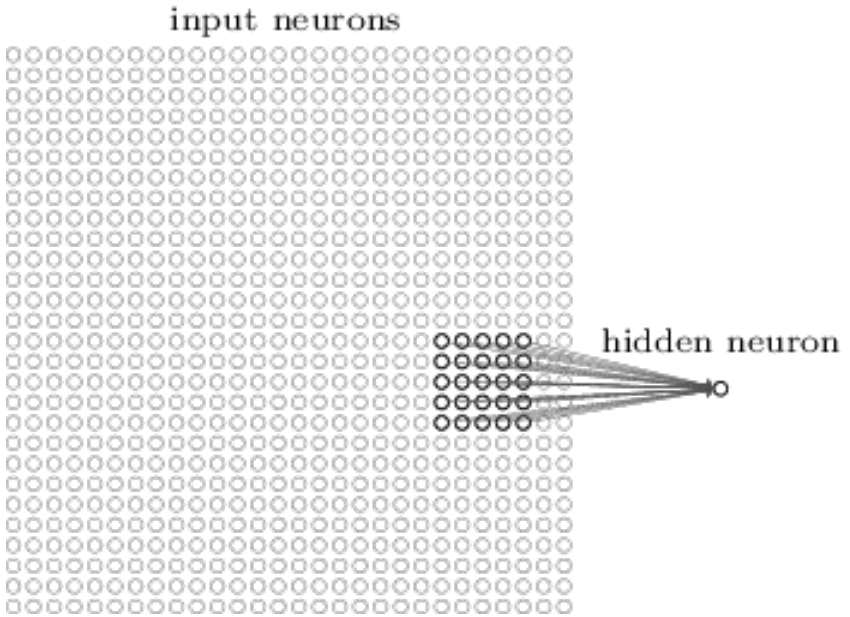
- Projetaram uma rede neural que implicitamente extraiam características relevantes da entrada.
  - Em uma imagem, o que tem de característica relevante?

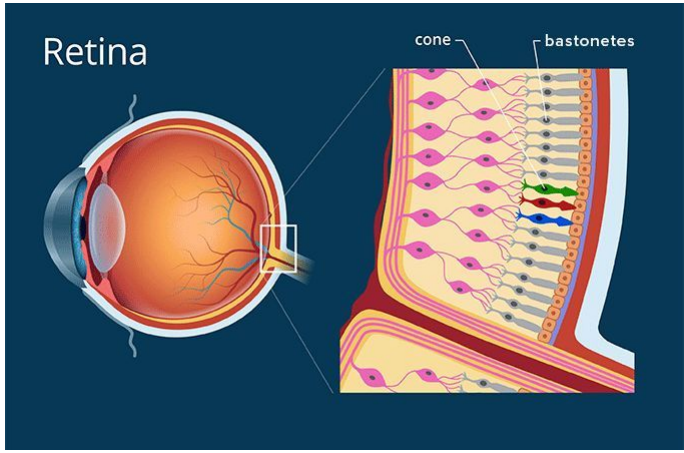
- Redes convolucionais são um tipo especial de multi-layer Perceptrons (MLP).

Rede Neural Convolucional

- Problema: Entrada pode ter uma dimensão muito alta. Multi-Layer Perceptron.
- Inspirado pelos experimentos de neurofisiologia [Hubel & Wiesel 1962], CNNs são um tipo especial de redes neurais que usam o conceito de campo receptivo local.







Fonte: http://neuralnetworksanddeeplearning.com/chap6.html

Rede Neural Convolucional

- Problema: Entrada pode ter uma dimensão muito alta. Multi-Layer Perceptron.
- Inspirado pelos experimentos de neurofisiologia [Hubel & Wiesel 1962], CNNs são um tipo especial de redes neurais que usam o conceito de campo receptivo local.

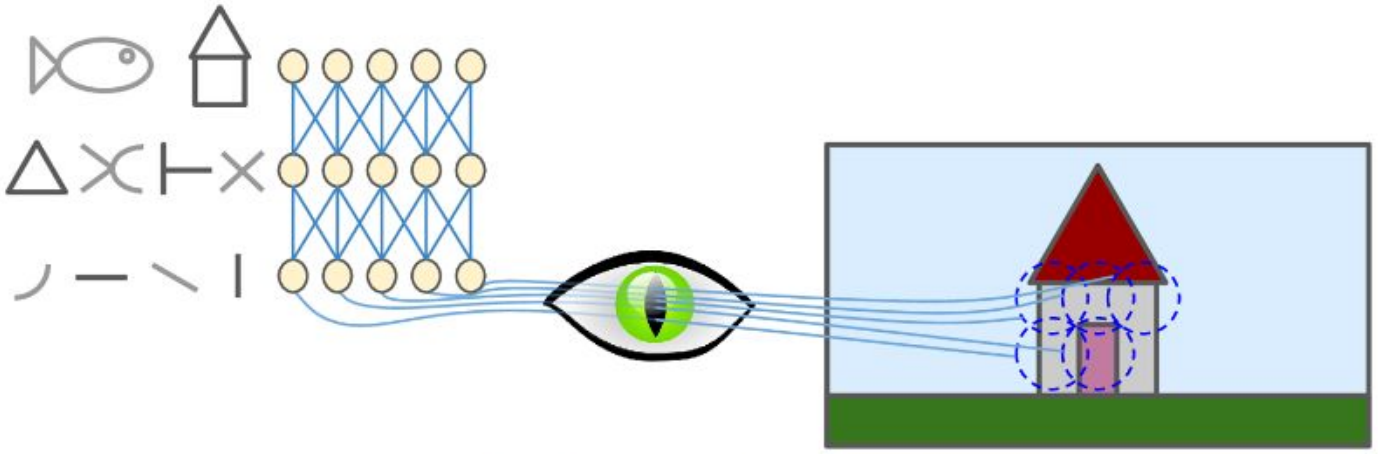


Figure 3-1. Local receptive fields in the visual cortex

Exemplo: 200x200 imagem

- MLP: 40,000 na camada de entrada.
  + 300 hidden units =>
  12 milhões de parâmetros
- CNN: 5x5 kernel, 100 campos receptivos => 2,500 parametros

Fonte: Aurélien Géron. Neural networks and deep learning , cap3

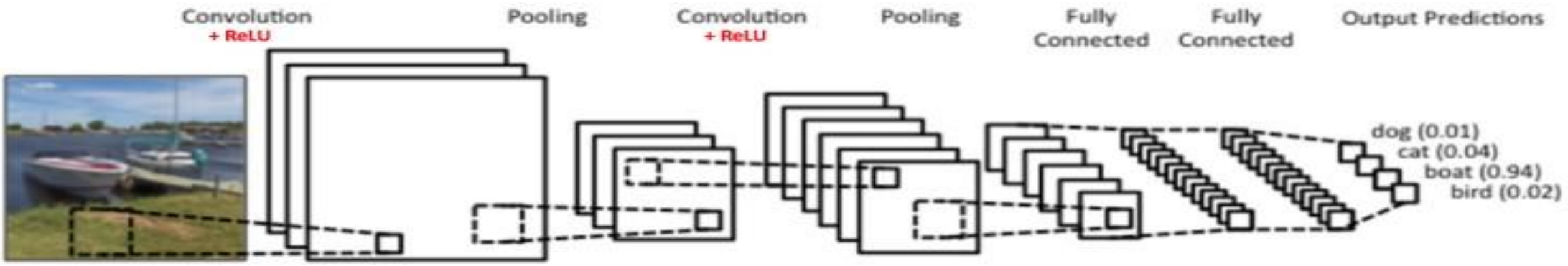
Convolução

- De forma geral, convolução é um método matemático para combinar dois sinais e produzir um terceiro.
  - Soma do produto das entradas.
- Convolução também está relacionada com conceitos de correlação/similaridade:
  - ex.:

$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}},$$

Interessante - Pedro Domingo “Deep Networks Are Kernel Machines”
https://www.youtube.com/watch?v=m3b0qEQHlUs

Camadas de Convolução



Filtros: Aplicando filtros de convolução

Etapas:

1. *Multiplicar cada elemento da matriz (pixel da imagem) pelo elemento correspondente do filtro.*

2. *Adicionar o resultado ao somatório.*

3. *Dividir pelo total de elementos.*

Filtros: Aplicando filtros de convolução

Filtro =



Objetivo: Obter uma nova matriz com filtro que representa a presença de padrões do filtro na matriz original.

Matriz =



Filtros: Aplicando filtros de convolução

Filtro =

-1	1

Matriz =

0	0	0	1	1	1	1	1

Matriz com filtro =

0	0	1	0	0	0	0

Filtros: Aplicando filtros de convolução

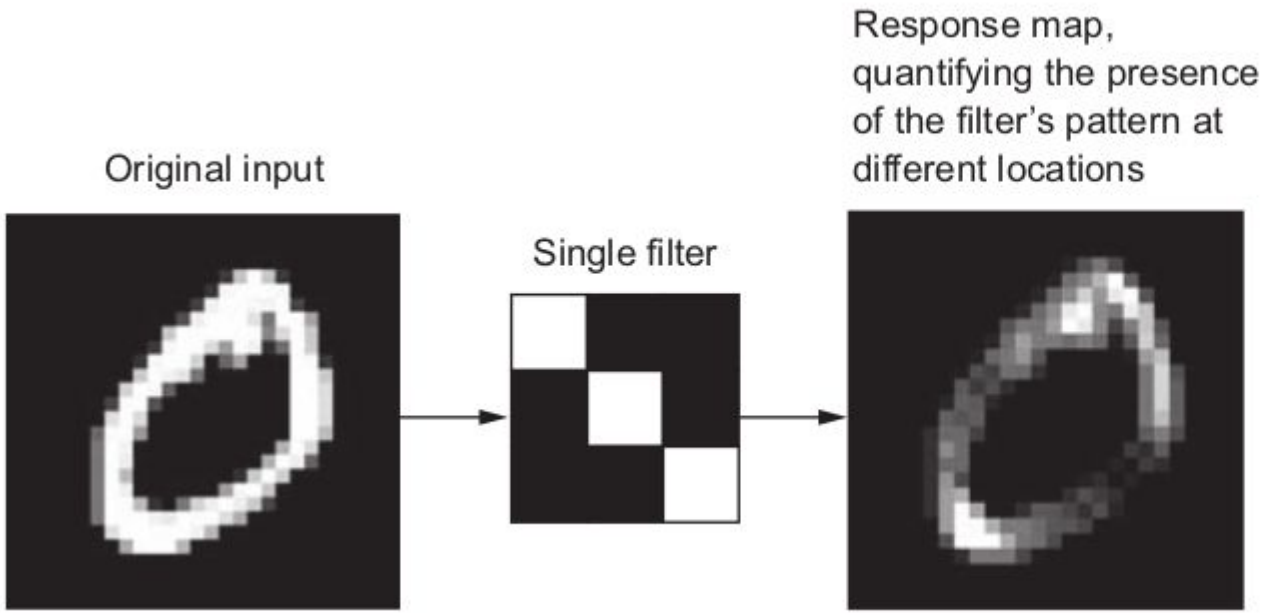


Figure 5.3 The concept of a *response map*: a 2D map of the presence of a pattern at different locations in an input

Filtros: Aplicando filtros de convolução

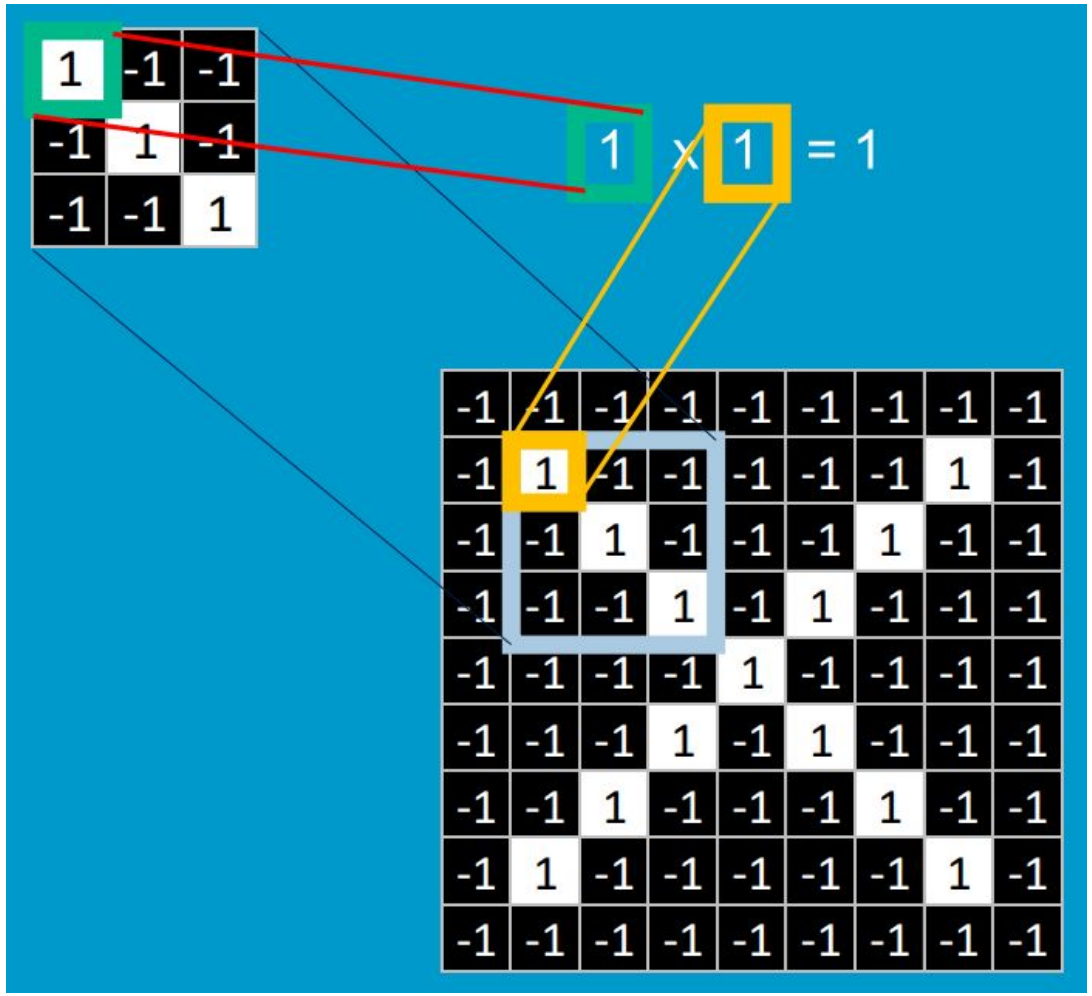
Etapas (relembrando):

1. *Multiplicar cada elemento da matriz (pixel da imagem) pelo elemento correspondente do filtro.*

2. *Adicionar o resultado ao somatório.*

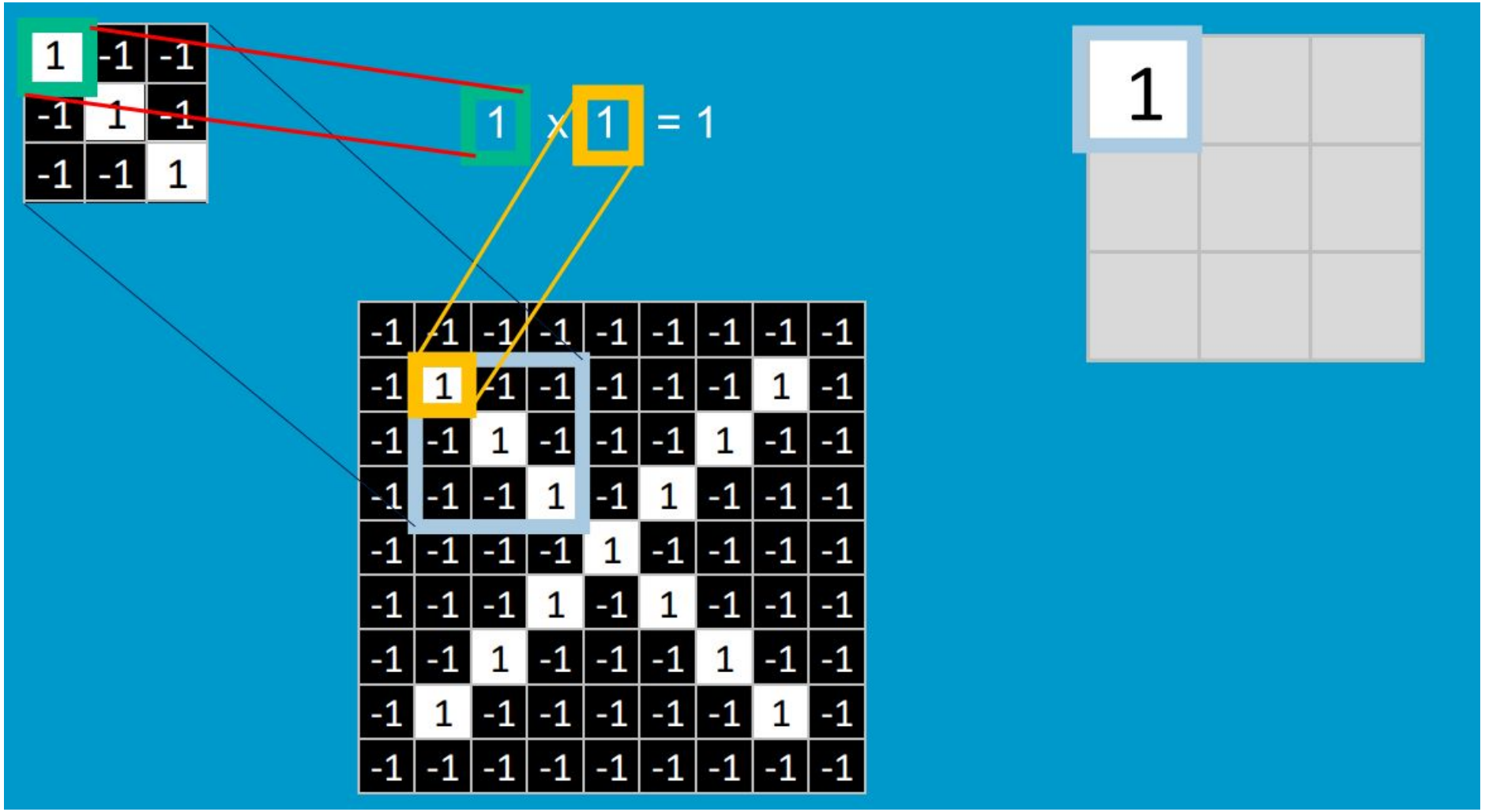
3. *Dividir pelo total de elementos.*

Filtros: Aplicando filtros de convolução

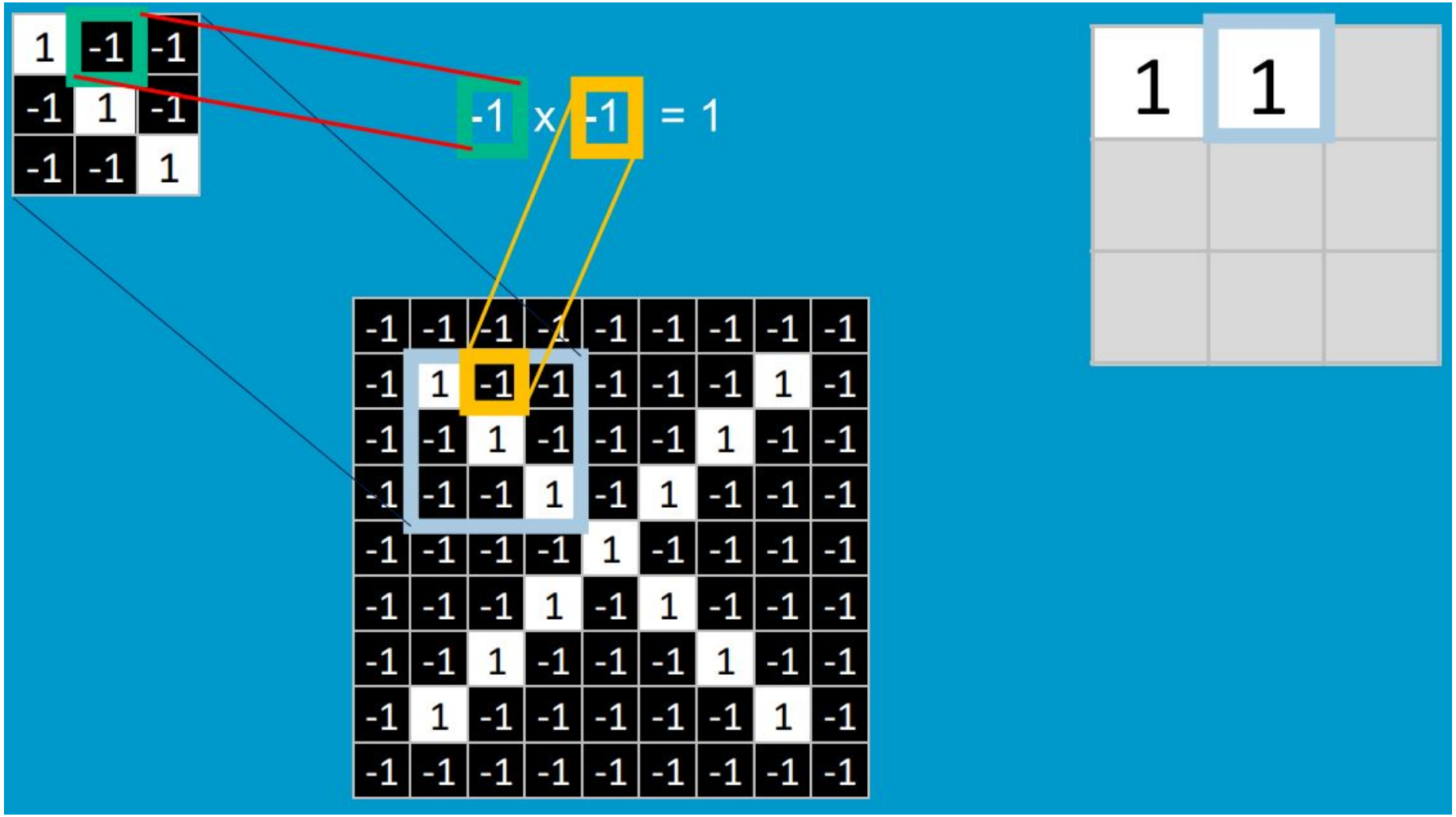


Produto escalar -> Similaridade?

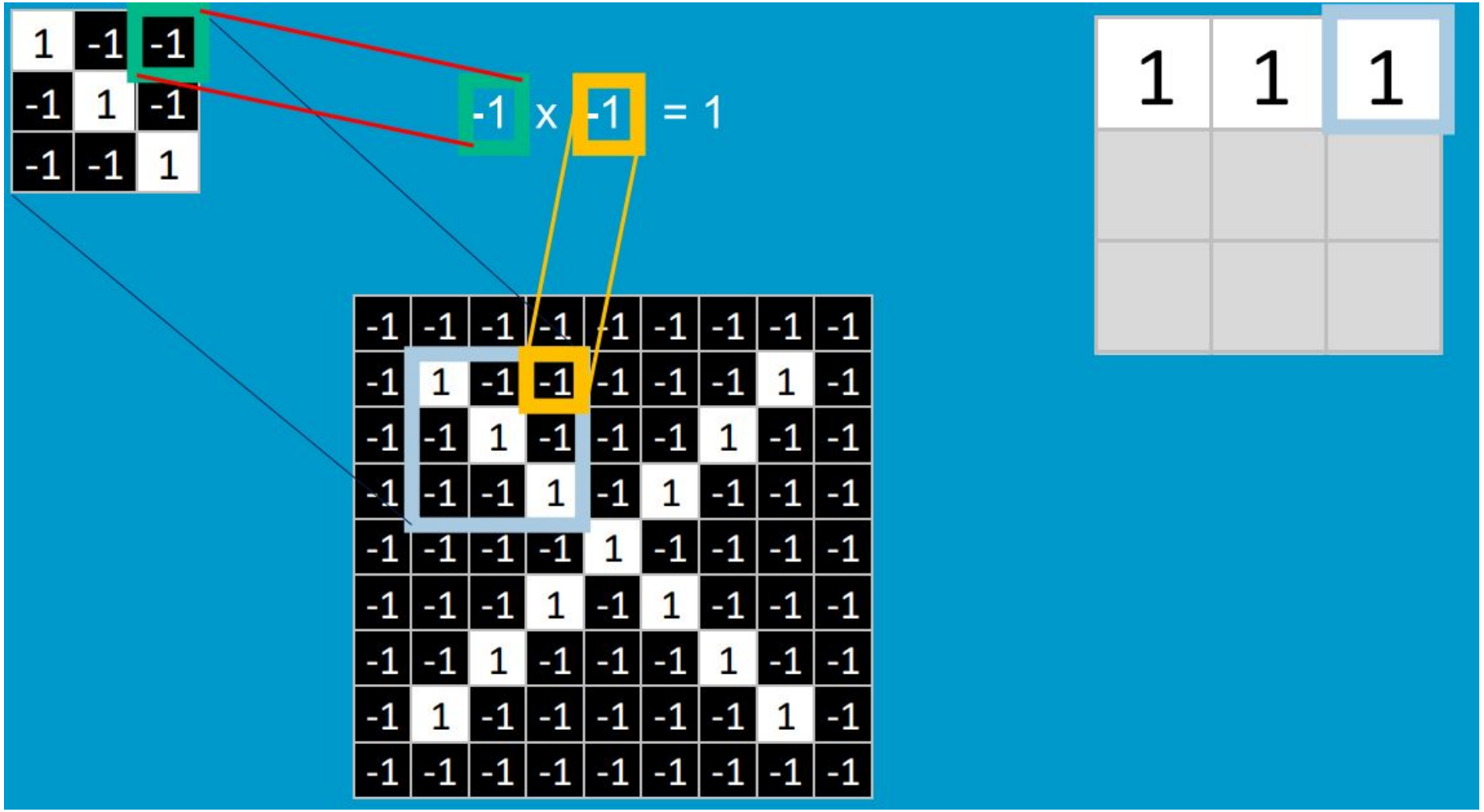
Filtros: Aplicando filtros de convolução



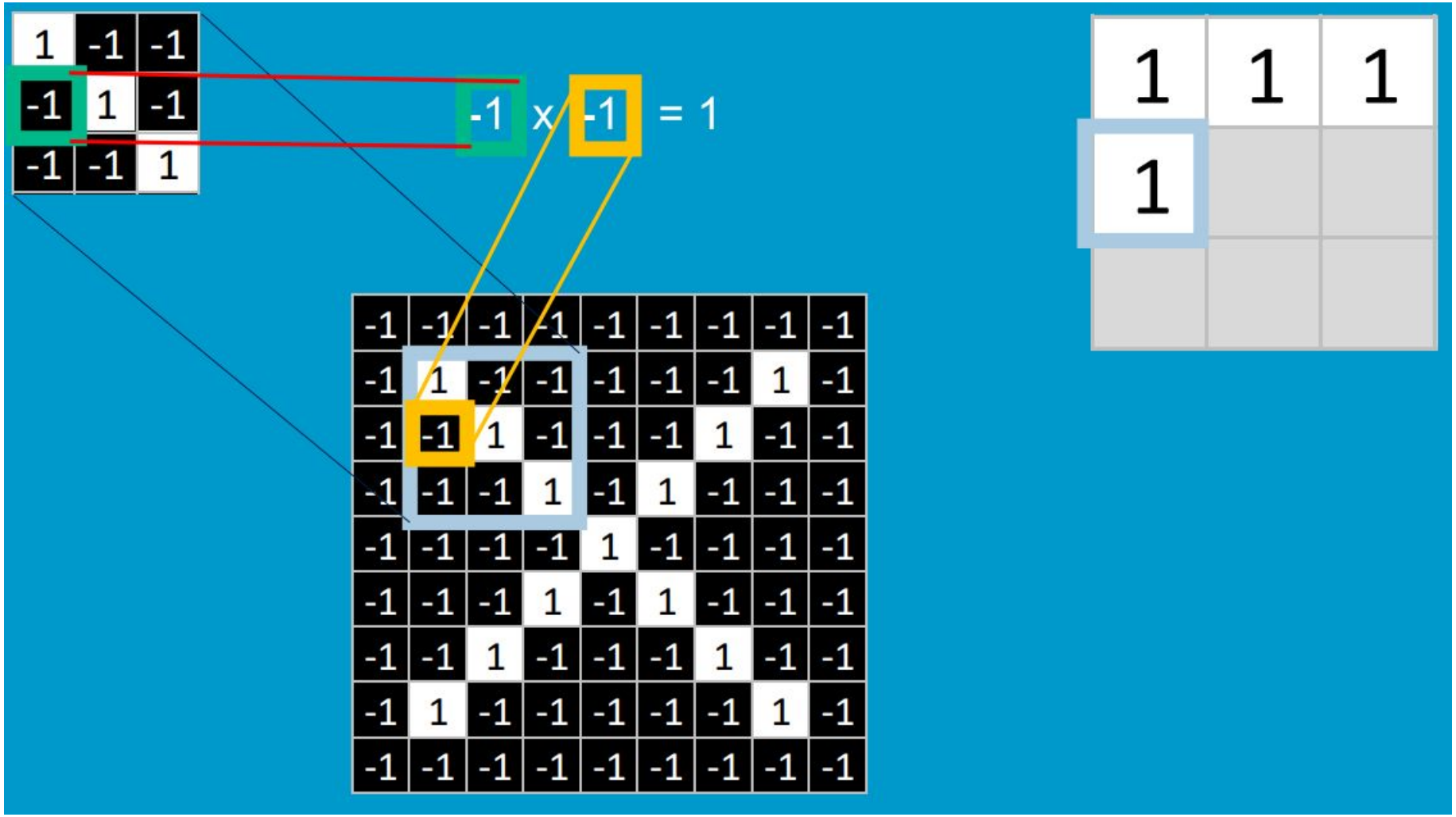
Filtros: Aplicando filtros de convolução



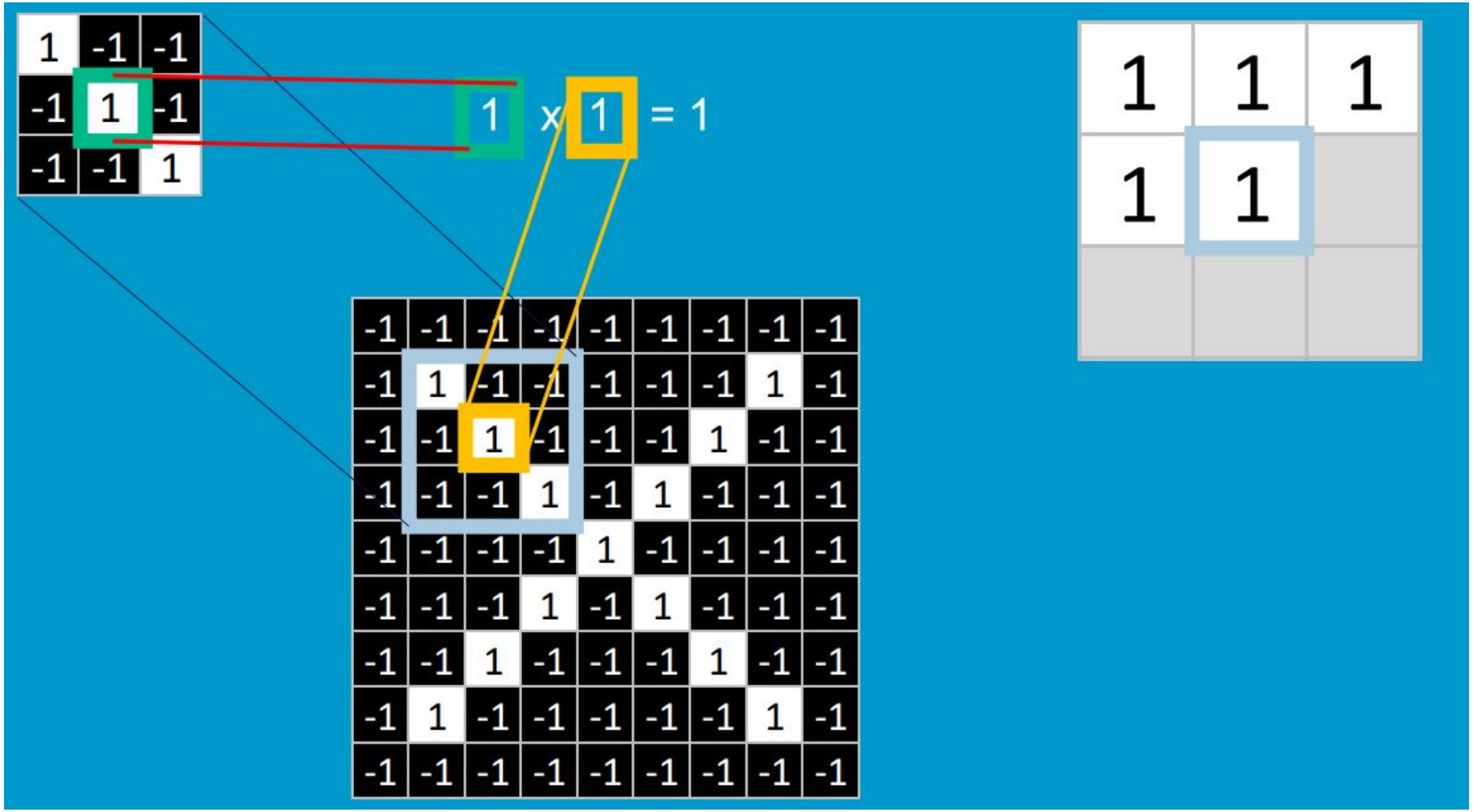
Filtros: Aplicando filtros de convolução



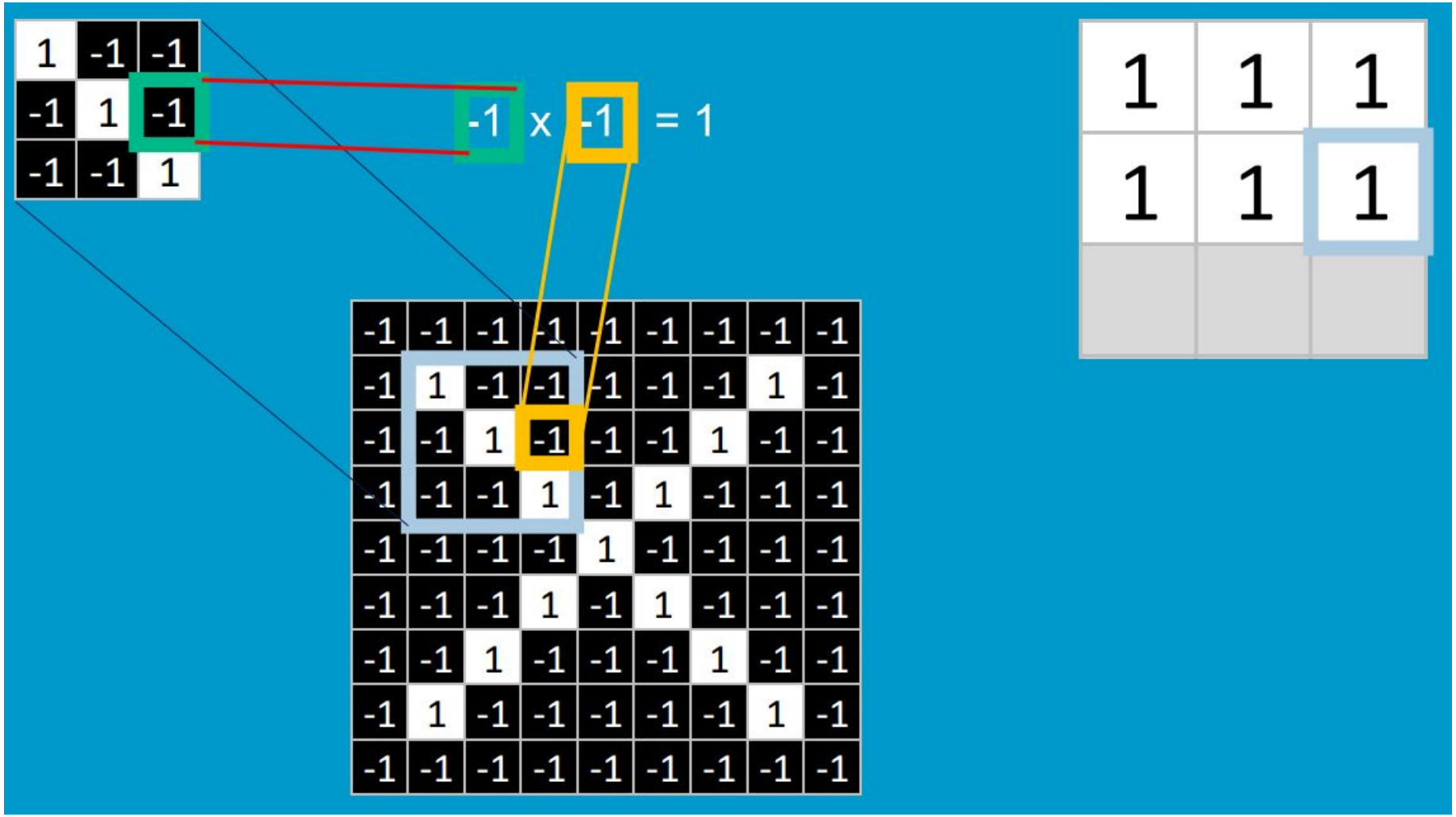
Filtros: Aplicando filtros de convolução



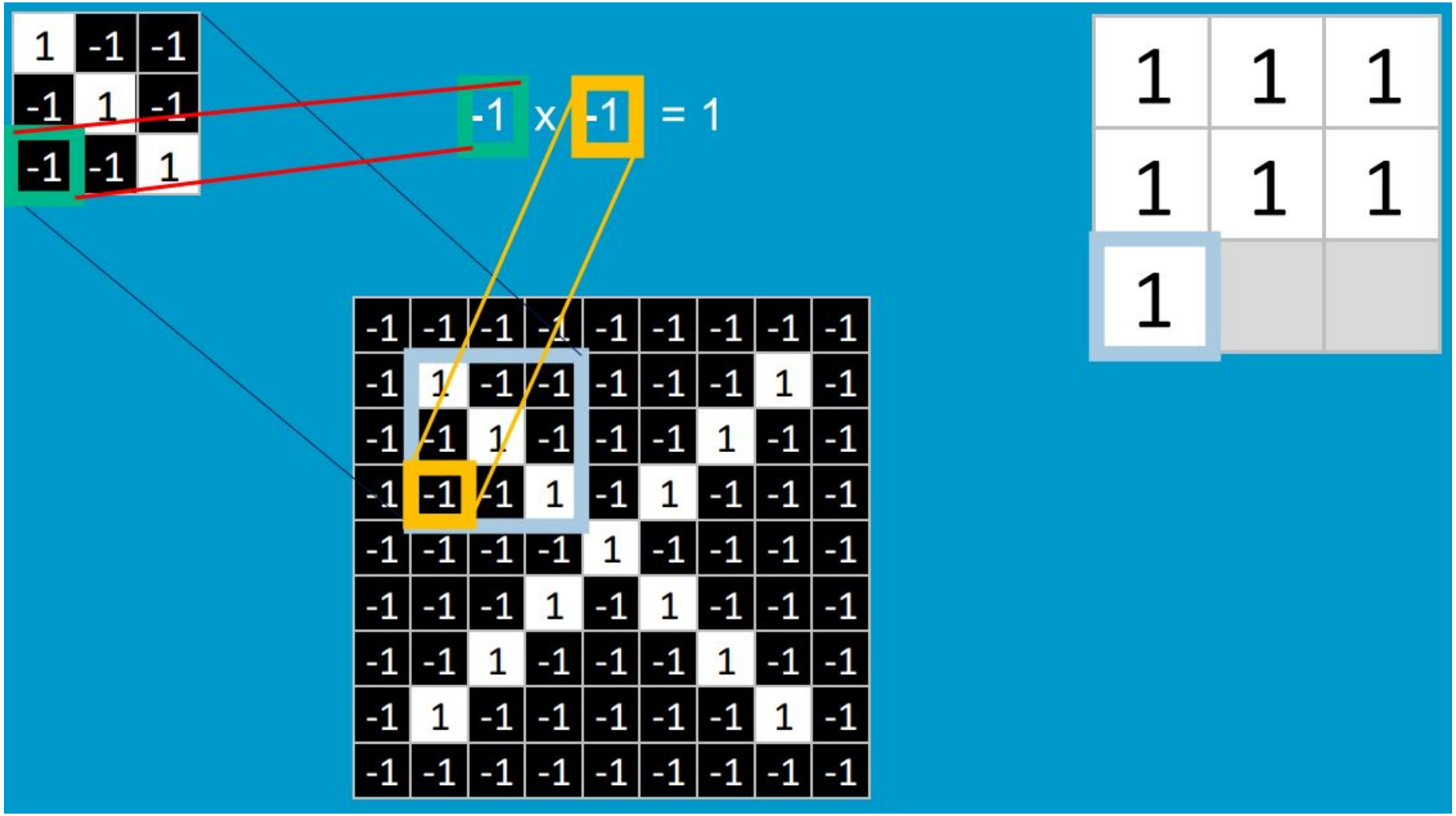
Filtros: Aplicando filtros de convolução



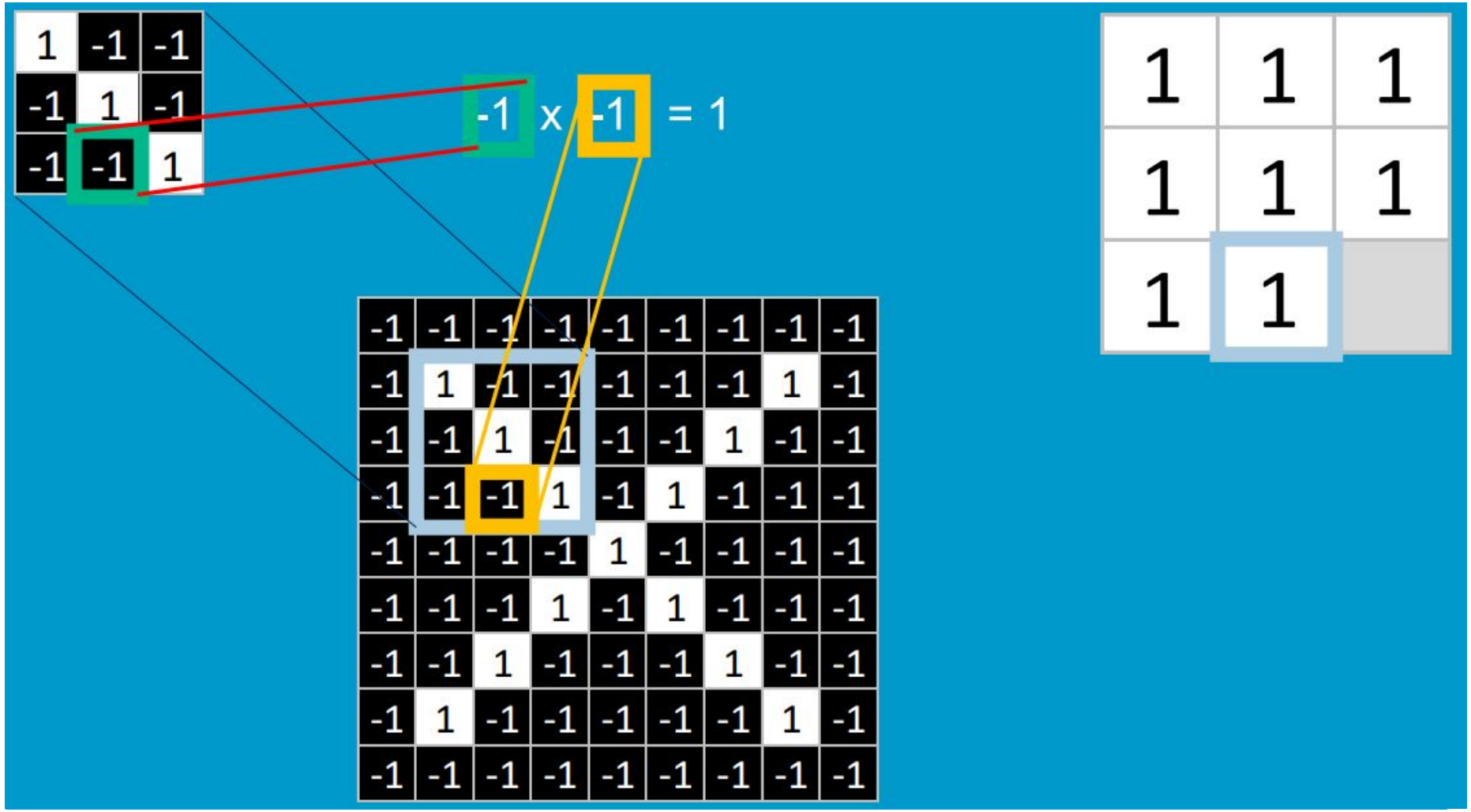
Filtros: Aplicando filtros de convolução



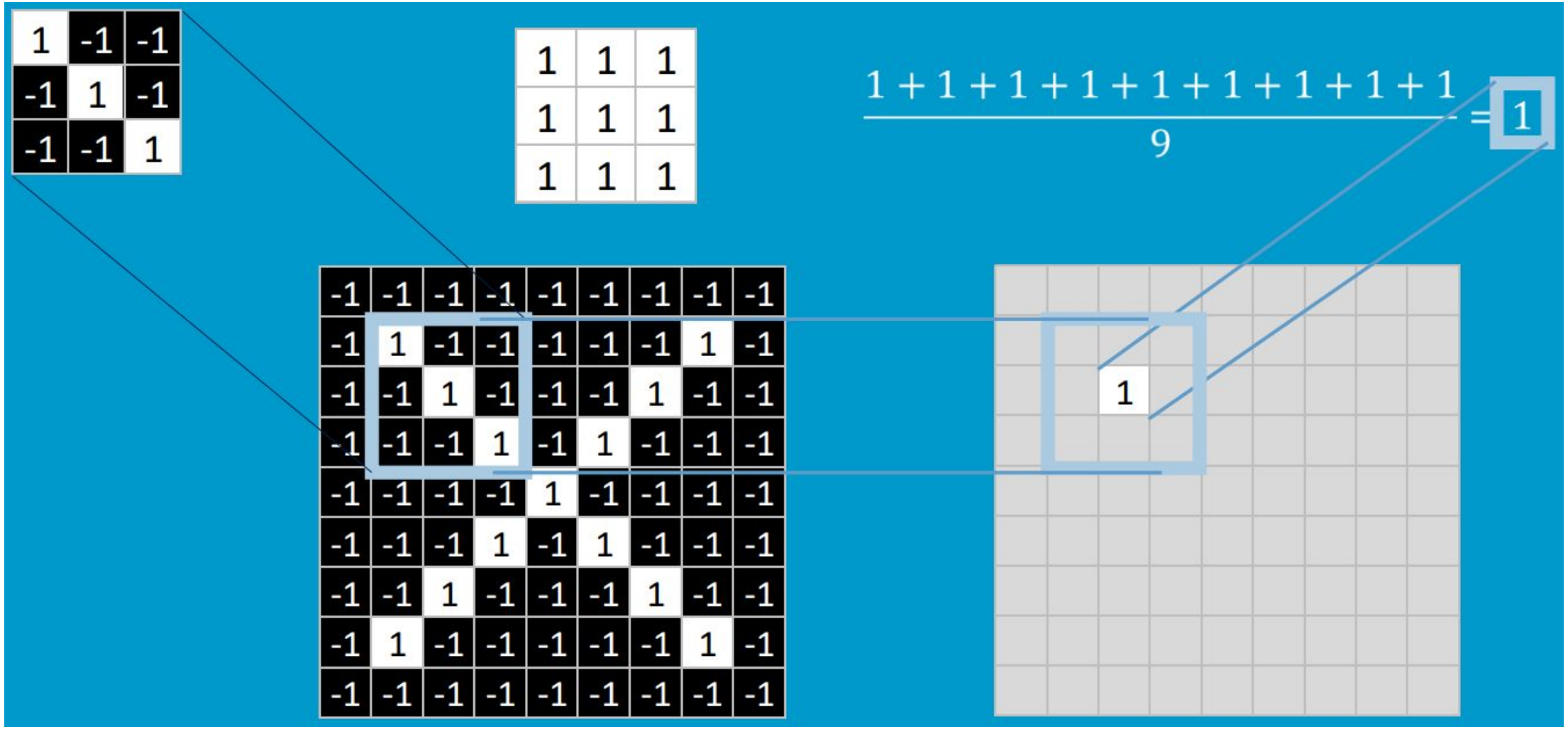
Filtros: Aplicando filtros de convolução



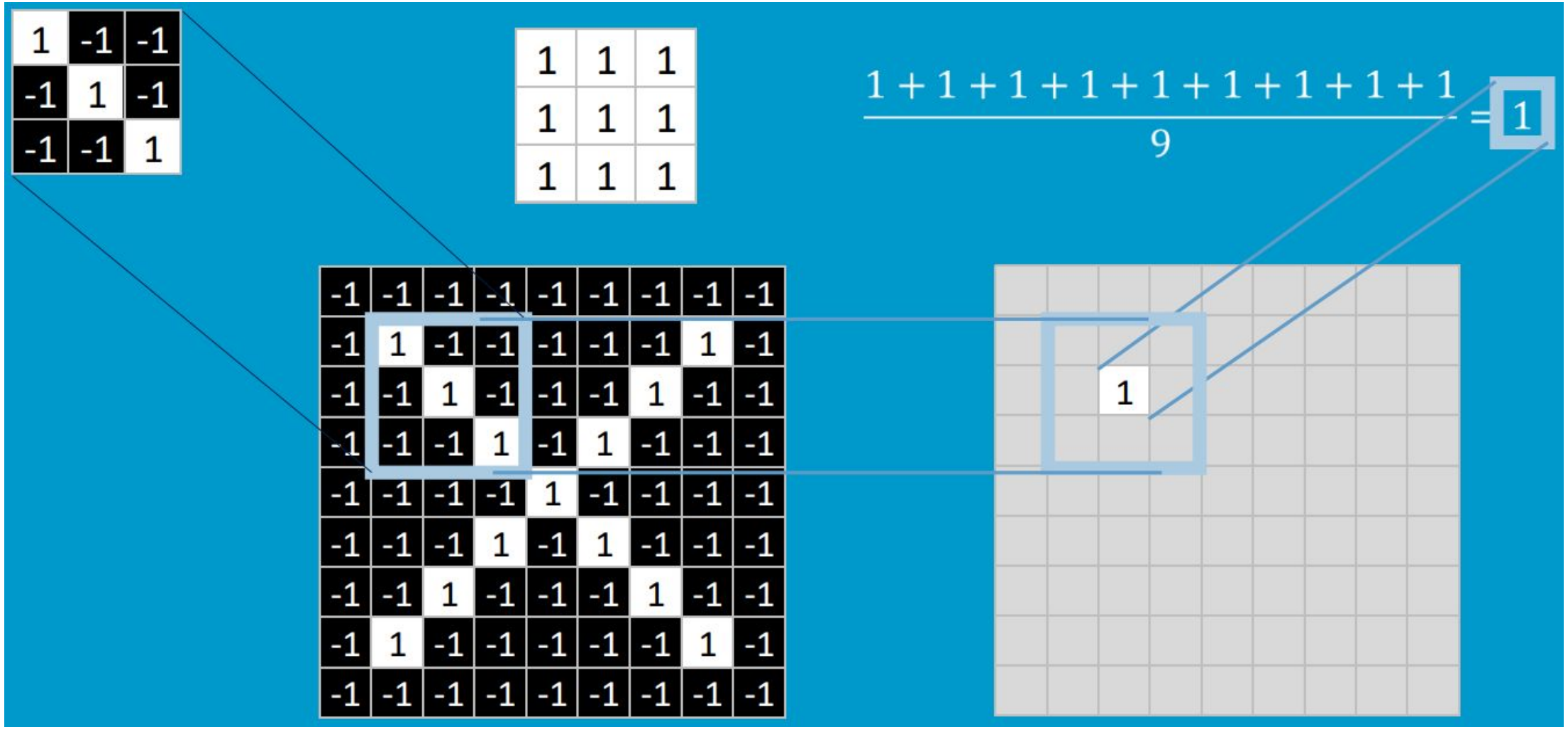
Filtros: Aplicando filtros de convolução



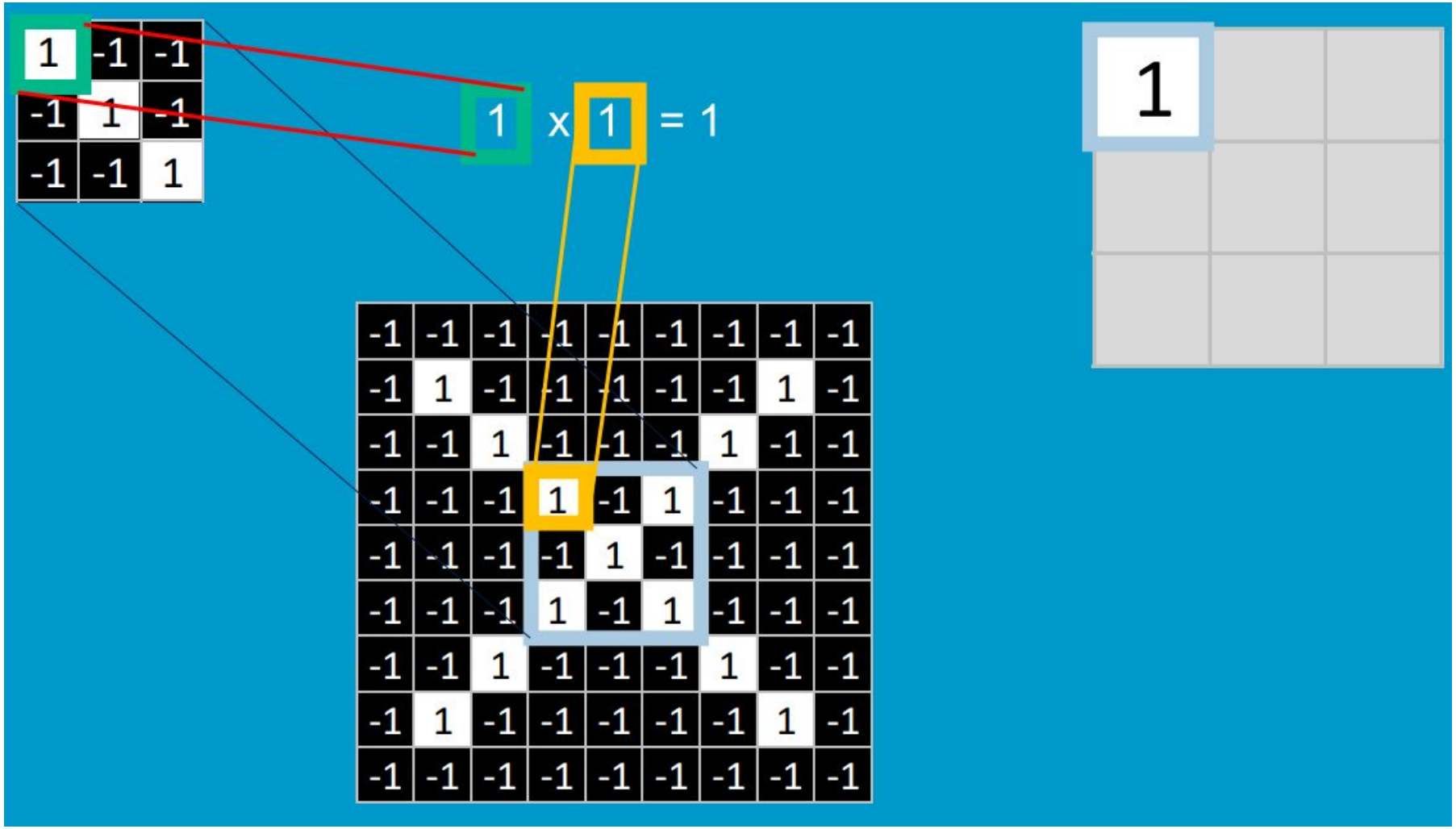
Filtros: Aplicando filtros de convolução



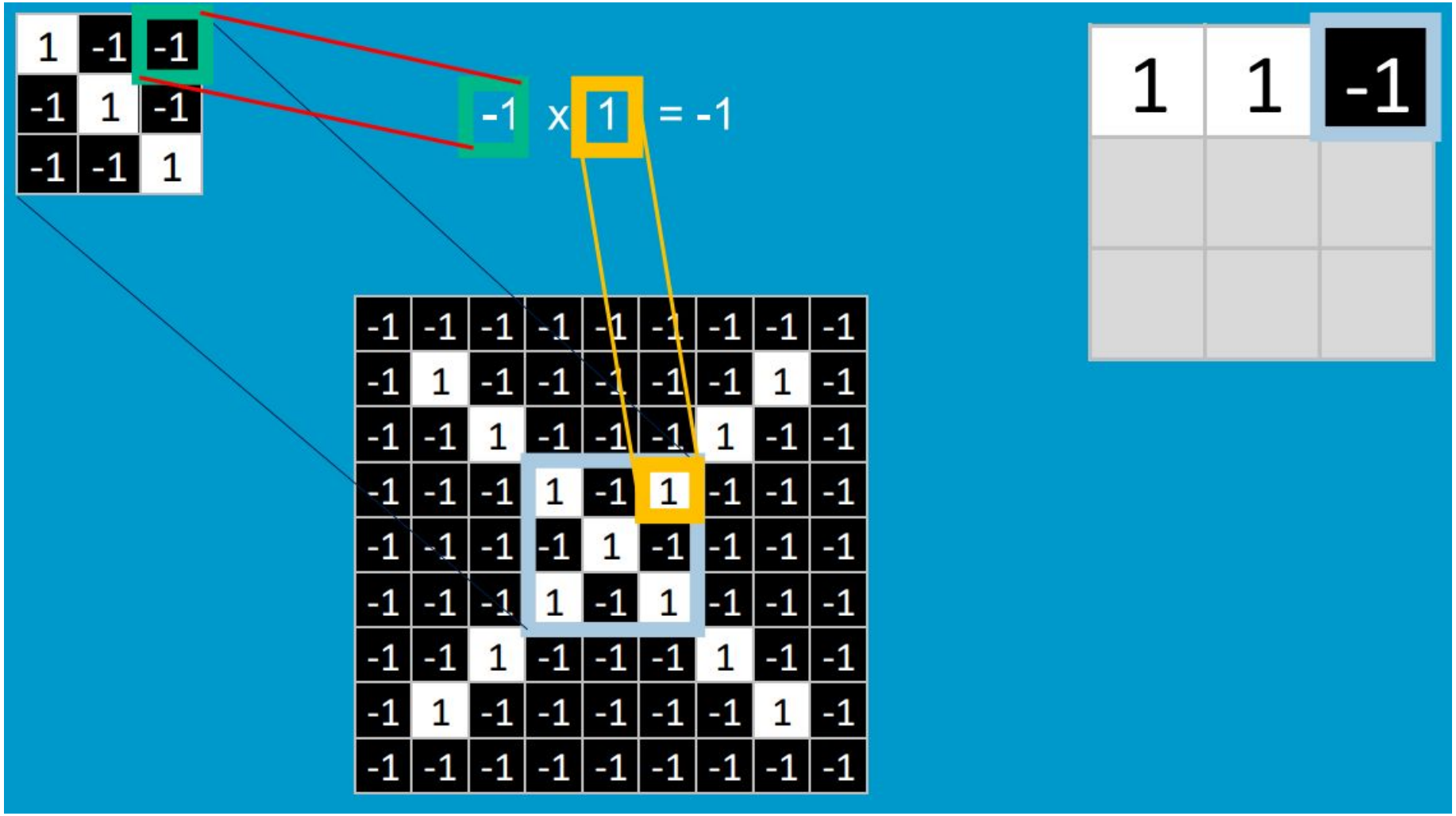
Filtros: Aplicando filtros de convolução



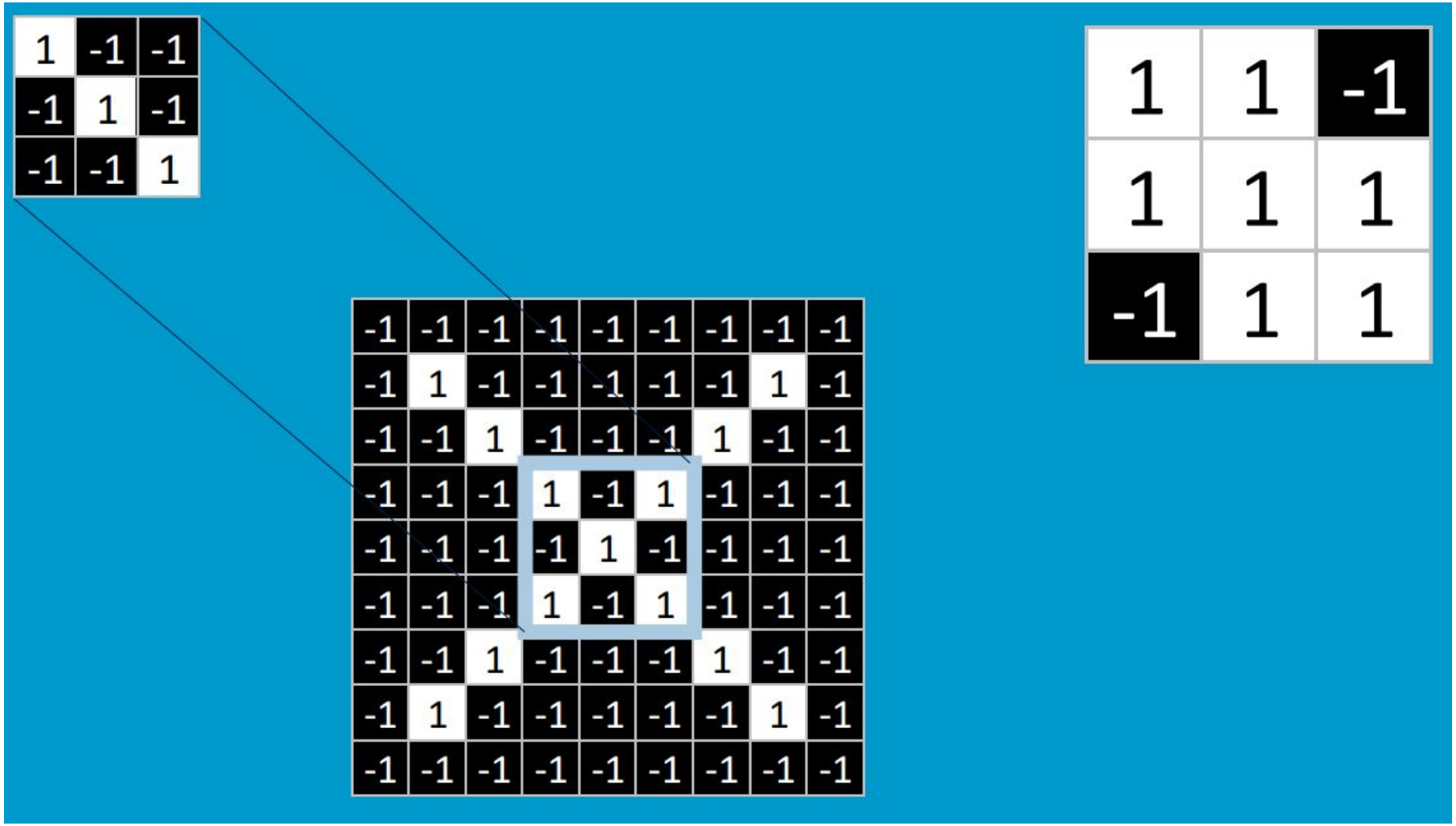
Filtros: Aplicando filtros de convolução



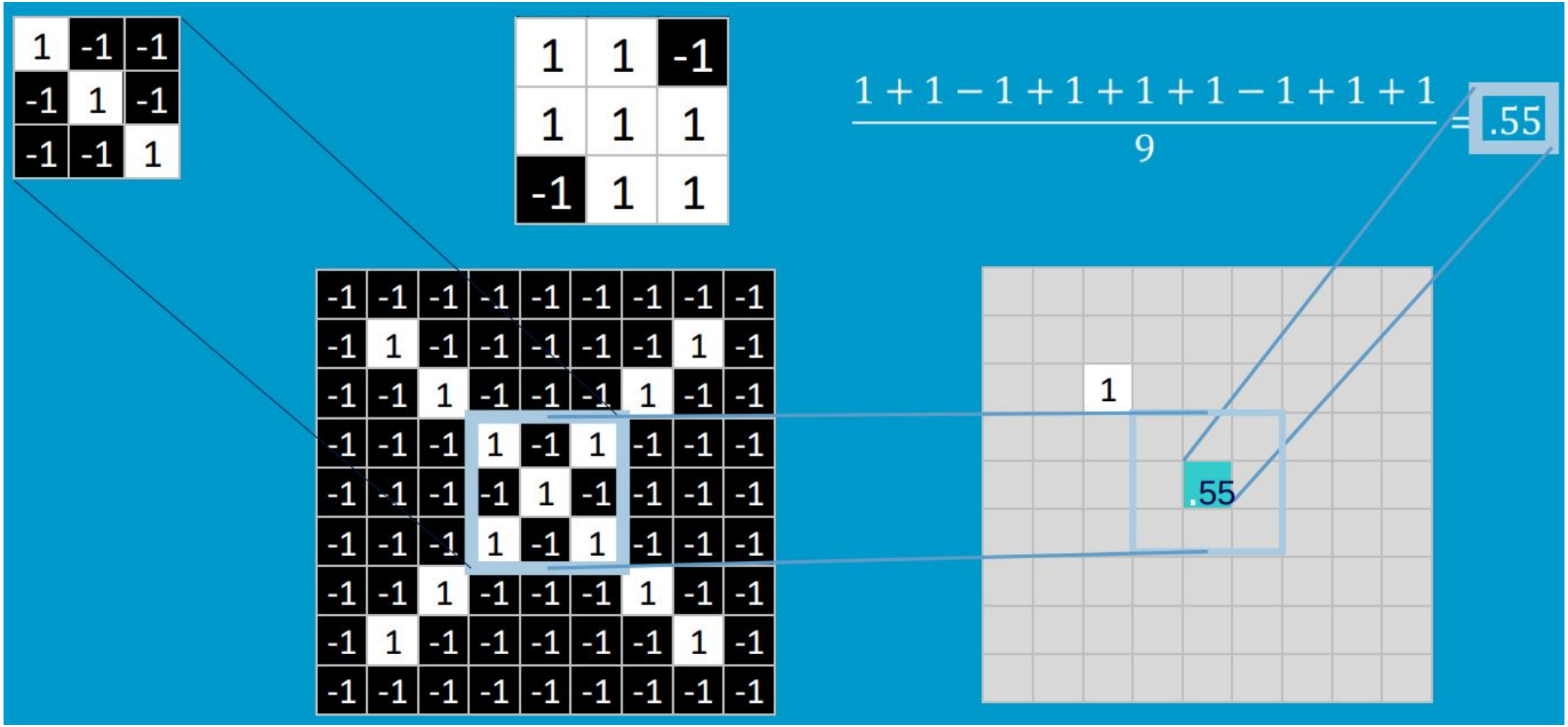
Filtros: Aplicando filtros de convolução



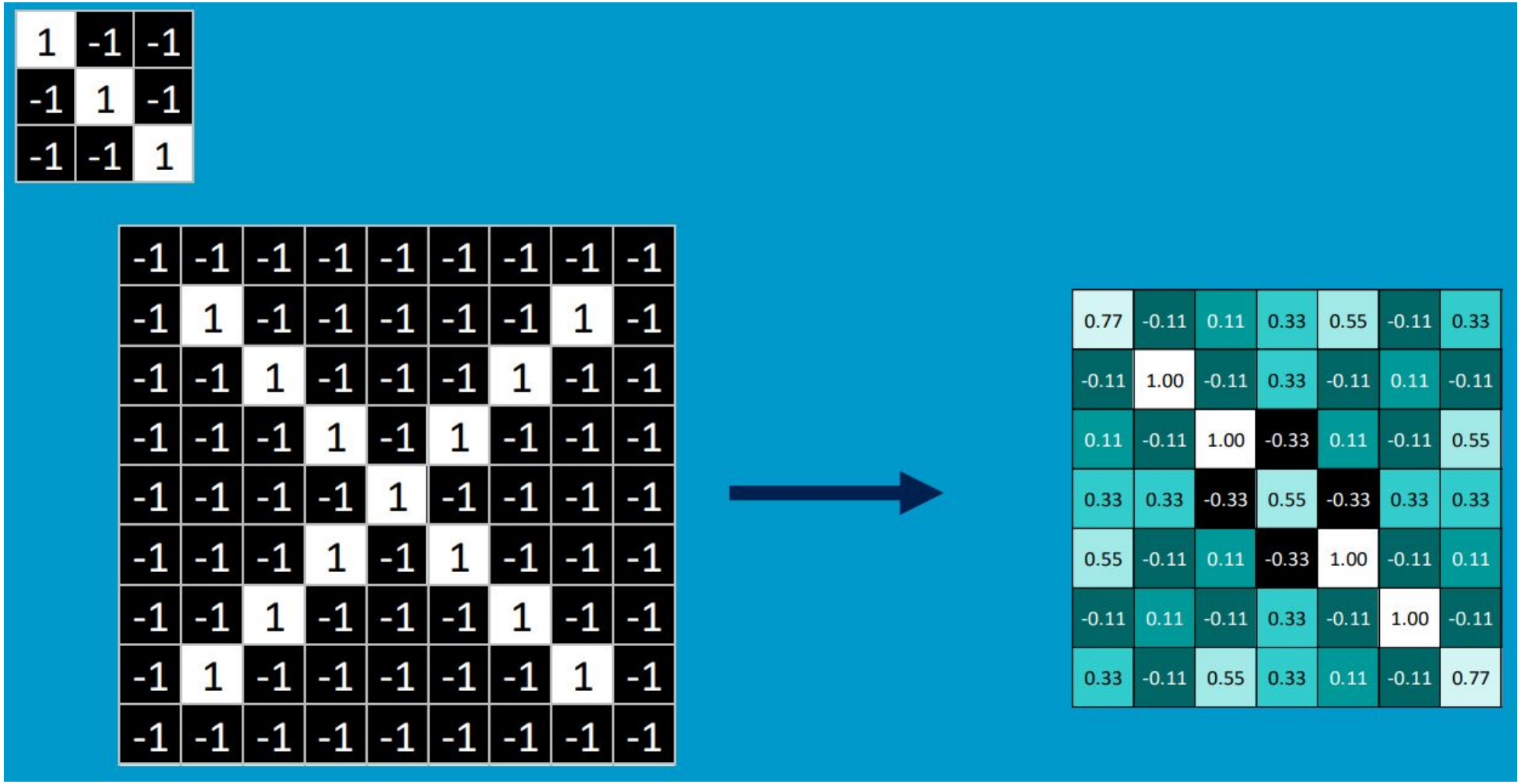
Filtros: Aplicando filtros de convolução



Filtros: Aplicando filtros de convolução



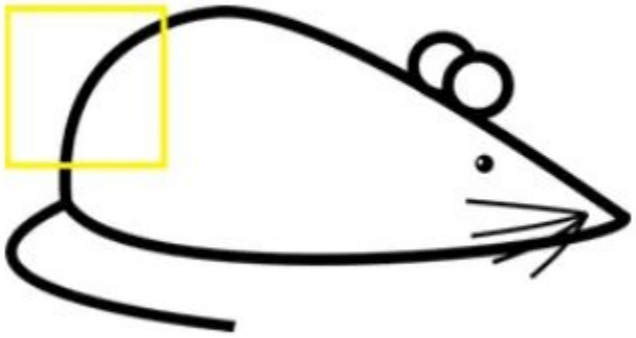
Filtros: Aplicando filtros de convolução



Filtros: Aplicando filtros de convolução



Original image



Visualization of the filter on the image



Visualization of the receptive field

0	0	0	0	0	0	30
0	0	0	0	50	50	50
0	0	0	20	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0

Pixel representation of the receptive field

*

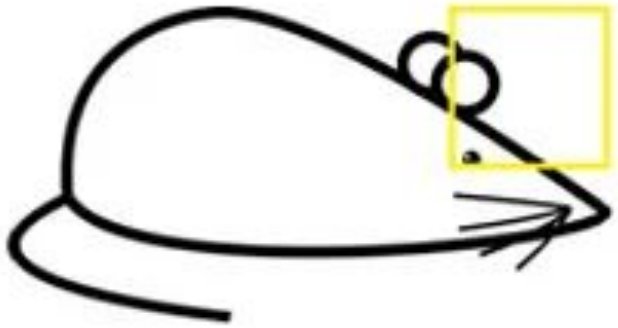
0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

Soma = (50·30) + (50·30) + (50·30) + (20·30) + (50·30) = 6600

Fonte: https://medium.com/turing-talks/turing-talks-23-modelos-de-predi%C3%A7%C3%A3o-redes-neurais-convolucionais-d364654a34de

Filtros: Aplicando filtros de convolução



Visualization of the filter on the image

0	0	0	0	0	0	0
0	40	0	0	0	0	0
40	0	40	0	0	0	0
40	20	0	0	0	0	0
0	50	0	0	0	0	0
0	0	50	0	0	0	0
25	25	0	50	0	0	0

Pixel representation of receptive field

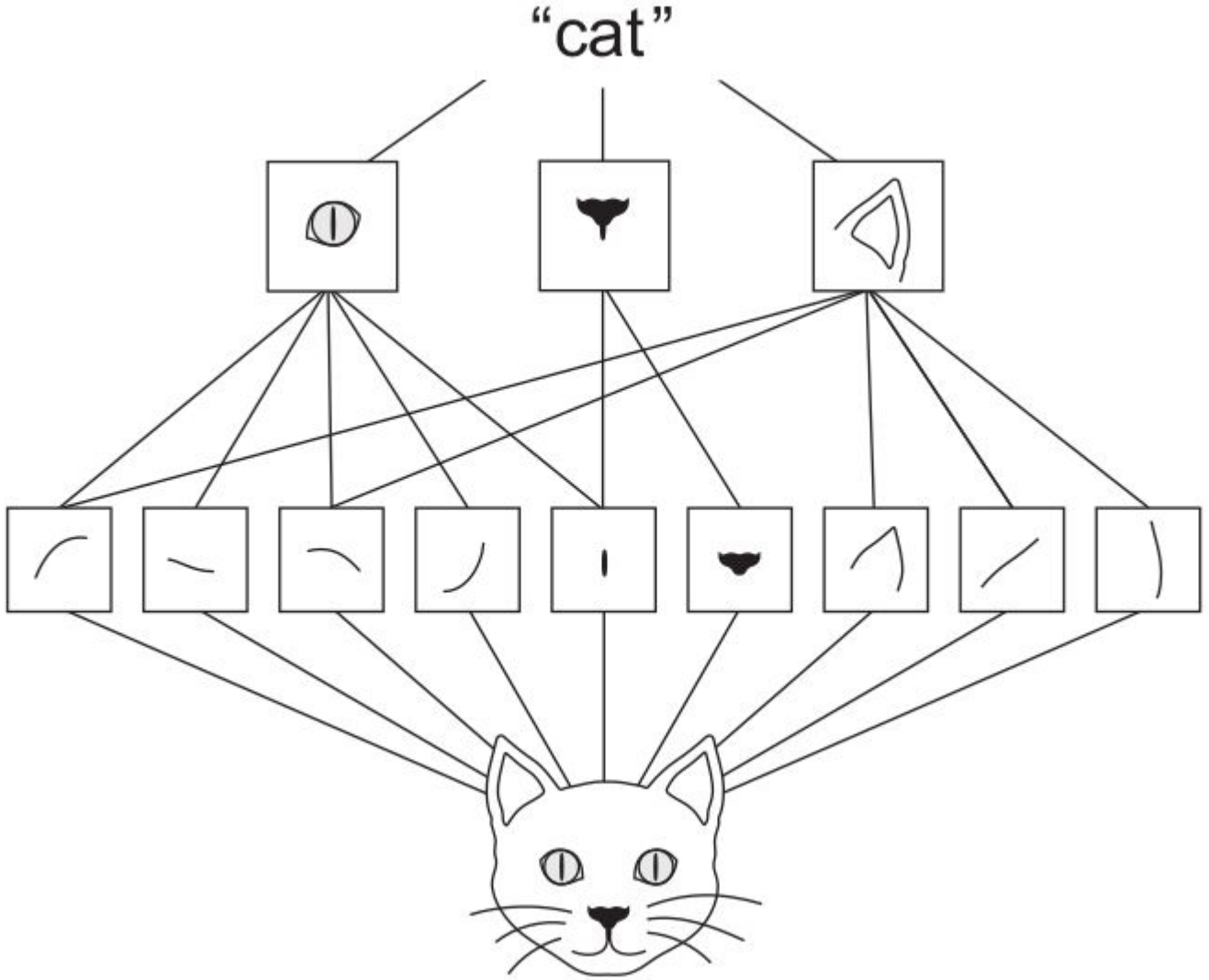
*

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

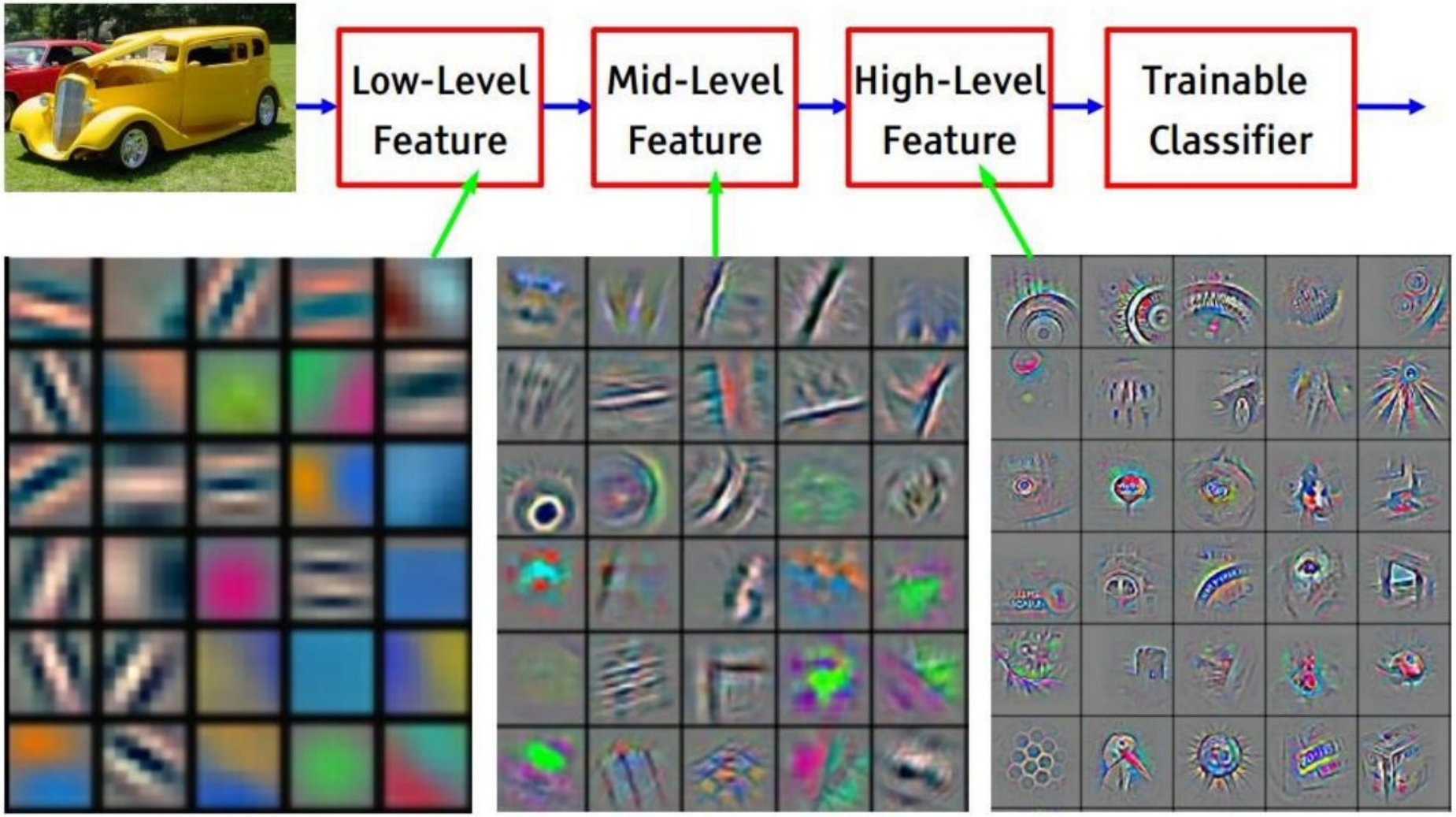
Pixel representation of filter

Soma=0

Aplicando filtros de convolução



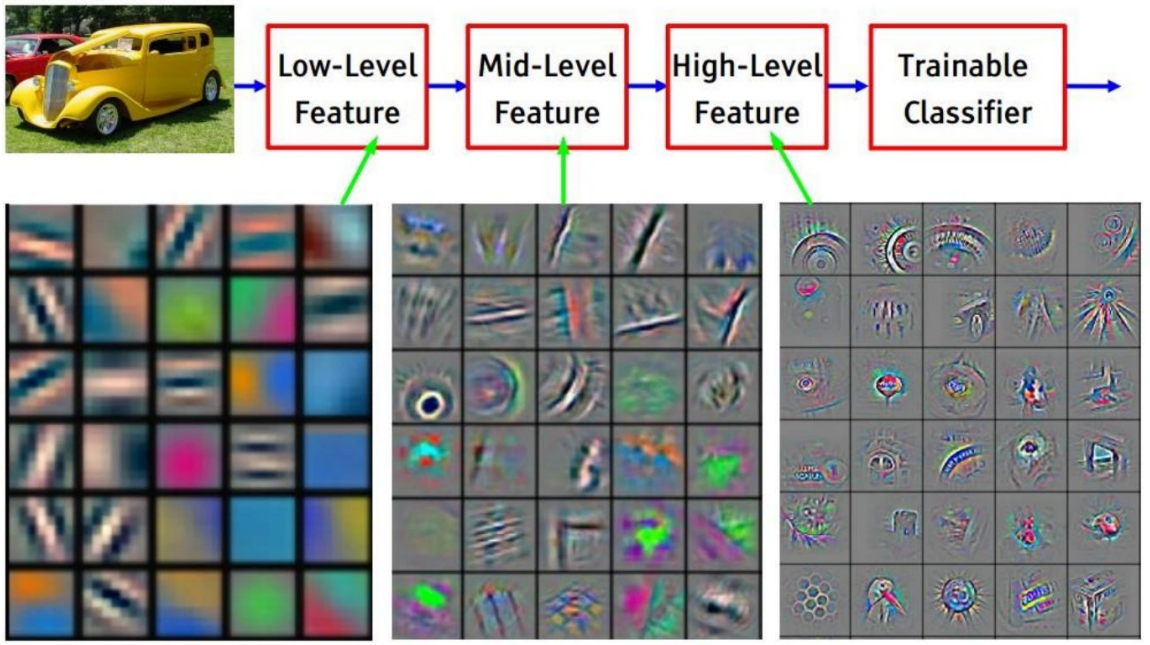
Aplicando filtros de convolução



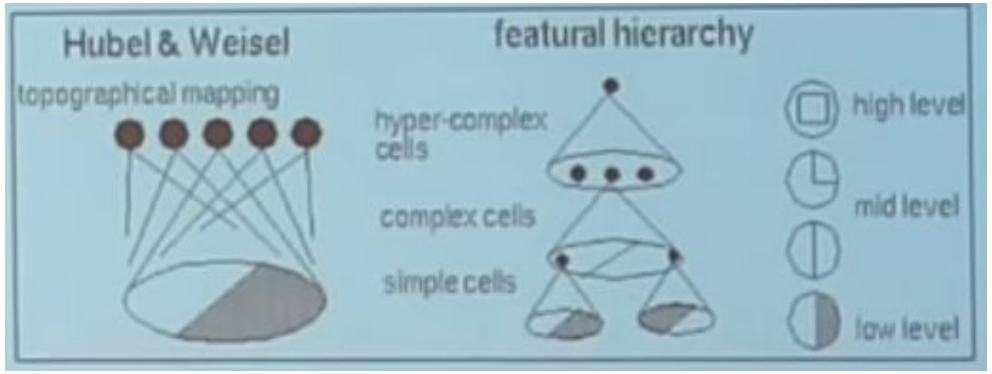
Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Exemplo: Filtros de convolução
nos seus diferentes níveis da rede.

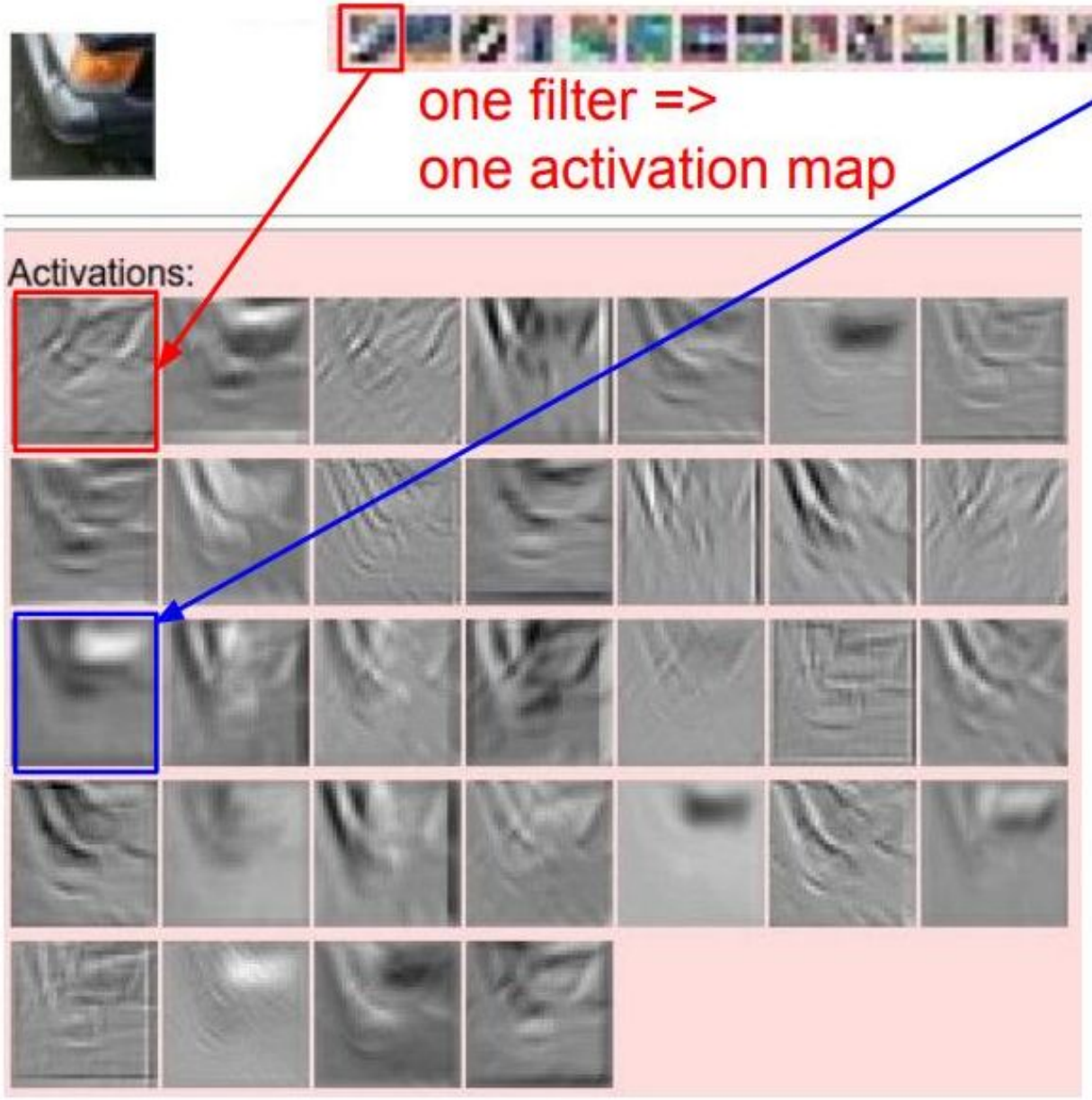
Aplicando filtros de convolução



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]



Aplicando filtros de convolução

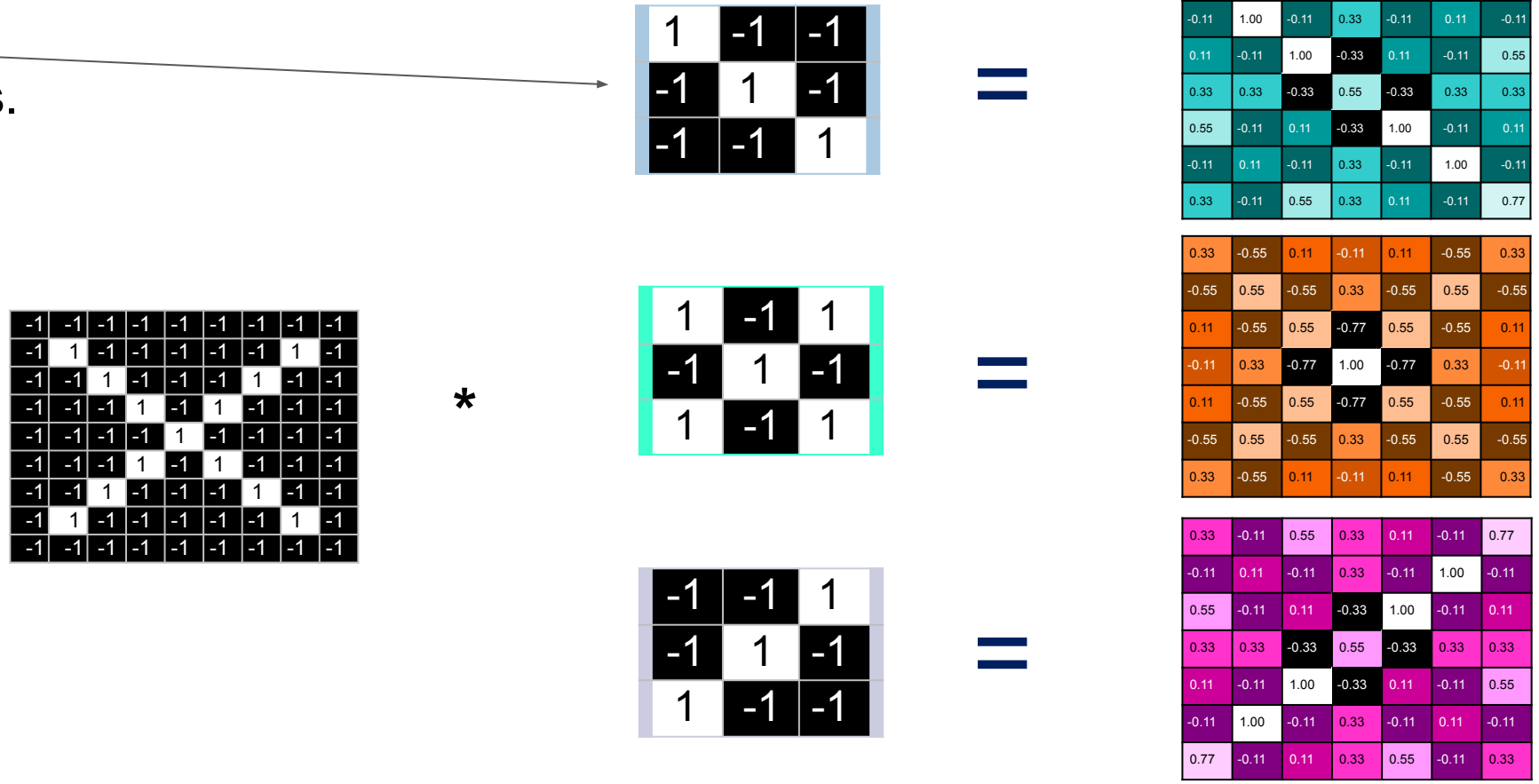


Exemplo: Mapas de ativação -- aplicação dos diferentes filtros de convolução na imagem.

Uma imagem gera um conjunto de imagens…

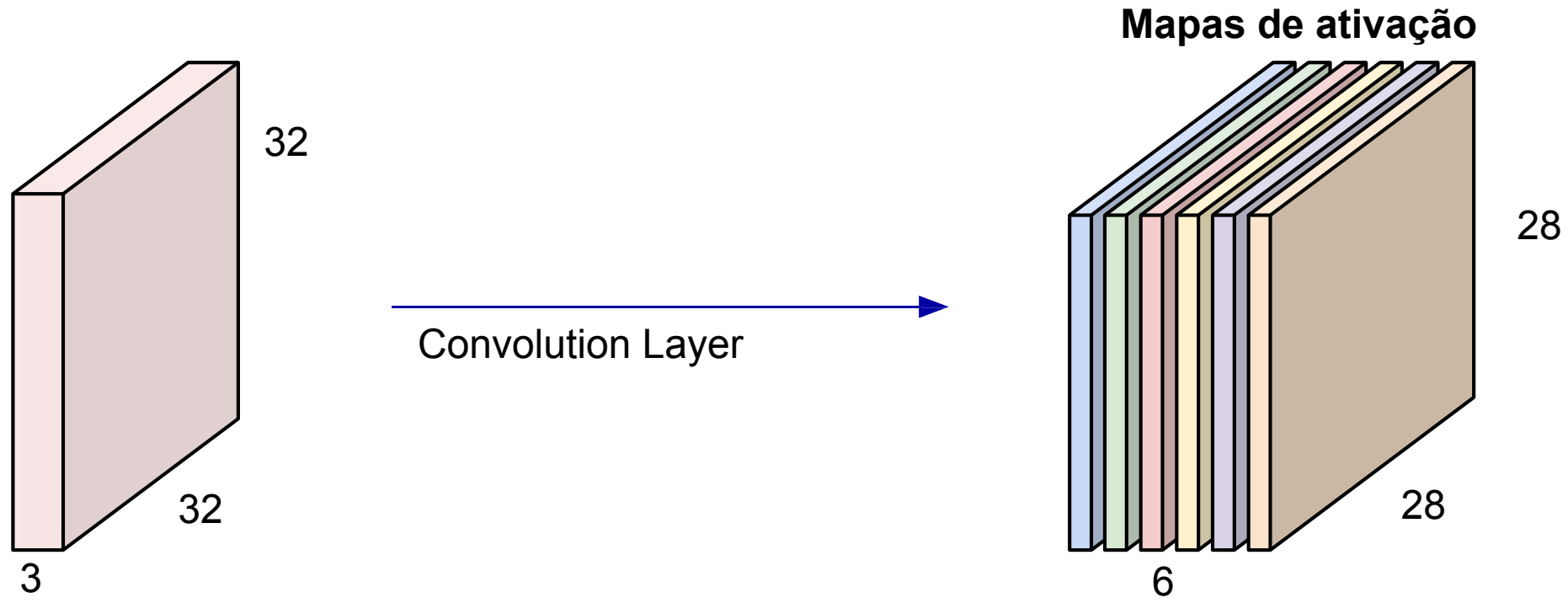
Mapas de ativação:
- Respostas aos filtros.

Filtros:
- Kernels.



Camada de convolução

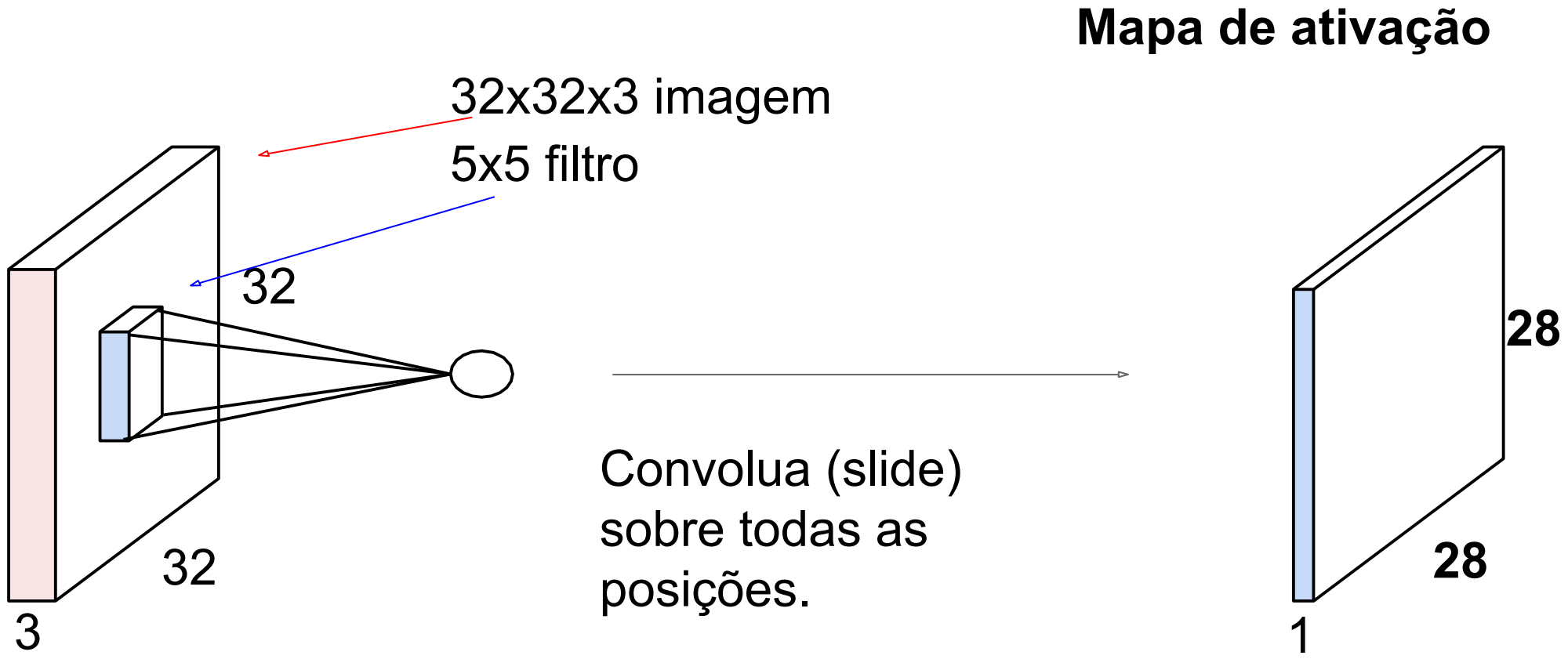
Por exemplo, se nós temos 6 filtros 5x5, nós iremos obter 6 mapas de ativação:



Nós empilhamos estas novas imagens 28x28x6!

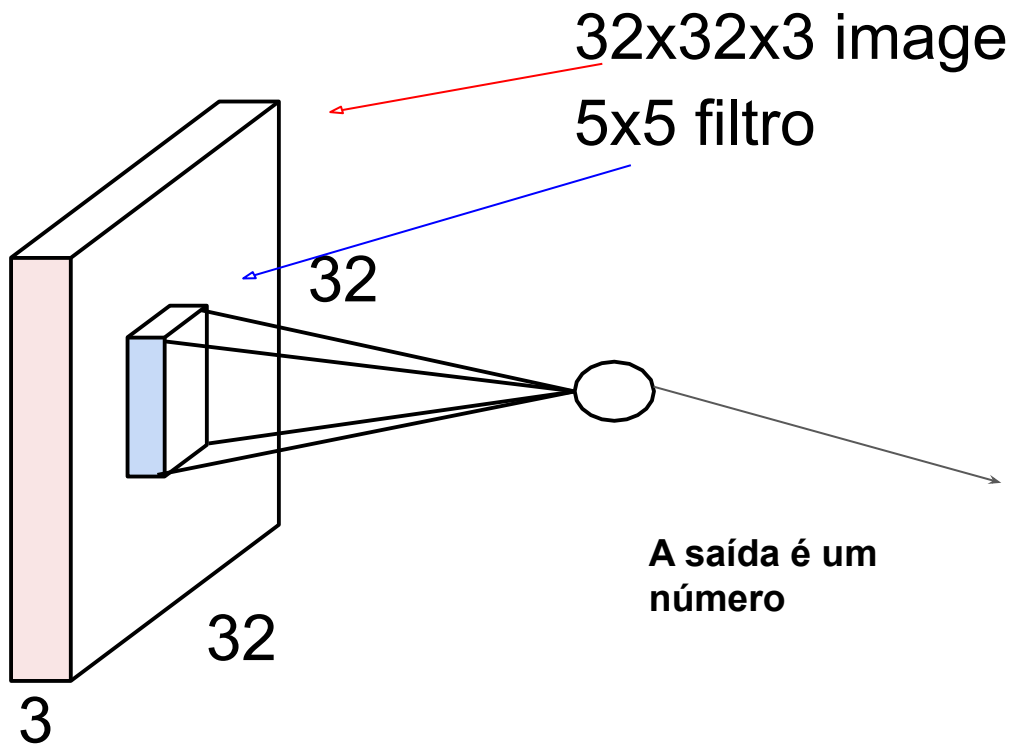
Camada de convolução: detalhes

Olhando mais de perto as dimensões:

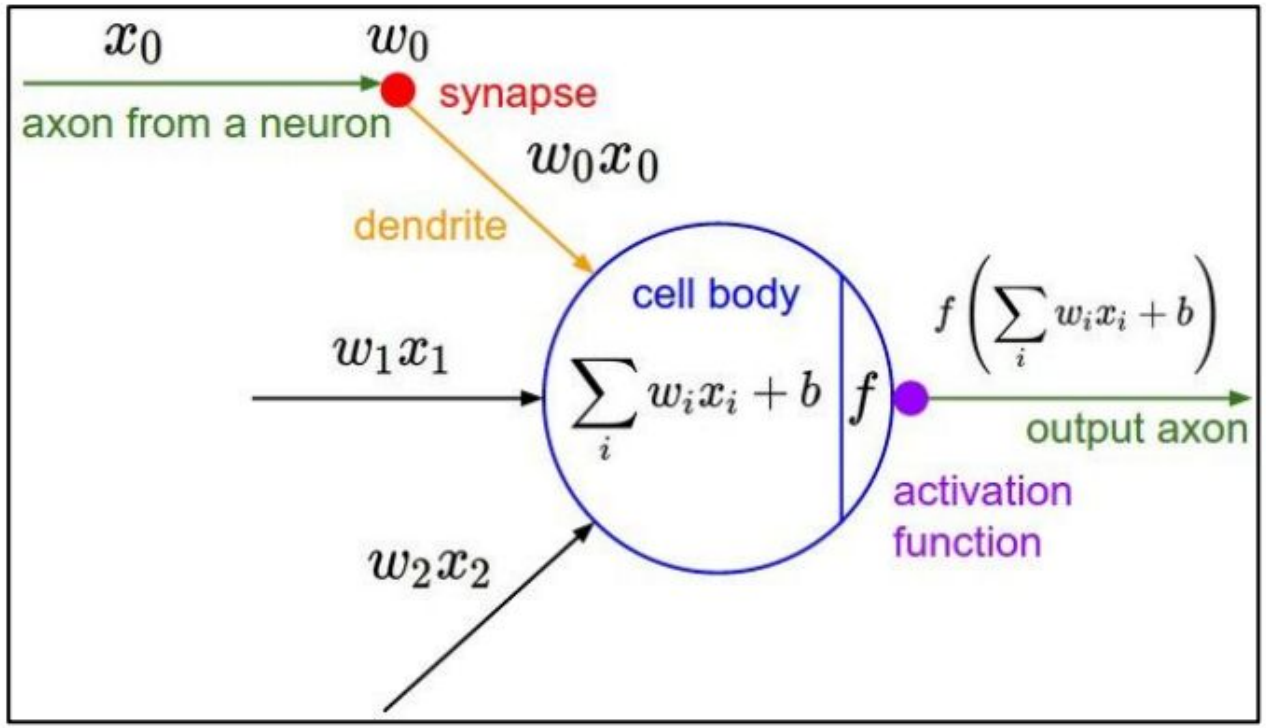


Camada de convolução: detalhes

Olhando mais de perto as dimensões:



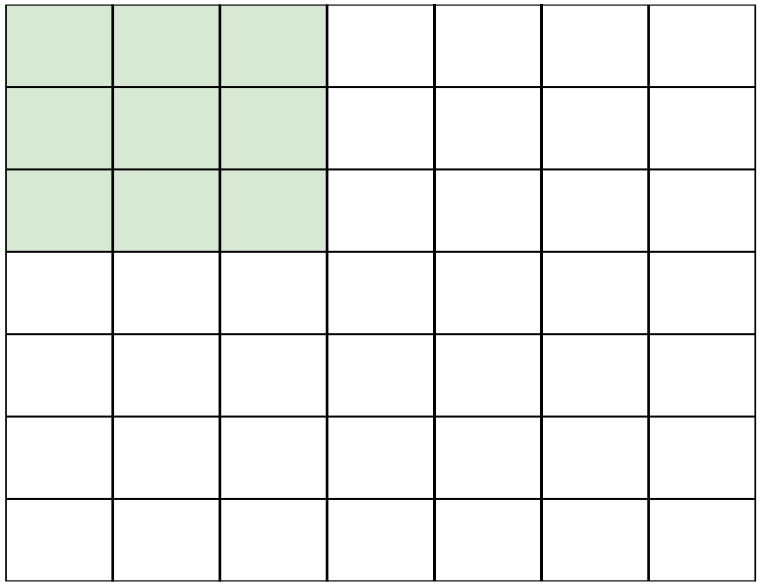
A saída é um número



… que podemos ver como a saída de um neurônio com conectividade local!

Camada de convolução: detalhes

Olhando mais de perto as dimensões:

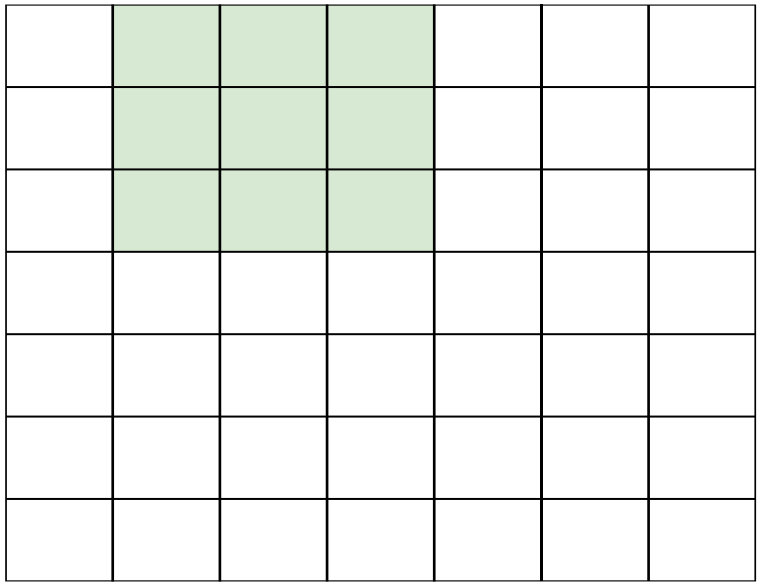


Entrada 7x7

Filtro 3x3

Camada de convolução: detalhes

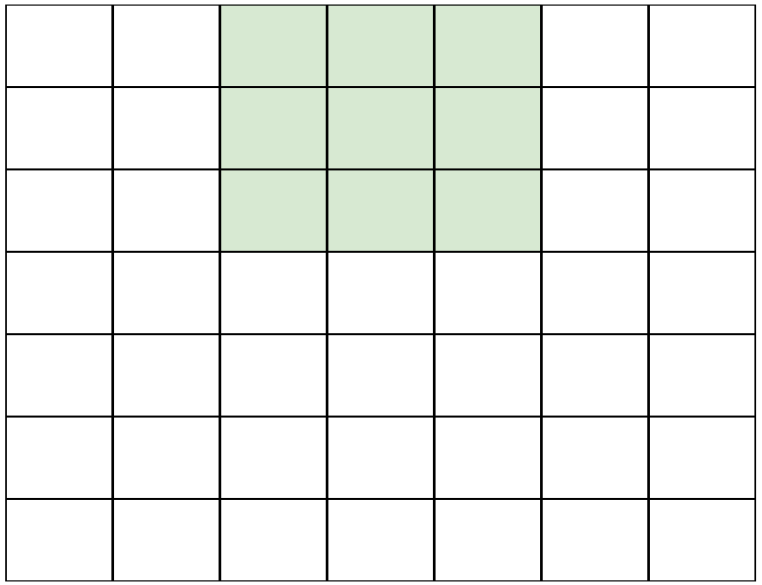
Olhando mais de perto as dimensões:



- Entrada 7x7
- Filtro 3x3

Camada de convolução: detalhes

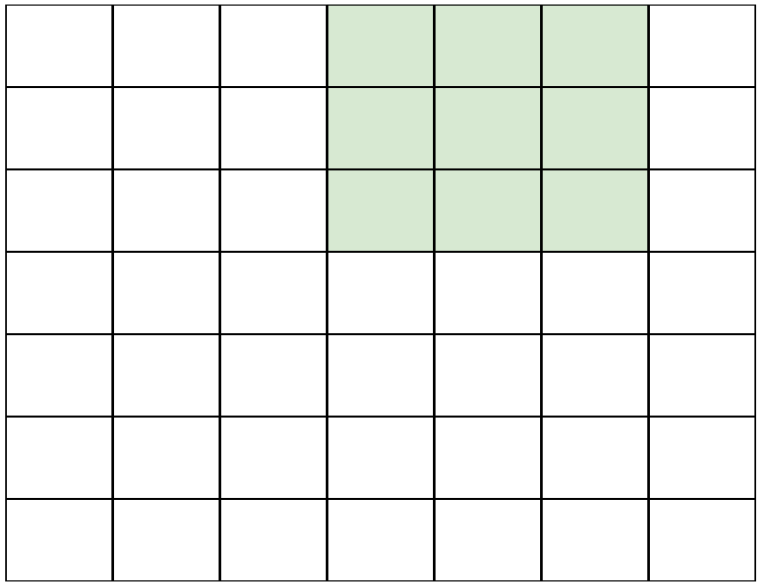
Olhando mais de perto as dimensões:



- Entrada 7x7
- Filtro 3x3

Camada de convolução: detalhes

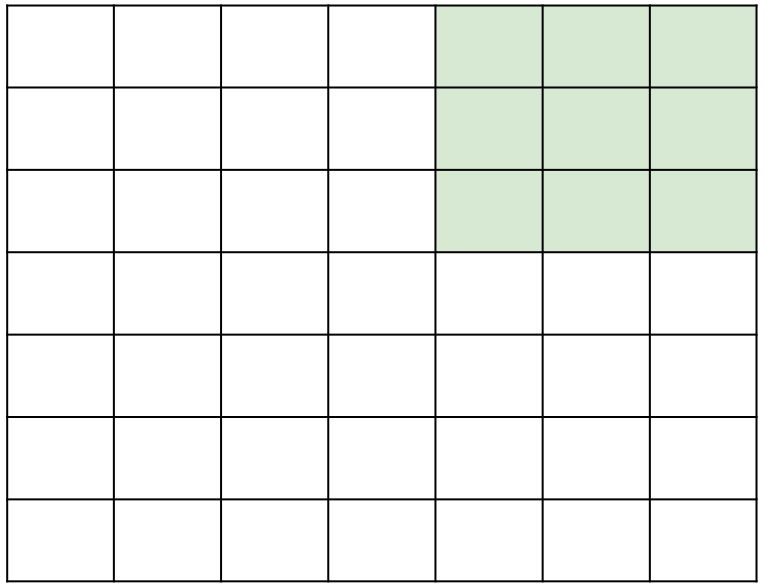
Olhando mais de perto as dimensões:



- Entrada 7x7
- Filtro 3x3

Camada de convolução: detalhes

Olhando mais de perto as dimensões:



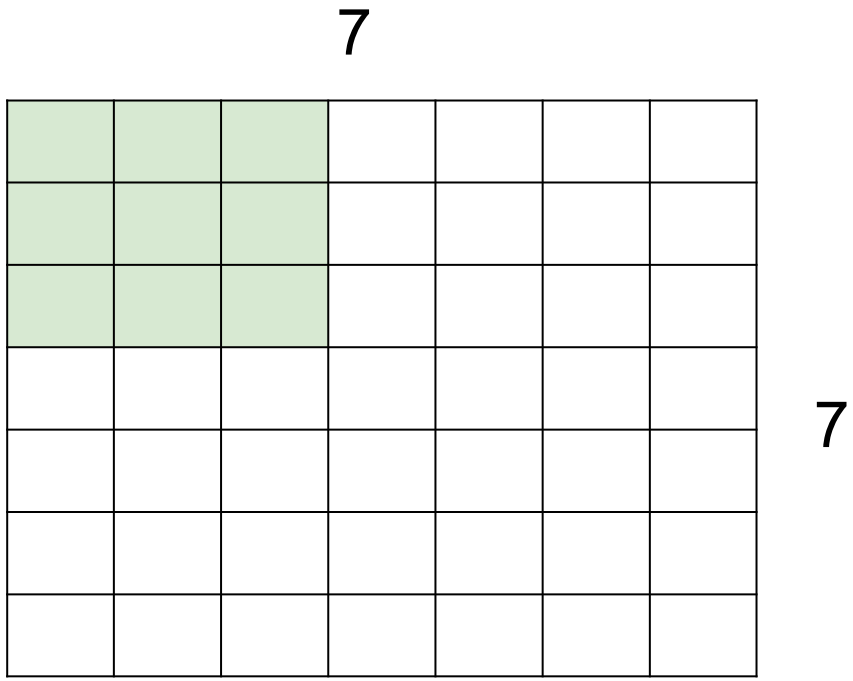
7

- Entrada 7x7
- Filtro 3x3

Matriz de saída: 5x5

Camada de convolução: detalhes

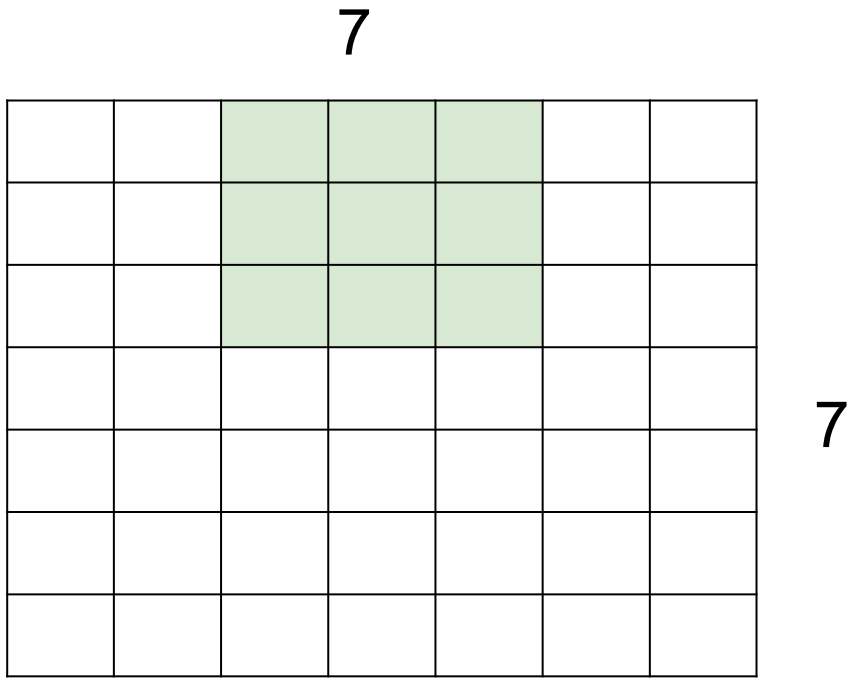
Olhando mais de perto as dimensões:



Entrada 7x7
Filtro 3x3
com stride 2

Camada de convolução: detalhes

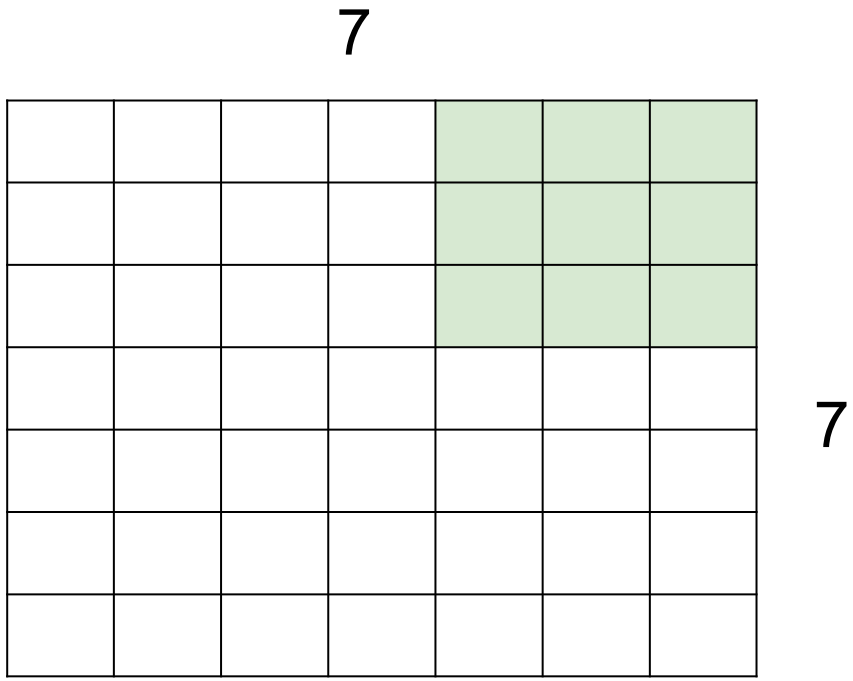
Olhando mais de perto as dimensões:



Entrada 7x7
Filtro 3x3
com stride 2

Camada de convolução: detalhes

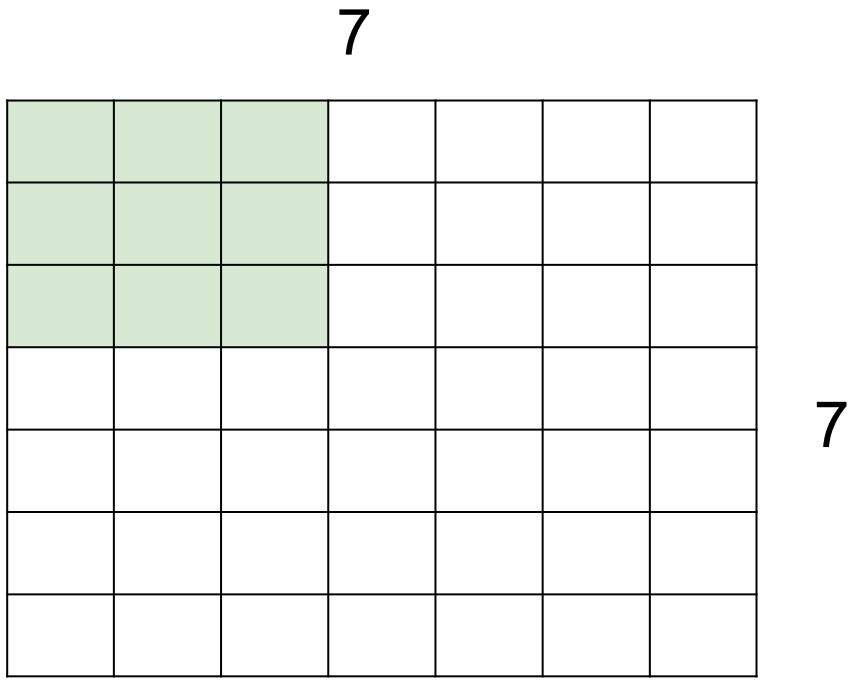
Olhando mais de perto as dimensões:



Entrada 7x7
Filtro 3x3
com stride 2
Saída: 3x3

Camada de convolução: detalhes

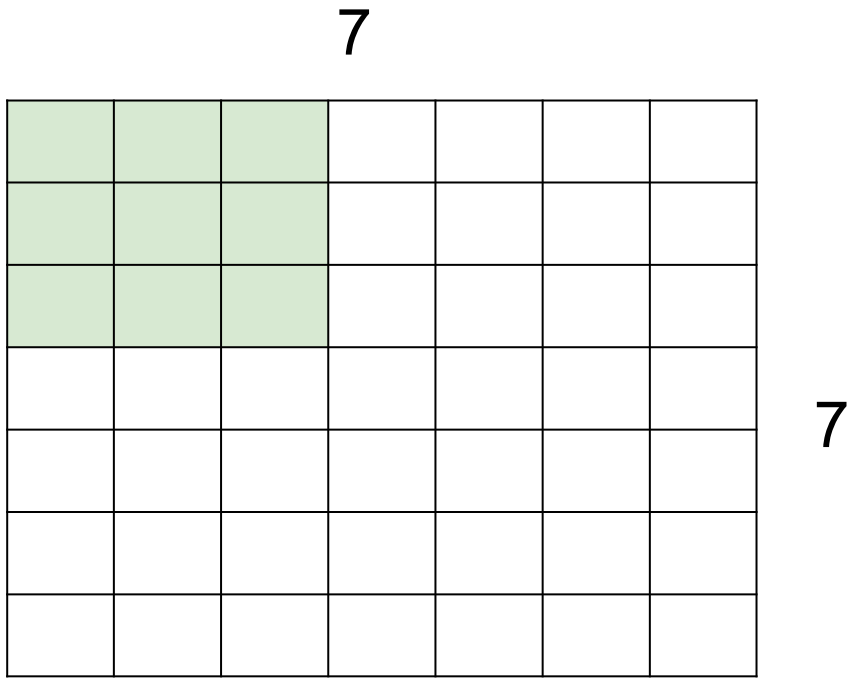
Olhando mais de perto as dimensões:



Entrada 7x7
Filtro 3x3
com stride 3

Camada de convolução: detalhes

Olhando mais de perto as dimensões:

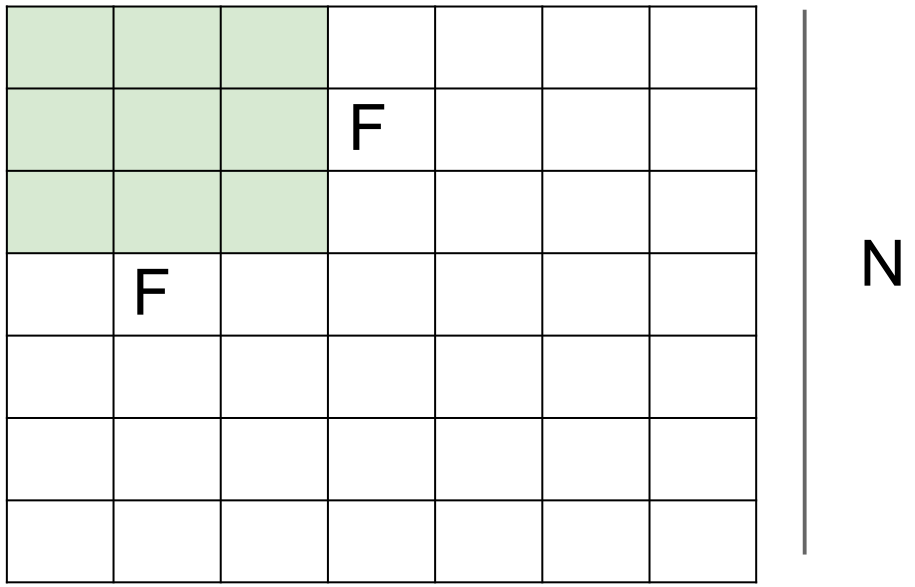


Entrada 7x7
Filtro 3x3
com stride 3

Bibliotecas usualmente não permitem usar um filtro 3x3 em uma imagem 7x7 com stride 3.

Camada de convolução: detalhes

Olhando mais de perto as dimensões:

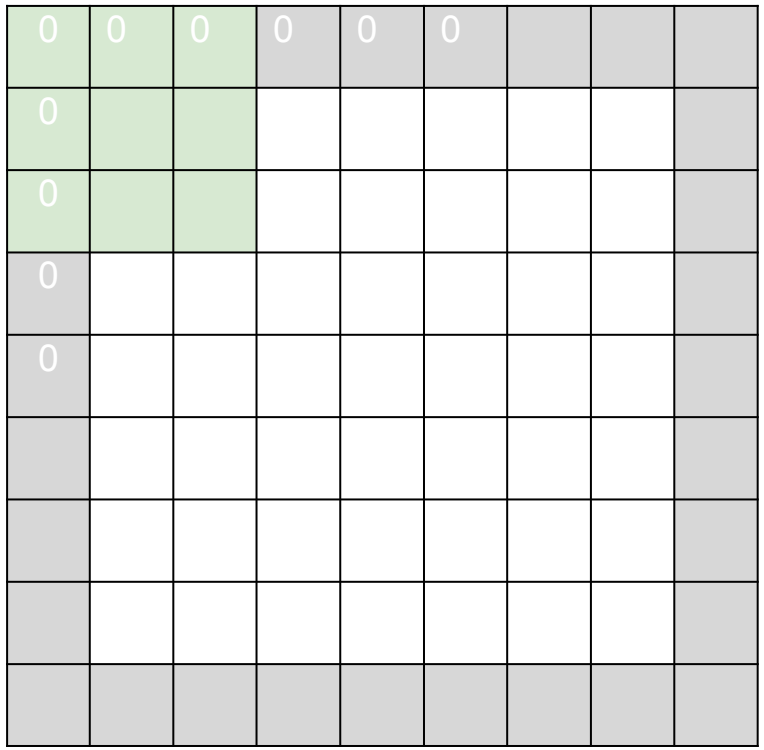


Calculando a saída:
(N - F) / stride + 1

e.g. N = 7, F = 3:
stride 1 => (7 - 3)/1 + 1 = 5
stride 2 => (7 - 3)/2 + 1 = 3
stride 3 => (7 - 3)/3 + 1 = 2.33 :\

Camada de convolução: detalhes

Na prática, costuma-se usar zero na borda

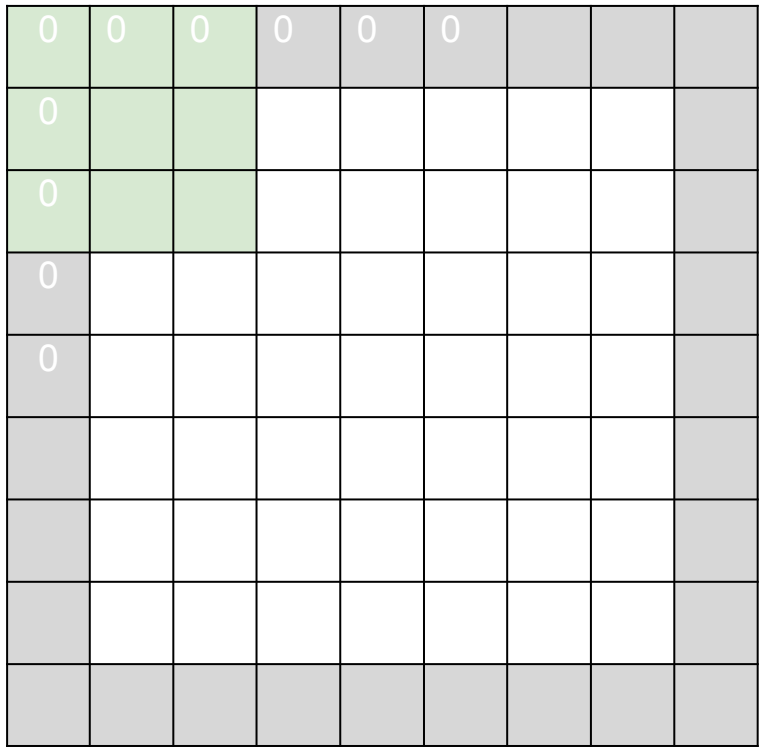


e.g. Entrada 7x7
Filtro 3x3 com **stride 1**
padding = 1 pixel de borda
=> Qual é a saída?

(N - F) / stride + 1

Camada de convolução: detalhes

Na prática, costuma-se usar zero na borda

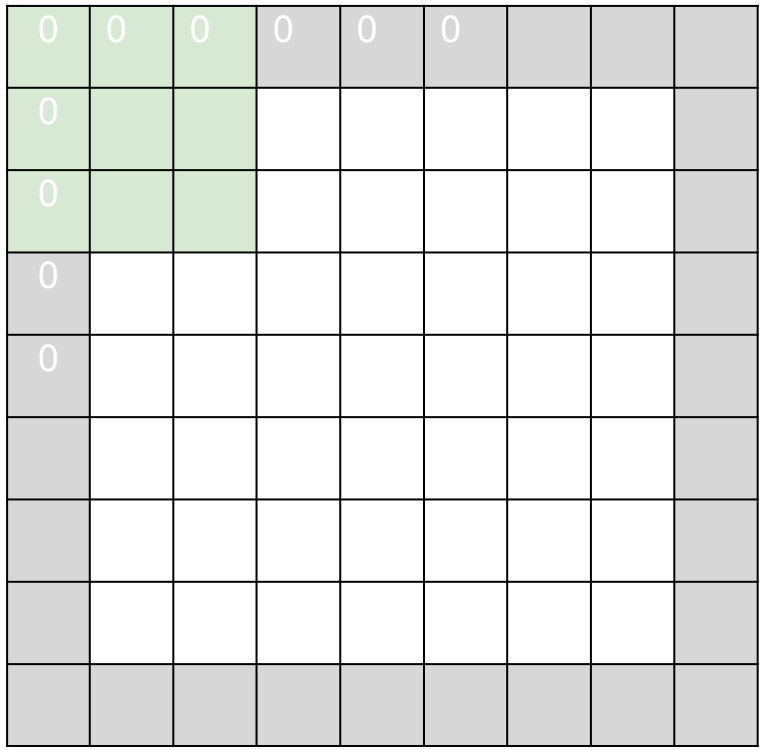


e.g. Entrada 7x7
Filtro 3x3 com **stride 1**
padding = 1 pixel de borda
=> Qual é a saída?

saída 7x7

Camada de convolução: detalhes

Na prática, costuma-se usar zero na borda



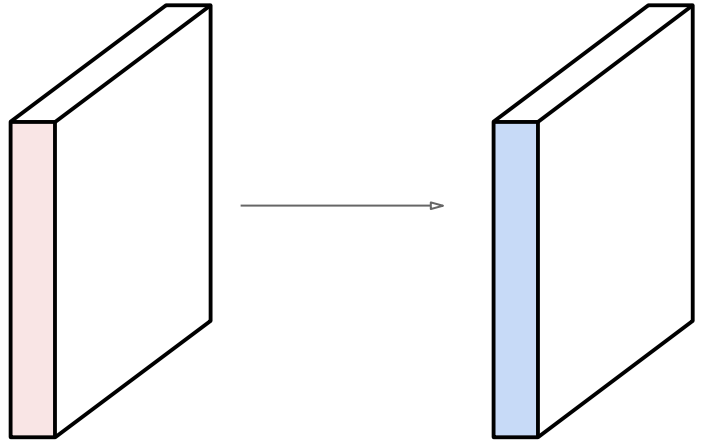
e.g. Entrada 7x7
Filtro 3x3 com **stride 1**
padding = 1 pixel de borda
=> Qual é a saída?

7x7

(N + 2*padding - F) / stride + 1

Camada de convolução: detalhes

Exemplos:

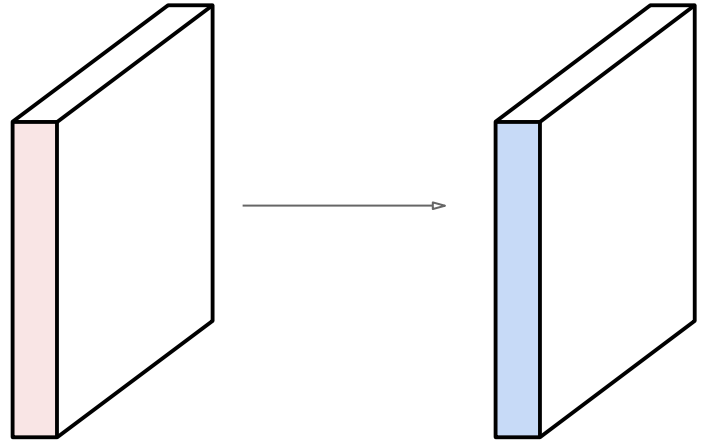


Entrada: **32x32x3**
10 filtros 5x5 com stride 1, pad 2

Dimensão da saída: ?

Camada de convolução: detalhes

Exemplos:



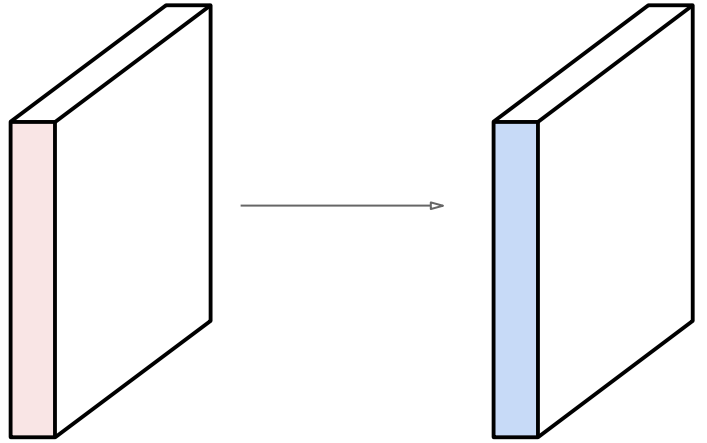
Entrada: **32x32x3**
10 filtros 5x5 com stride 1, pad 2

Dimensão da saída:
(32+2*2-5)/1+1 = 32

(N + 2*padding - F) / stride + 1

Logo: **32x32x10**

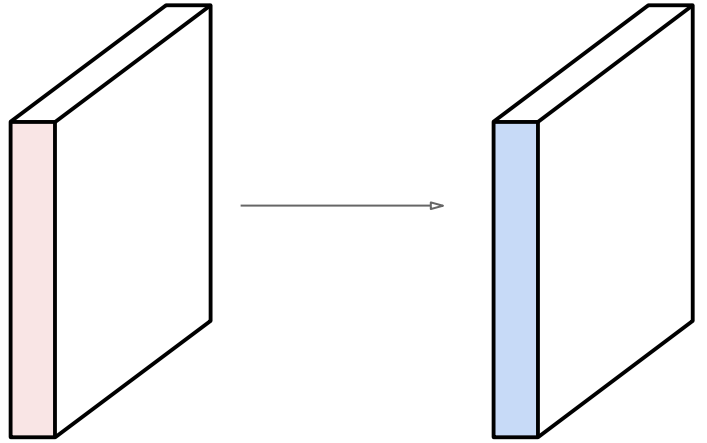
Camada de convolução: detalhes



Entrada: **32x32x3**
10 filtros 5x5 com stride 1, pad 2

Número de parâmetros na camada?

Camada de convolução: detalhes



Entrada: **32x32x3**
10 filtros 5x5 com stride 1, padding 2

Cada filtro possui 5*5*3 + 1 = 76 parâmetros (+1 para o bias)
=> 76*10 = **760**

Camada de convolução: detalhes

Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
  - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.

Camada de convolução: implementação

Camada de convolução com
  # 32 filtros
  # kernel de tamanho 3x3
  # padding = ‘same’ -> mantendo a saída (mapas de ativação) c/ mesma dimensão da entrada
  # input_shape(28,28,3) -> três matrizes 28x28
  # strides(1,1) -> “pulos” na horizontal e vertical

from tf.keras.layers import Conv2D

Sequential()

model.add(Conv2D(filters=32, kernel_size=(3, 3), activation="relu", kernel_initializer="he_uniform", padding='same', strides(1,1) input_shape=(28,28,3)))

https://keras.io/api/layers/convolution_layers/convolution2d/

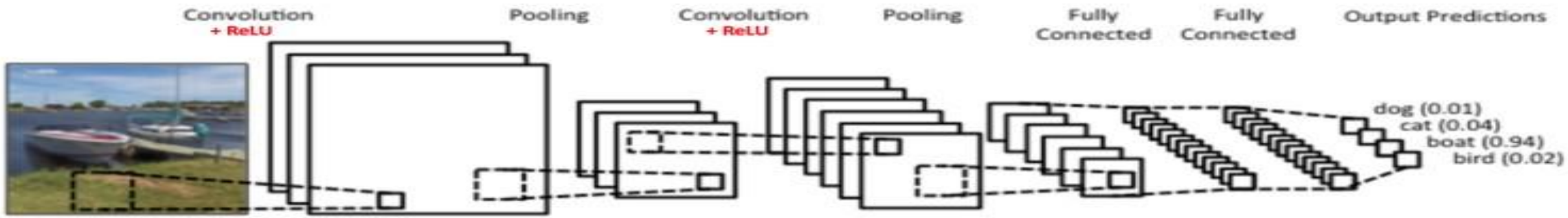
```
Importante -> as entradas para Conv2D são matrizes 3d.
No nosso caso:
X_train_full=X_train_full.reshape((-1, 28, 28, 1))
X_test=X_test.reshape((-1, 28, 28, 1))
```

Camada de convolução: Visualizações

Visualizações de parâmetros dos filtros convolucionais:

https://github.com/vdumoulin/conv_arithmetic

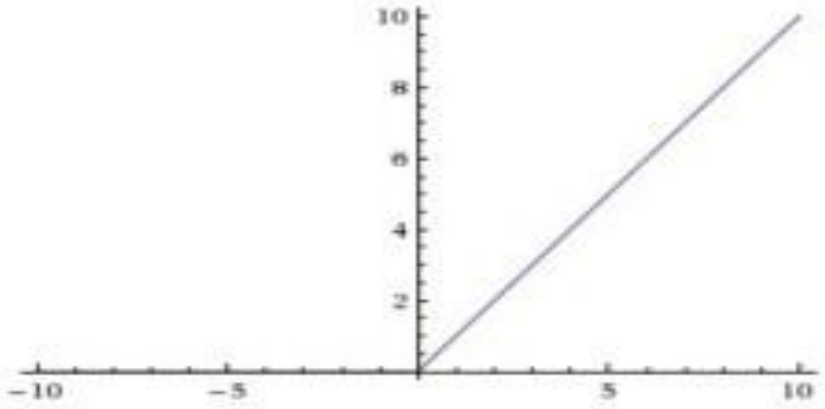
Camada de convolução: função de ativação



Camada de convolução: função de ativação

- Rectified Linear Units (ReLU)
- ReLU (e suas variações)
  - Normalmente, dentre as mais usadas em redes de convolução

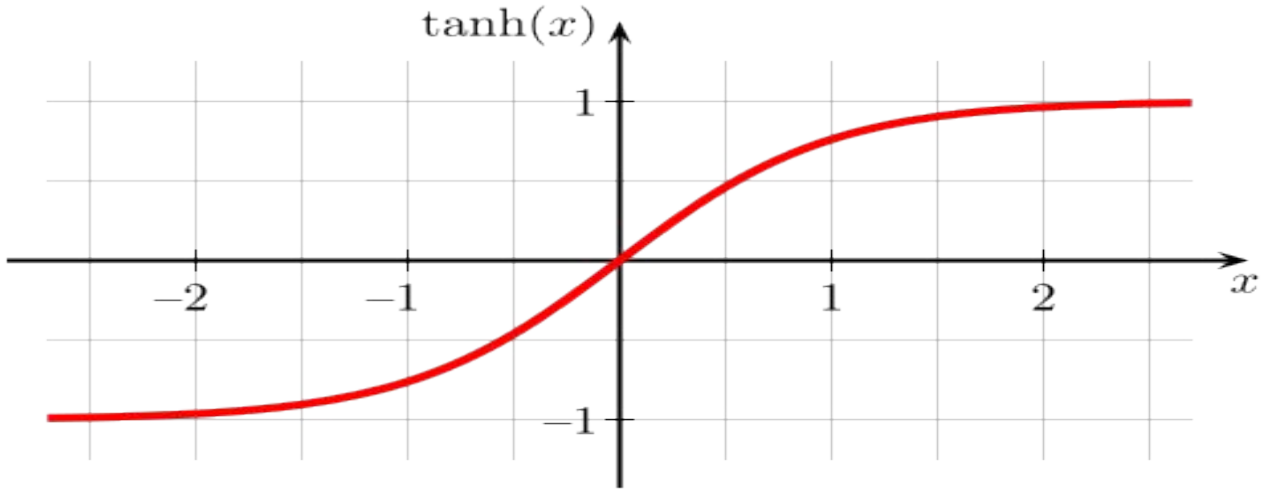
$$\text{Output} = \text{Max}(\text{zero}, \text{Input})$$



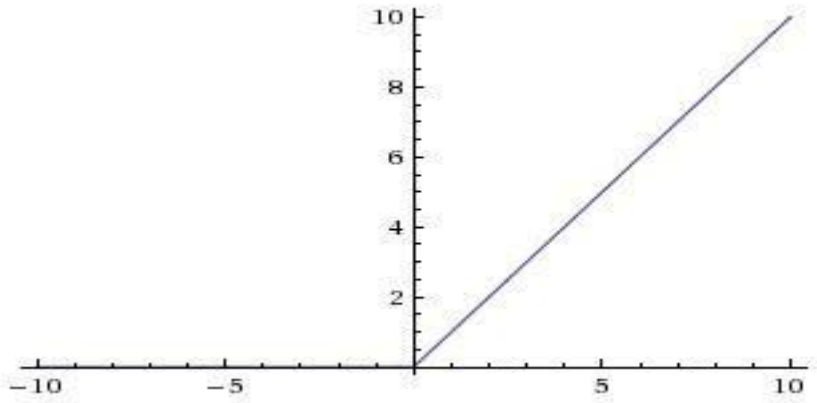
Camada de convolução: função de ativação

- Porque não outra, como a Tanh?
  - Saturação dos gradientes
    - Difícil treinar com dados ruidosos/reais.
    - Múltiplas camadas intensificam o problema.
- ReLU garante um gradiente constante (evita saturação).
  - Alterações dos pesos mais “grosseiras” no backpropagation.
    - Intuição: Dados ruidosos -> alterações grosseiras.

Tanh(x)

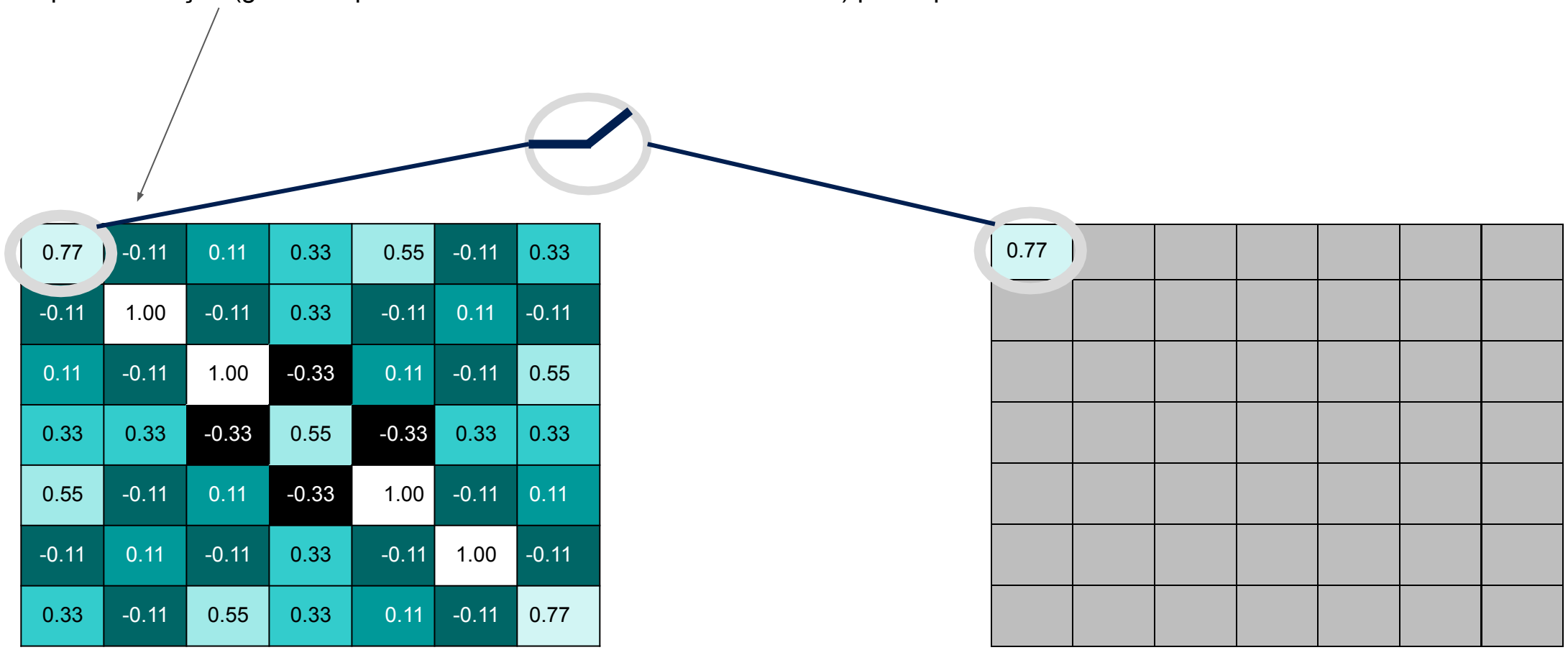


ReLU



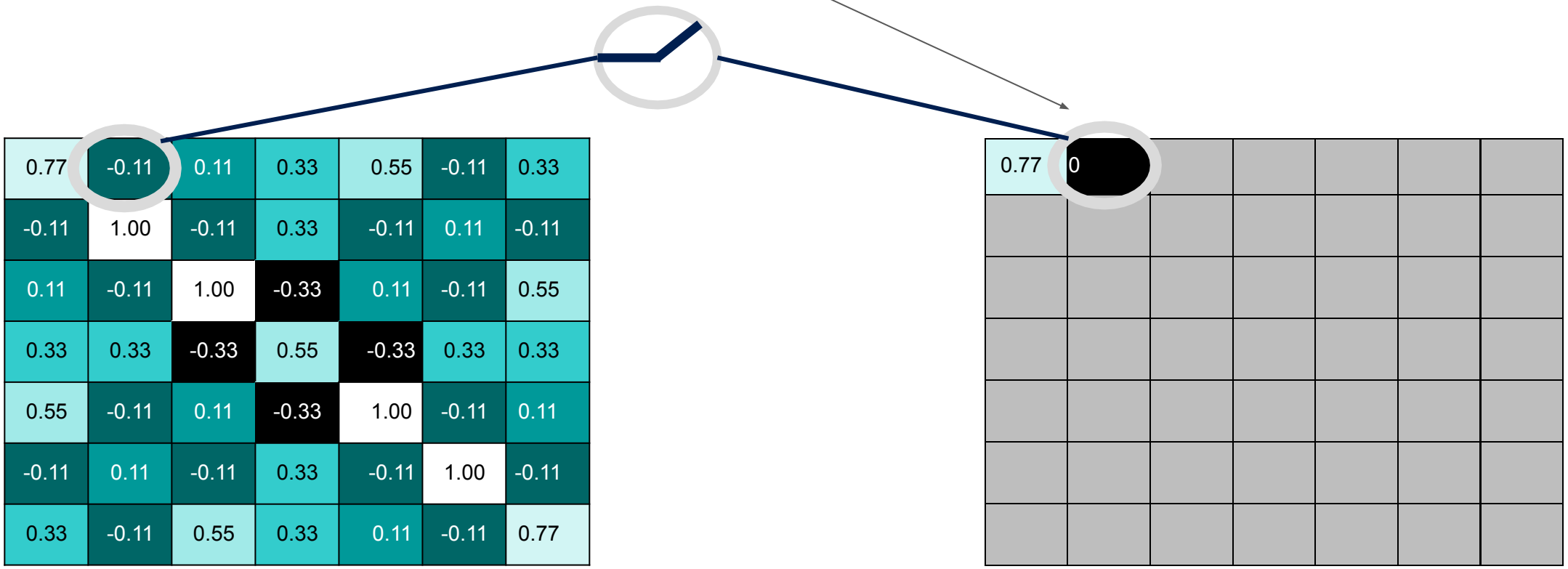
Camada de convolução: função de ativação

Utilizo o mapa de ativação (gerado a partir de uma entrada e um filtro treinado) para aplicar a relu

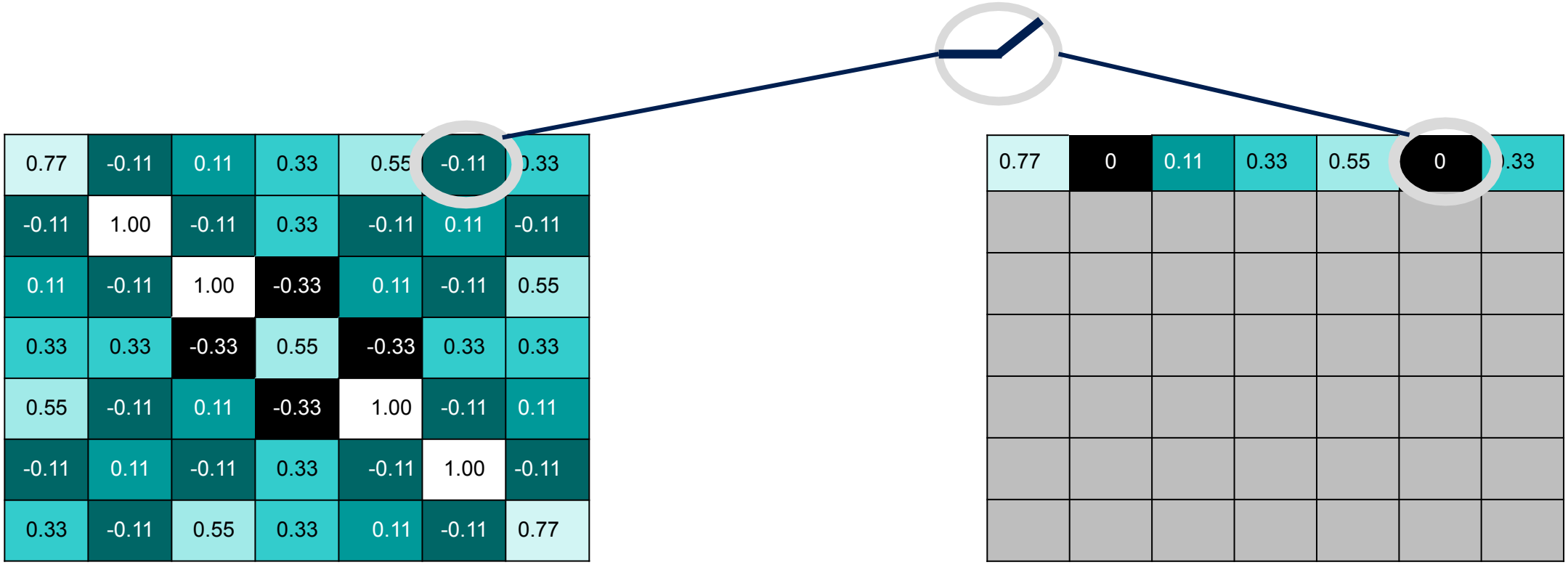


Camada de convolução: função de ativação

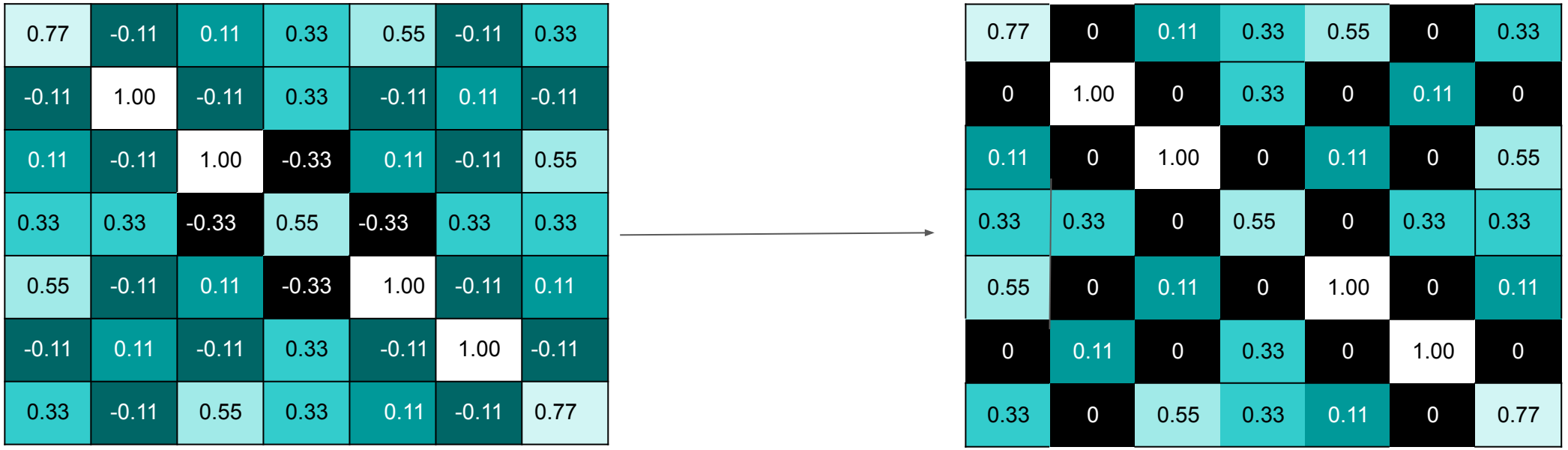
Regiões do mapa que não são importantes ficam com o valor 0



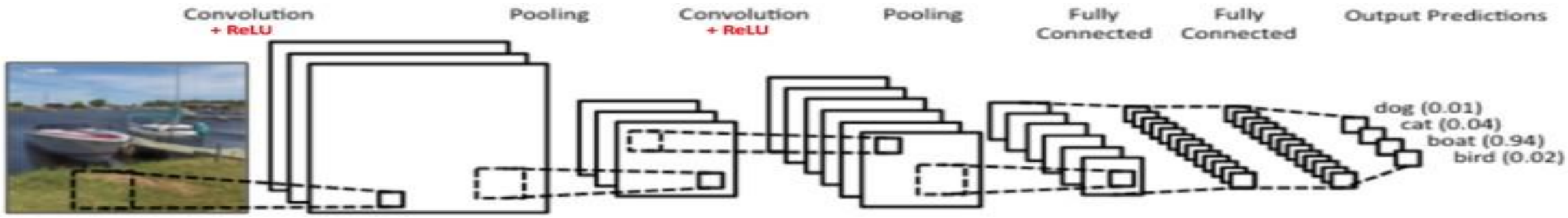
Camada de convolução: função de ativação



Camada de convolução: função de ativação



Camada de Pooling



Camada de Pooling

- Denominada “downsample”
  - “Reduz” a entrada por amostragem.
  - Opções:
    - Min-pooling
    - Max-pooling
    - Average-pooling

- Intuição:
  - Em geral, podemos ignorar pequenos detalhes da entrada
  - … e reduzir a quantidade de parâmetros.

Camada de Pooling

- Denominada “downsample”
  - “Reduz” a entrada por amostragem.
  - Opções:
    - Min-pooling
    - Max-pooling
    - Average-pooling

- Pooling mitiga problemas de translação/janelamento e rotação.

> *“In all cases, pooling helps to make the representation become approximately invariant to small translations of the input. Invariance to translation means that if we translate the input by a small amount, the values of most of the pooled outputs do not change.”*
>
> — p. 342, Ian Goodfellow. Deep Learning

Single depth slice

x

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

y

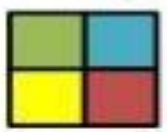
max pool with 2x2 filters
and stride 2

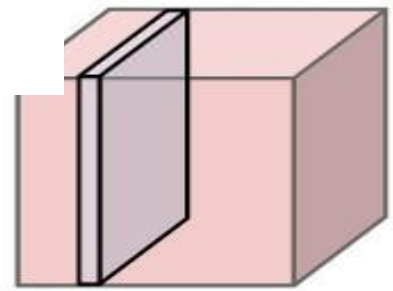
6	8
3	4

Max pooling

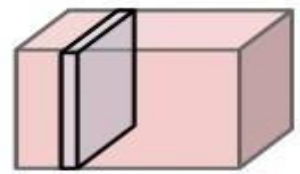


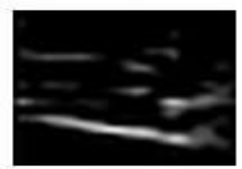
Max in a 2x2
filter





pool





downsampling

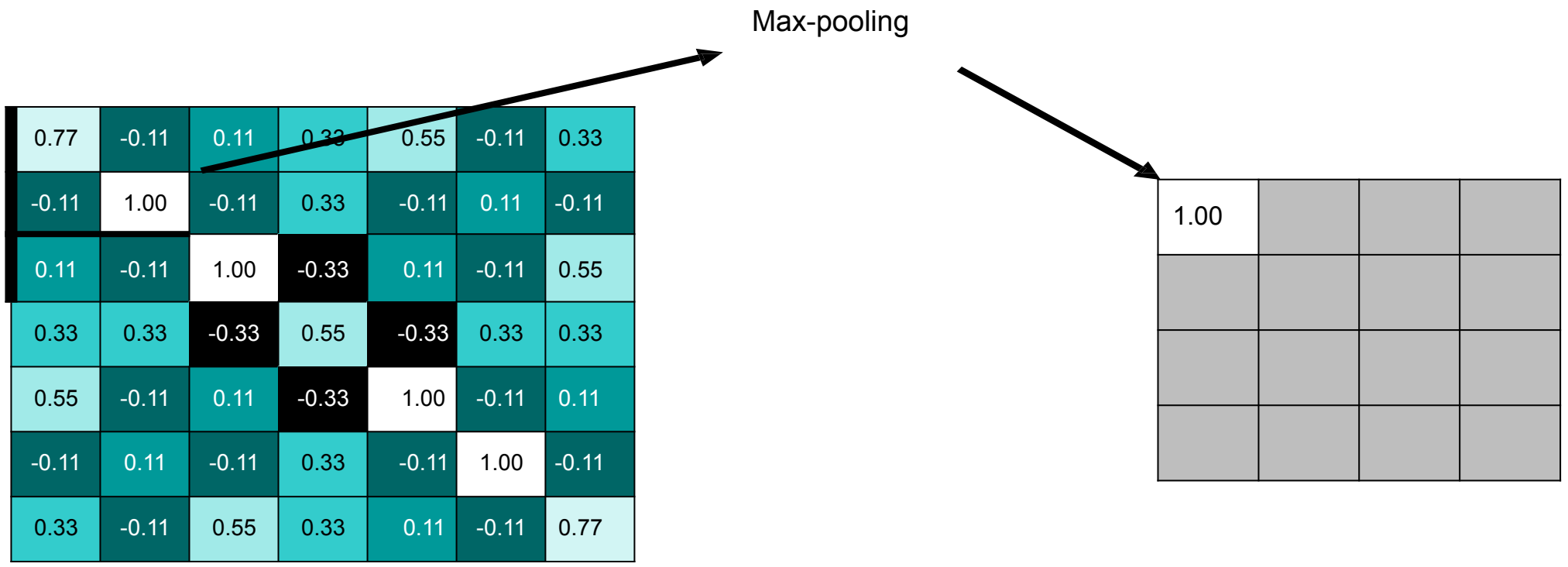


Algoritmo de Pooling

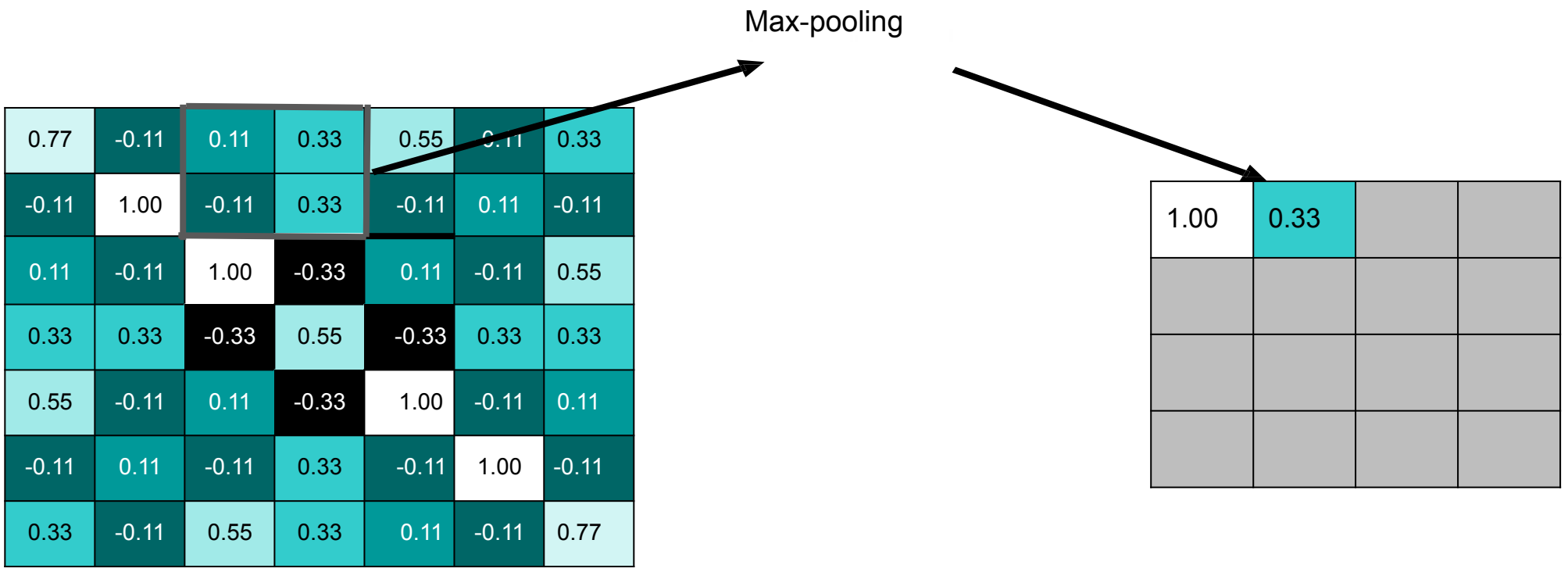
1. Escolha um tamanho de janela (ex. 3x3).
2. Caminhe sua janela com os strides (pulos).
3. Para cada Janela, pegue o valor máximo/mínimo/médio

```
model.add(keras.layers.MaxPooling2D(pool_size = (3,3)))
```

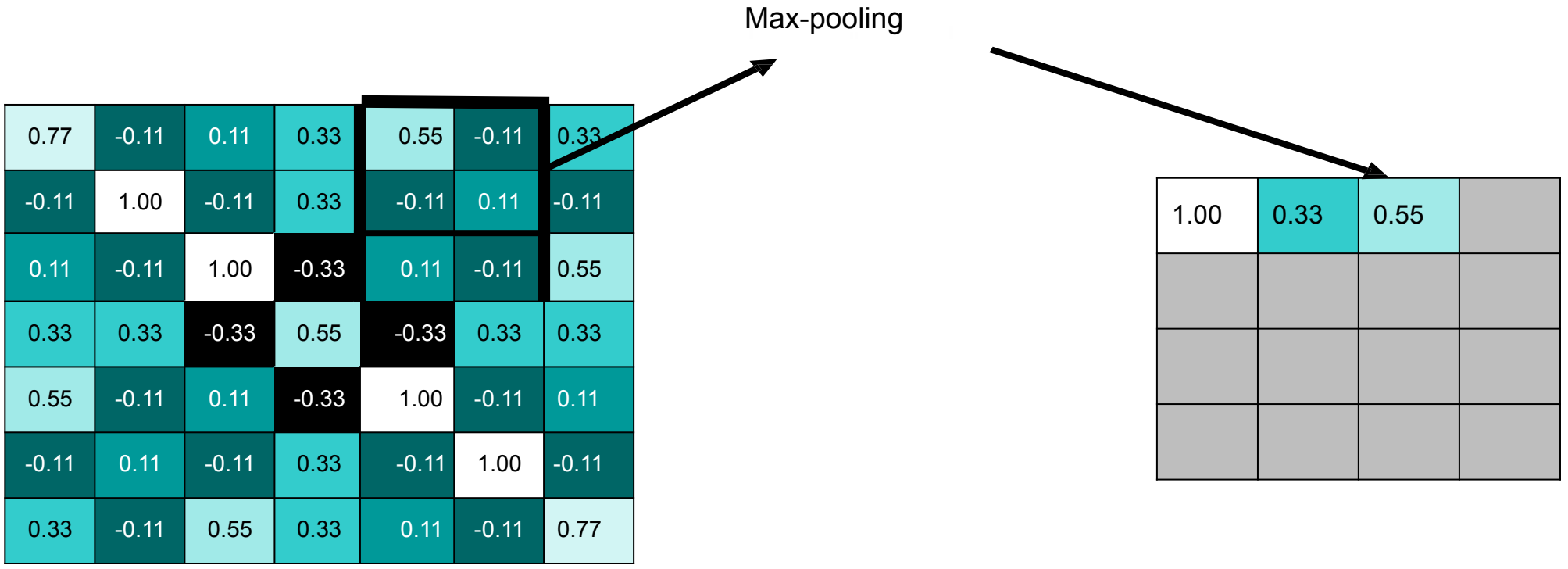
Pooling


Max-pooling
0.77 -0.11 0.11 0.33 0.55 -0.11 0.33
-0.11 1.00 -0.11 0.33 -0.11 0.11 -0.11
0.11 -0.11 1.00 -0.33 0.11 -0.11 0.55
0.33 0.33 -0.33 0.55 -0.33 0.33 0.33
0.55 -0.11 0.11 -0.33 1.00 -0.11 0.11
-0.11 0.11 -0.11 0.33 -0.11 1.00 -0.11
0.33 -0.11 0.55 0.33 0.11 -0.11 0.77
1.00

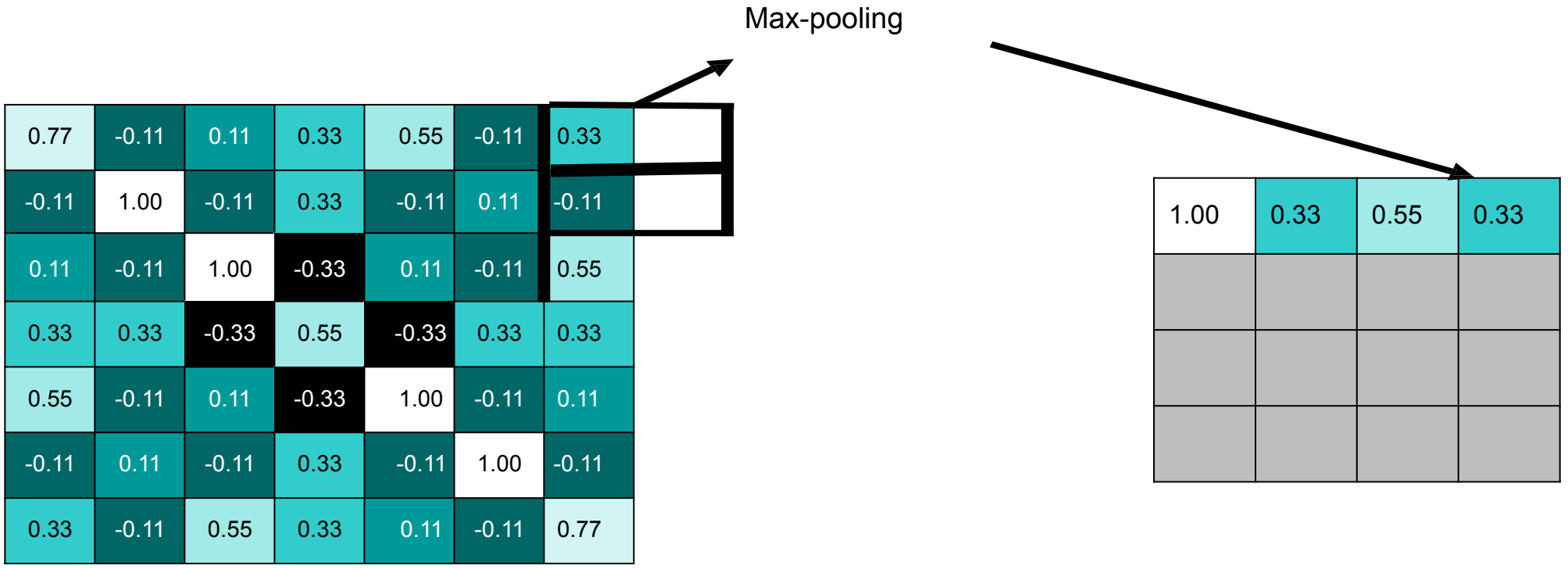
Pooling


Max-pooling
0.77 -0.11 0.11 0.33 0.55 0.11 0.33
-0.11 1.00 -0.11 0.33 -0.11 0.11 -0.11
0.11 -0.11 1.00 -0.33 0.11 -0.11 0.55
0.33 0.33 -0.33 0.55 -0.33 0.33 0.33
0.55 -0.11 0.11 -0.33 1.00 -0.11 0.11
-0.11 0.11 -0.11 0.33 -0.11 1.00 -0.11
0.33 -0.11 0.55 0.33 0.11 -0.11 0.77
1.00 0.33

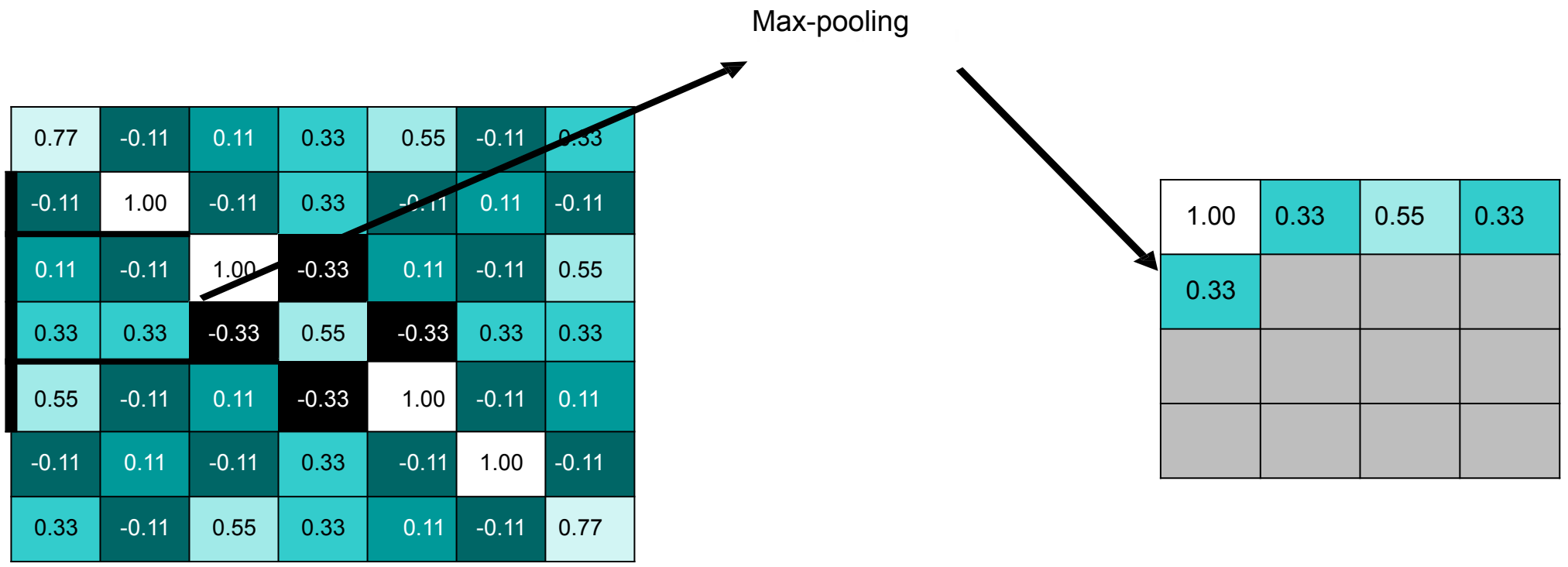
Pooling


Max-pooling
0.77 -0.11 0.11 0.33 0.55 -0.11 0.33
-0.11 1.00 -0.11 0.33 -0.11 0.11 -0.11
0.11 -0.11 1.00 -0.33 0.11 -0.11 0.55
0.33 0.33 -0.33 0.55 -0.33 0.33 0.33
0.55 -0.11 0.11 -0.33 1.00 -0.11 0.11
-0.11 0.11 -0.11 0.33 -0.11 1.00 -0.11
0.33 -0.11 0.55 0.33 0.11 -0.11 0.77
1.00 0.33 0.55

Pooling


Max-pooling
0.77 -0.11 0.11 0.33 0.55 -0.11 0.33
-0.11 1.00 -0.11 0.33 -0.11 0.11 -0.11
0.11 -0.11 1.00 -0.33 0.11 -0.11 0.55
0.33 0.33 -0.33 0.55 -0.33 0.33 0.33
0.55 -0.11 0.11 -0.33 1.00 -0.11 0.11
-0.11 0.11 -0.11 0.33 -0.11 1.00 -0.11
0.33 -0.11 0.55 0.33 0.11 -0.11 0.77
1.00 0.33 0.55 0.33

Pooling


Max-pooling
0.77 -0.11 0.11 0.33 0.55 -0.11 0.33
-0.11 1.00 -0.11 0.33 -0.11 0.11 -0.11
0.11 -0.11 1.00 -0.33 0.11 -0.11 0.55
0.33 0.33 -0.33 0.55 -0.33 0.33 0.33
0.55 -0.11 0.11 -0.33 1.00 -0.11 0.11
-0.11 0.11 -0.11 0.33 -0.11 1.00 -0.11
0.33 -0.11 0.55 0.33 0.11 -0.11 0.77
1.00 0.33 0.55 0.33
0.33

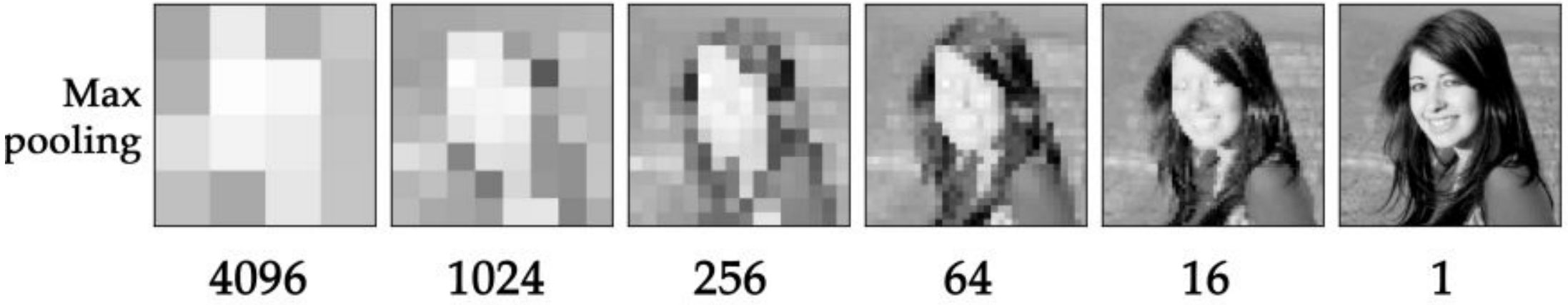
Pooling

0.77	-0.11	0.11	0.33	0.55	-0.11	0.33
-0.11	1.00	-0.11	0.33	-0.11	0.11	-0.11
0.11	-0.11	1.00	-0.33	0.11	-0.11	0.55
0.33	0.33	-0.33	0.55	-0.33	0.33	0.33
0.55	-0.11	0.11	-0.33	1.00	-0.11	0.11
-0.11	0.11	-0.11	0.33	-0.11	1.00	-0.11
0.33	-0.11	0.55	0.33	0.11	-0.11	0.77



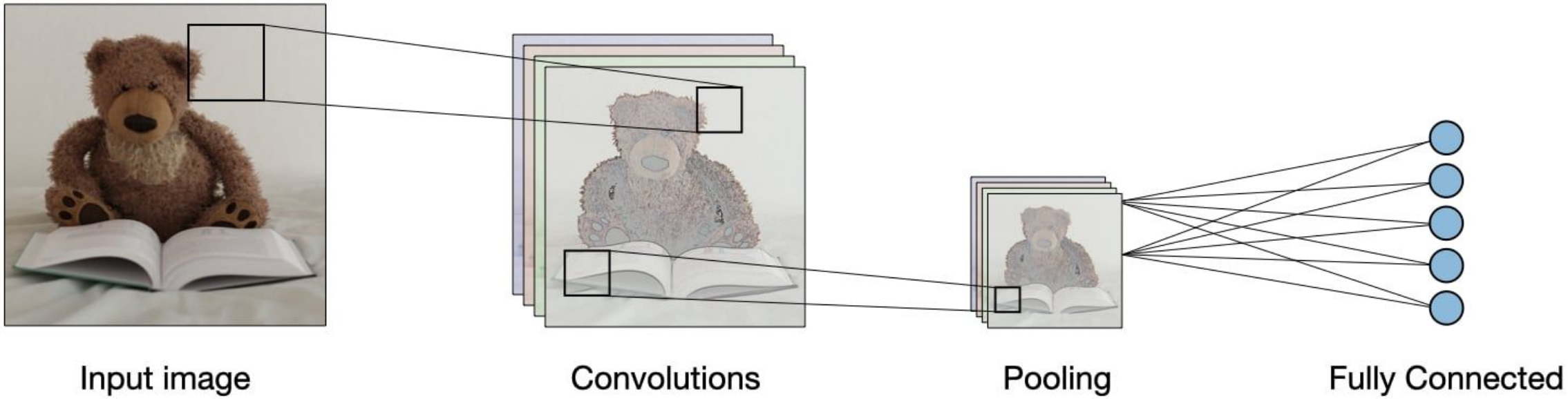
1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

Pooling

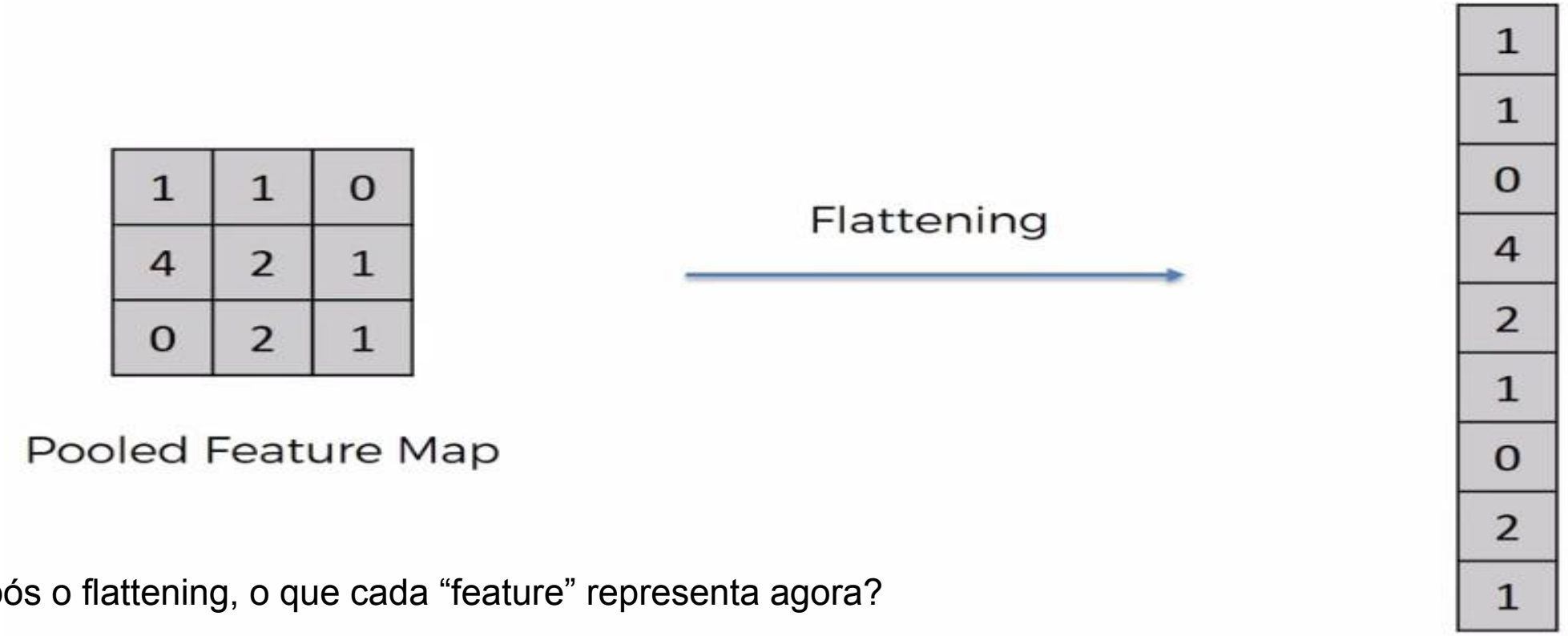


O. Rippel. Spectral Representations for Convolutional Neural Networks

Pooling



Fully connected layer



Após o flattening, o que cada “feature” representa agora?

Fully connected layer

1	1	0
4	2	1
0	2	1

Pooled Feature Map

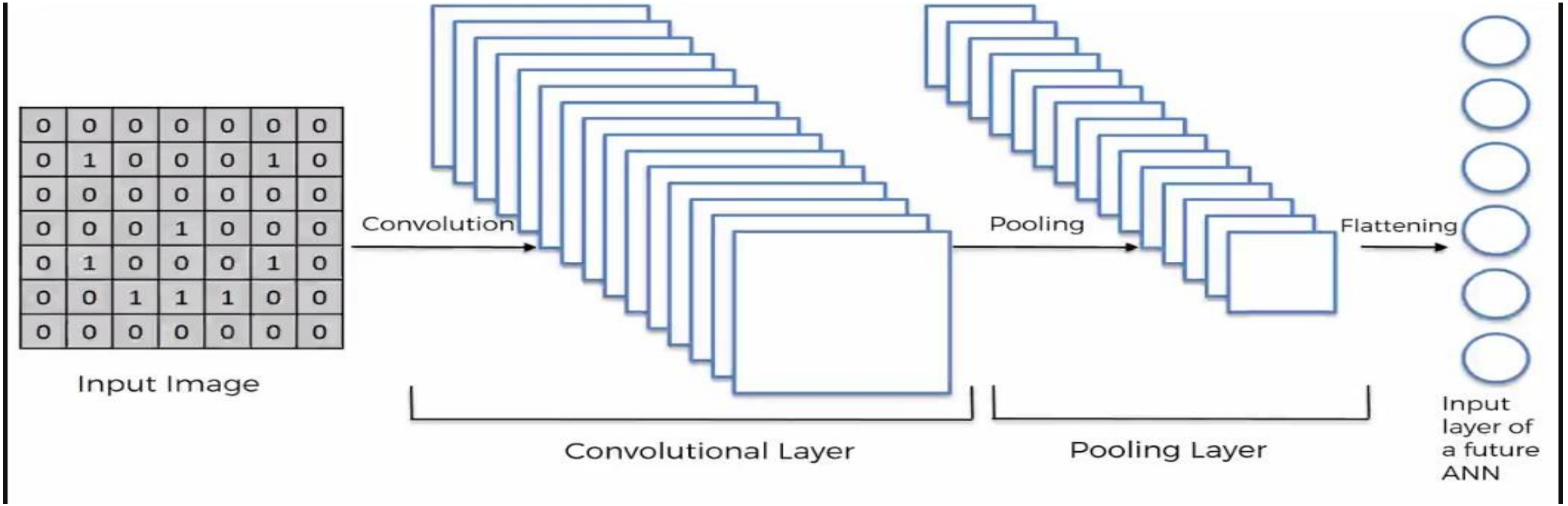


1
1
0
4
2
1
0
2
1

Após o flattening, o que cada “feature” representa agora?

- Presença/ausência de um padrão mais complexo (representada pela última camada de convolução) na imagem.

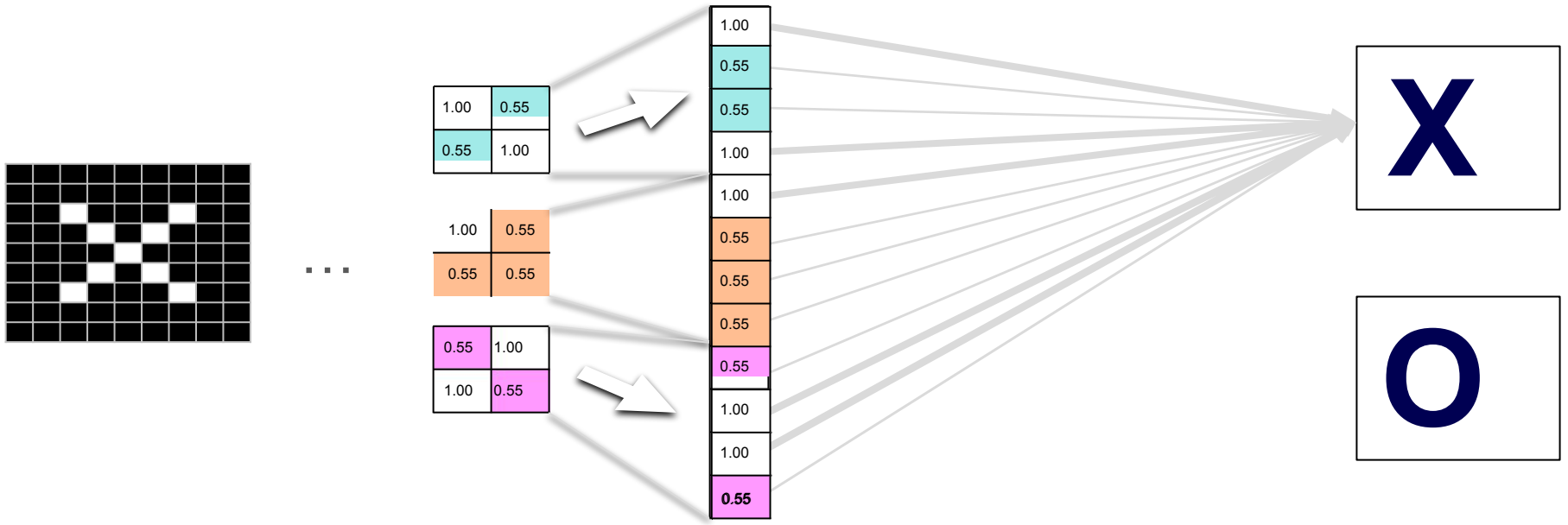
Fully connected layer



Fully connected layer

Para cada valor do pooling:

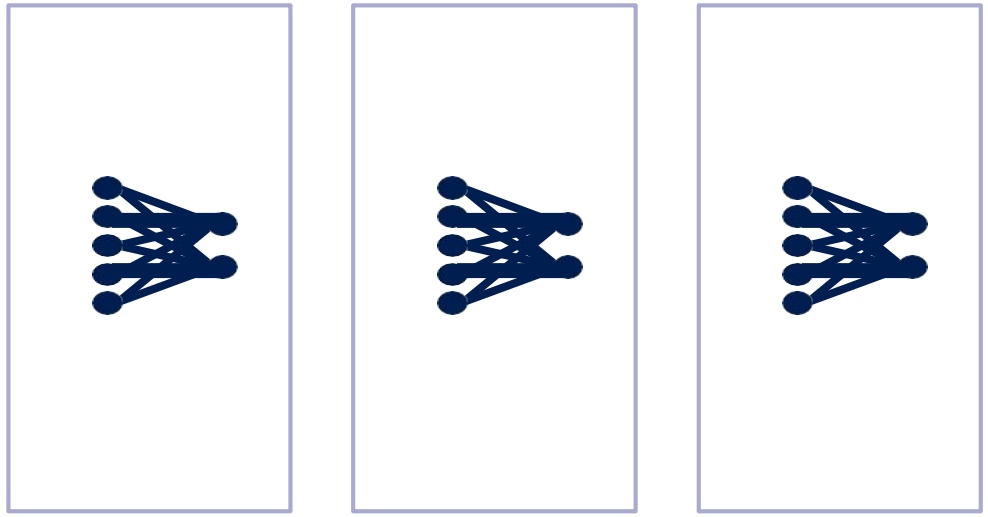
- Valor de entrada para os neurônios de saída



Fully connected layer

Possibilidade de adicionar outras camadas ocultas.

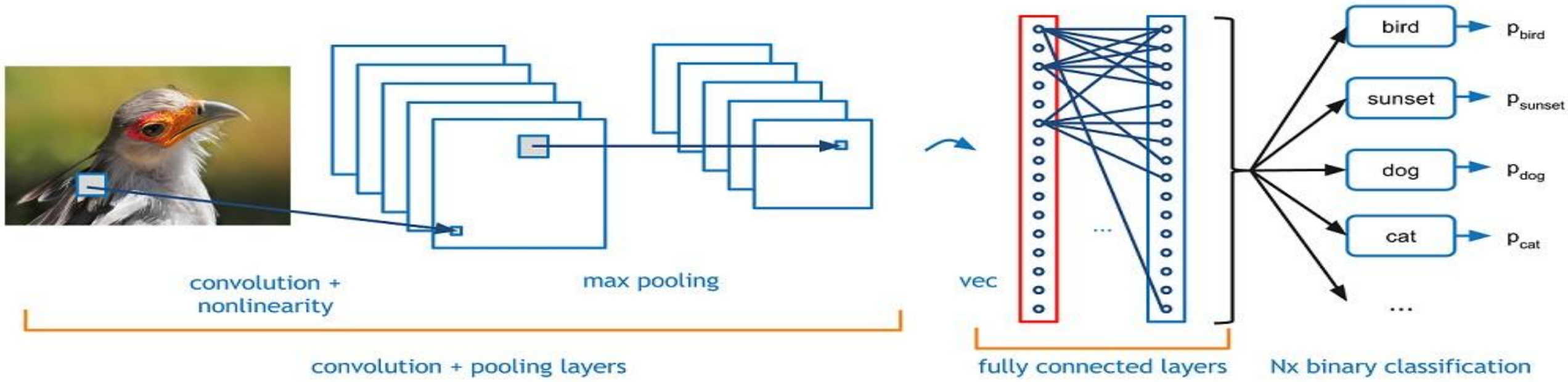
0.9
0.65
0.45
0.87
0.96
0.73
0.23
0.63
0.44
0.89
0.94
0.53



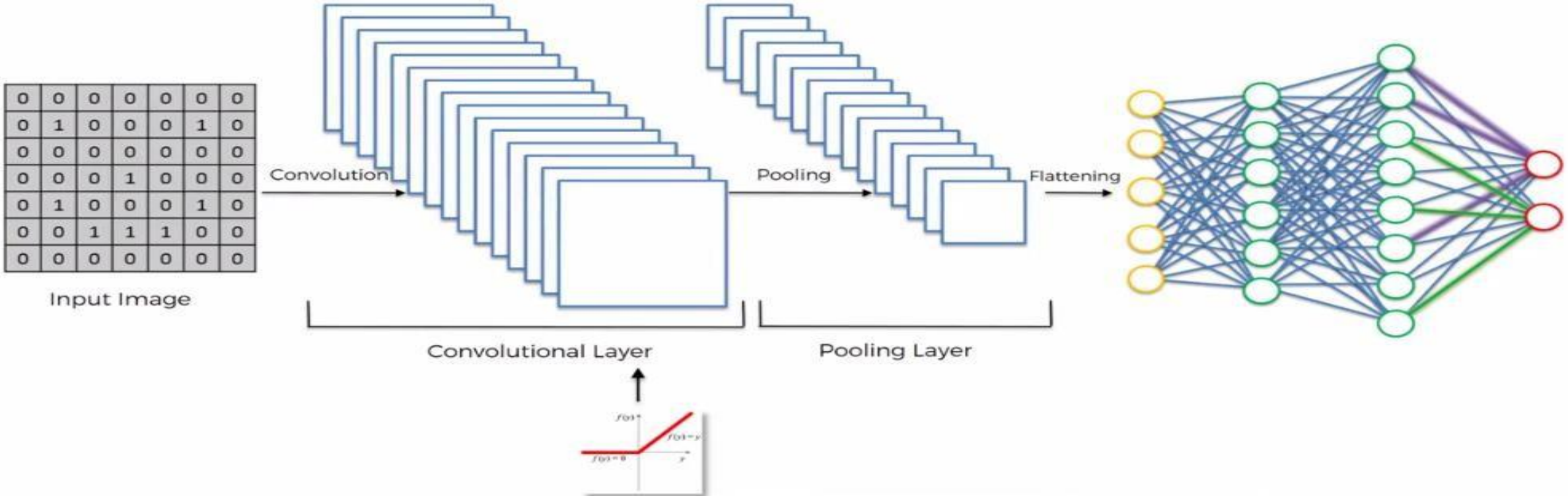
X

O

Fully connected layer



Fully connected layer



Atividades

1) Adapte o código da última aula (fashion_parametrizacao_aula_original.ipynb) para lidar com convoluções utilizando a arquitetura de convolução lecun, (descrito em https://www.kaggle.com/code/curiousprogrammer/lenet-5-cnn-with-keras-99-48/notebook).
2) Liste a quantidade de parâmetros treináveis obtidos em cada camada da arquitetura de convolução obtida no exercício (1), por meio da linha "model.summary()" do código. Explique como o Keras obteve tais quantificações de parâmetros.

Referências

- François Chollet. Deep Learning with Python. Manning publications. 2019
- Aurélien Géron. Neural networks and deep learning https://www.oreilly.com/library/view/neural-networks-and/9781492037354/ch01.html

- Interessantes:
  - https://github.com/vdumoulin/conv_arithmetic
  - Interessante - Pedro Domingo “Deep Networks Are Kernel Machines” https://www.youtube.com/watch?v=m3b0qEQHlUs
  - https://youtube.com/watch?v=AQirPKrAyDg -> Aula de Stanford sobre CNNs
  - https://www.kaggle.com/code/curiousprogrammer/lenet-5-cnn-with-keras-99-48/notebook